

GRID TRADING SERIES · หนังสือเล่มที่ 8

Grid Trading Mastery

จาก Grid พื้นฐาน สู่ Grid ระดับ Quant

ครอบคลุมทุกมิติ: Close System, Zone Tricks, Soft Martingale, Bloating Pattern, Volatility-Adaptive Sizing, Regime Detection, Pair Grid, Grid Snowball และ Grid-Framework + Indicator Execution — ประยุกต์ความรู้จากทุกเล่มในซีรีส์

16 ตอน + ภาคผนวก · ~40 ชั่วโมงเนื้อหา · ตัวอย่าง BTC/USDT Bybit ตลอดเล่ม

ปรัชญาหลักก่อนเริ่มอ่าน

Grid คือ Grid — Enhancement ไม่ใช่ Magic

ศัพท์ที่ใช้ในกล่องนี้ (อธิบายละเอียดเต็มใน Part 0): **alpha** = กำไรส่วนเกินที่มาจากความได้เปรียบจริง ไม่ใช่แค่โชคดีหรือถ้อ
เฉยๆ · **H (Hurst Exponent)** = ตัววัดว่าราคามีแนวโน้ม "กลับสู่ค่ากลาง" แค่ไหน ($H < 0.5$ = mean-reverting) · **DD**
(Drawdown) = ขนาดการขาดทุนจากจุดสูงสุด · **Sharpe (ratio)** = ผลตอบแทนต่อหน่วยความเสี่ยง ยิ่งสูงยิ่งดี

Grid trading framework มี alpha จริง **เฉพาะในตลาดที่ mean-revert จริง** ($H < 0.5$ — ในตลาด
สุมแท่ง grid เปล่ามี expected value = 0 พอดี ดู Part 0 §0.4) + บังคับวินัยการซื้อขาย — แต่ทุก
enhancement (indicator, zone trick, martingale) ช่วยปรับ **คุณภาพของ equity curve** มากกว่า
เพิ่ม total return อย่างมีนัย

**ในตลาดที่เหมาะสม แม้อยินหัวก้อยเพื่อเลือกว่าจะเปิด order หรือไม่ — Grid framework ก็ยัง
สร้างกำไรได้ใกล้เคียงเดิม** เพราะ alpha มาจากการดักจับ mean-reversion ของสแนมนั้น ไม่ได้มา
จาก indicator ที่ใส่เพิ่ม Indicator ช่วยลด DD และทำให้ Sharpe ดีขึ้น — ไม่ใช่สร้าง alpha ใหม่จาก
อากาศ

สารบัญรวม

รากฐาน (Foundation)

Part 0	รู้จัก Grid Trading + ปรัชญา — กลไก, ประเภท, เปรียบเทียบ, ปรัชญา	~40 นาที	พื้นฐาน
Part 1A	Close System + Zone Tricks + Soft Martingale — ระบบปิด, Zone Waterfall, Bell Curve, Front-Loading, r-parameter	~2.5 ชม.	กลาง
Part 1B	Buy-and-Sell Grid (Bidirectional) — สองทาง, Long/Short Layer, Capital Allocation ทั้งสองฝั่ง	~2.5 ชม.	กลาง
Part 1C	Grid Hedge (Spot-Perp) — ป้องกัน inventory risk ด้วย perp short, delta-neutral grid	~2.25 ชม.	กลาง
Part 1D	Asymmetric Hybrid Grid — Buy-only core + Dynamic Hedge + Capped Short Layer, ทางสายกลางระหว่าง 1A กับ 1B/1C	~1.5 ชม.	กลาง

Zone & Sizing (จัดการ Zone และขนาด)

Part 2	Bloating Patterns + Step Pyramid — บวมบน/บวมล่าง/บวมกลาง, dynamic sizing, overlap guard, step vs size tradeoff, TP policy ฝั่งขาย + Basket TP	~2 ชม.	กลาง
Part 3	Zone Sizing — Vol/ATR/Bootstrap/Donchian/Keltner — Volatility-adaptive step, Monte Carlo + Block Bootstrap, channel-based zone, GARCH(1,1) forecast	~2.5 ชม.	กลาง
Part 3B	Indicator Filter + Order Accumulation — RSI/Bollinger/Volume/Hurst filter ผ่าน Composite Score, รวบรวม order ที่ฟลาดไป, ลดต้นทุน DD	~1.75 ชม.	กลาง

Regime & Risk (ระบอบตลาดและความเสี่ยง)

Part 4	Regime Detection — เมื่อไหร่ Grid ทำงาน เมื่อไหร่หยุด — Hurst, ADX, Normalized Slope veto, Bollinger, HMM, Regime-Switching Grid	~2.25 ชม.	สูง
--------	--	-----------	-----

Part 5**Sizing & Risk — Kelly, Position Limit, Drawdown Control — Kelly**

criterion บน grid, fractional Kelly, max notional per level

~2.5
ชม.

กลาง

Advanced Strategy (กลยุทธ์ขั้นสูง)**Part 6****Pair Grid + Relative Value Switching —** Cointegration-based pair,

Inverse Correlation Grid, BTC ↔ ETH grid

~2.25
ชม.

สูง

Part 6B**Grid Snowball + DCA Stack —** ทบผลกำไร 3 แบบ, Grid → DCA Wealth Port,

Smart DCA by IV/Hurst

~1.75
ชม.

สูง

Part 7**Advanced Integrations — IV, Funding Rate & Options —** IV rank →

step size, funding rate bias, CSP เป็น grid level, Short Strangle/Iron Condor, GARCH-based step

~2.75
ชม.

สูง

Part 7B**Grid-Framework + Indicator Execution —** Grid คัดทุน/risk, Indicator ตัดสิน

ใจเปิด order: 4 styles + Composite Score

~2
ชม.

สูง

Supplementary & Implementation (กลยุทธ์เสริมและการปฏิบัติ)**Part 8****5 Supplementary Strategies —** Rebalancing Grid, Stablecoin Grid, Flash

Crash Grid, Cross-Exchange Arb Grid, Laddered CSP Grid

~3
ชม.

สูง

Part 9**Python Implementation Guide — WebSocket, Backtest & Live —** State

machine, async engine, backtester, live trading on Bybit

~4
ชม.

สูง

Appendix**สูตร, Glossary, Cross-Book Index —** ตารางสูตรทั้งหมด, คำศัพท์ไทย/

อังกฤษ, ดัชนีข้ามเล่ม

~1.5
ชม.

อ้างอิง

เส้นทางการอ่านที่แนะนำ

เส้นทาง "เริ่มต้น Grid จากศูนย์"

- 1 **Part 0** — เข้าใจ Grid พื้นฐานและปรัชญา
- 2 **Part 1A** — Close System คือหัวใจของ risk control
- 3 **Part 5** — จัดขนาดและ risk ก่อนลงทุนจริง
- 4 **Part 3** — ปรับ step ตาม volatility
- 5 **Part 4** — Regime Detection (Hurst/ADX เต็มรูปแบบ — Part 0/1A/3 อ้างถึงบทนี้หลายครั้ง แนะนำอ่านก่อนไป Part 9)
- 6 **Part 9** — ลงมือ implement

ถ้าคุณ...	เริ่มที่
เพิ่งรู้จัก Grid ครั้งแรก	Part 0 → Part 1A
อยากทำ Grid Hedge (ถือ Spot ป้อนกัน)	Part 1A → Part 1C
อยากใช้ Indicator กรอง entry	Part 3B → Part 7B
อยากทำ Pair Grid (BTC ↔ ETH)	Part 5 → Part 6
อยาก compound กำไรจาก Grid	Part 6B — Grid Snowball + DCA Stack
สนใจ Options เสริม Grid	Part 7 → Part 8

การเชื่อมต่อกับซีรีส์

หนังสือในซีรีส์ที่ใช้ความรู้มาต่อ

ความรู้จากเล่ม	ใช้ใน Grid Part
Statistical Arbitrage — Hurst, OU, Cointegration, Kelly	Part 3, 4, 5, 6
Volatility Mastery — IV, ATR, GARCH, Vol Regime	Part 3, 7
Arbitrage — Funding rate, Spot-Perp, Cash & Carry	Part 1C, 7, 8
Payoff Mastery — CSP, Short Strangle, Payoff diagram	Part 7, 8
คณิตศาสตร์สำหรับ Options — Normal dist, OLS, Optimization	Part 1A, 3, 5

GRID TRADING MASTERY · PART 0

รู้จัก Grid Trading + ประโยชน์

กลไกพื้นฐาน · เปรียบเทียบ Buy-Hold/DCA · ประเภท Grid · ประโยชน์ "Grid คือ Grid"

ตัวอย่าง: BTC/USDT บน Bybit

แก่นของ PART นี้ — อ่านก่อน

Grid Trading ไม่ใช่เวทมนตร์ — มันคือ **กรอบวินัย** ที่บังคับให้ซื้อต่ำขายสูงซ้ำๆ อย่างเป็นระบบ Alpha ที่แท้จริงมาจากการดักจับ **mean-reversion** ของตลาดจริง — ไม่ใช่จากโครงสร้าง grid ลอยๆ (ในตลาดสับสนแท้ grid เปล่าไม่มี alpha เลย ดูเหตุผลเต็มใน 0.3) และไม่ใช่จาก indicator หรือ trick ที่ใส่เพิ่ม

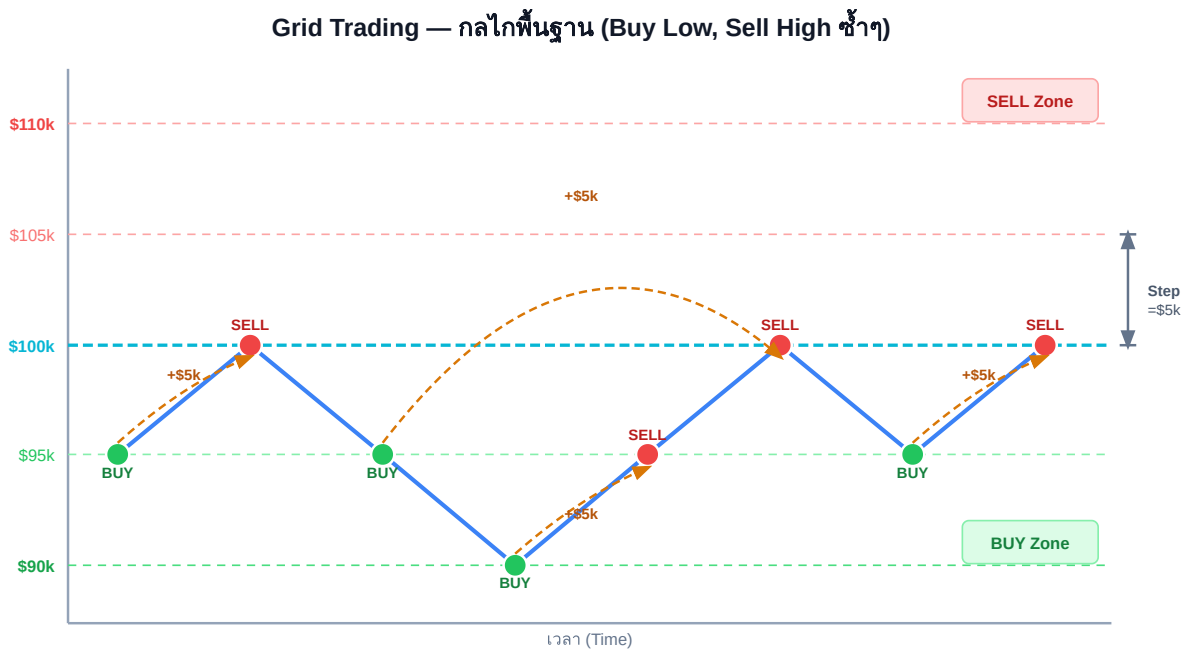
กำไรต่อรอบ = $Q \times (P_{\text{sell}} - P_{\text{buy}}) - \text{Fees}$ [ทุกรอบที่ราคา
แกว่งผ่าน step หนึ่งครั้ง]

0.1 Grid Trading คืออะไร

ลองนึกภาพคุณตั้ง "กั๊บดีก" ราคาไว้หลายระดับ:

- **ซื้อ** เมื่อราคาตกลงถึงระดับที่กำหนด
- **ขาย** ที่ระดับถัดขึ้นไป (ต่างกัน 1 step)
- **ทำซ้ำ** — ตลอดเวลา ไม่ดูแผนภูมิ ไม่ตัดสินใจเพิ่ม

ผลลัพธ์: ทุกครั้งที่ราคา **แกว่งขึ้น-ลง** ผ่านระดับ grid หนึ่งคู่ คุณได้กำไร 1 step โดยอัตโนมัติ



แต่ละคู่ level ปิดกำไร 1 step อิสระ — ข้าม 2 levels รวดเดียว = เปิด BUY 2 ครั้งแยกกัน ไม่ใช่กำไรก้อนเดียว

กลไก Grid: BUY ที่ระดับล่าง → SELL ที่ระดับบน = กำไรต่อรอบ ทุกครั้งที่ราคาแกว่งผ่าน

ระวัง: ราคาข้ามหลาย level ในการเคลื่อนไหวครั้งเดียว

แต่ละคู่ level ปิดกำไรของตัวเองเท่านั้น — ถ้าราคาตกจาก \$105,000 ลงไป \$95,000 รวดเดียว (ข้าม 2 levels) grid จะ **BUY ที่ทุก level ที่ผ่าน** (สมมติมี order/capital พอ) คือ BUY ที่ \$100,000 และ BUY ที่ \$95,000 แยกกัน 2 order — ไม่ใช่ BUY แค่ level ปลายทางเดียว

ตอนราคาขึ้นกลับ กำไรที่ได้ก็เป็นผลรวมของ 2 รอบอิสระ (\$5k + \$5k) ไม่ใช่กำไรก้อนเดียวข้าม step (\$10k ในเทรดเดียว)

Running Example: BTC/USDT บน Bybit ตลอดเล่น

Setup พื้นฐาน: BTC/USDT อยู่ที่ \$100,000 · Grid 5 ระดับ · Step = \$2,000 · ขนาด order = 0.01 BTC/ระดับ ($\approx$ \$1,000/level ที่ BTC=\$100k — ประมาณ 1% ของ capital \$100k ต่อ level)

ระดับ	ราคา	Action	กำไรถ้า Fill ทั้งคู่
+2	\$104,000	SELL (TP ของคู่ +1/+2)	+\$20 (0.01 BTC × \$2k)
+1	\$102,000	SELL (TP ของคู่ 0/+1) / BUY (จุดเข้าเริ่มของคู่ +1/+2)	
0	\$100,000	ราคาปัจจุบัน	—
-1	\$98,000	BUY (จุดเข้าของคู่ 0/-1) / SELL (TP ของคู่ -1/-2)	+\$20 (0.01 BTC × \$2k)
-2	\$96,000	BUY (จุดเข้าเริ่มของคู่ -1/-2)	

แต่ละ level "กลาง" (± 1) ทำหน้าที่คู่: เป็น TP (SELL) ของคู่ level ที่อยู่ใกล้ราคาปัจจุบันกว่า และเป็นจุดเข้า (BUY) ของคู่ level ถัดไปที่ไกลออกไป — คนละหน้าที่กันคนละคู่ ไม่ใช่ทำสองอย่างพร้อมกันในคู่เดียว

Capital ที่ต้องใช้: ซื้อครบทุก level ล่าง = $0.01 \times (\$98k + \$96k) = \$1,940$ (ไม่นับ margin)

0.2 กลไกพื้นฐาน — สิ่งที่ต้องรู้ก่อน

ตัวแปรหลัก 4 ตัว

ตัวแปร	คำอธิบาย	ตัวอย่าง
Zone (เขต)	ช่วงราคาที่ grid ทำงาน [ล่าง, บน]	\$90,000 – \$110,000
Step (ก้าว)	ระยะห่างระหว่าง grid level	\$2,000 (2% ของราคา)
N (จำนวน level)	$= (Zone_top - Zone_bottom) / Step$	$(110k - 90k) / 2k = 10 \text{ levels}$
Q (ขนาด order)	จำนวน BTC ต่อ order ต่อ level	0.01 BTC $\approx$ \$1,000/level

สูตรหลักที่ต้องจำ

กำไรต่อรอบ = $Q \times Step - Fees$

$= 0.01 \text{ BTC} \times \$2,000 - (\$1,000 \times 0.1\% \times 2)$

$= \$20 - \$2 = \$18 \text{ สุทธิ}$

Capital ทั้งหมด (Close System ถึง zone_bottom):

$C_total = Q \times (P_1 + P_2 + \dots + P_N_below)$

$= 0.01 \times (\$98k + \$96k + \$94k + \dots + \$90k) \leftarrow \text{ทุก BUY level}$

$= 0.01 \times \sum P_i \text{ [ถ้า } N=5 \text{ levels ด้านล่าง} = 0.01 \times \$470k = \$4,700]$

รอบต่อวัน (ประมาณ) = $ATR_daily / Step$

$= \$3,000/\text{วัน} / \$2,000 = 1.5 \text{ รอบ/วัน (โดยเฉลี่ย)}$

กำไรต่อวัน (ประมาณ) $\approx 1.5 \times \$18 = \$27/\text{วัน}$ บน capital \$4,700 $\approx 0.57\%/\text{วัน}$

ATR (Average True Range) คืออะไร: ตัวเลขมาตรฐานที่บอกว่า "โดยเฉลี่ยแล้วราคาขยับไปเท่าไรต่อวัน" — $ATR=\$3,000$ แปลว่าราคามักแกว่งประมาณ \$3,000/วัน (ไม่ใช่ราคาขึ้นหรือลง \$3,000 เป๊ะทุกวัน แต่เป็นค่าเฉลี่ยของช่วงราคาสูง-ต่ำในแต่ละวัน) เอา ATR ทหารด้วย Step จึงประมาณได้ว่าราคาจะ "ข้าม" กี่ step ต่อวัน = ก็รอบ BUY-SELL ต่อวันโดยประมาณ

⚠ ตัวเลขนี้เป็น mechanical illustration ภายใต้สมมติฐาน (ราคาอยู่ใน zone, ATR คงที่, fill ครบทุกรอบ, ไม่มี slippage)

ไม่ใช่ expected return – ดู "ทฤษฎีที่ต้องรู้" ใน 0.4

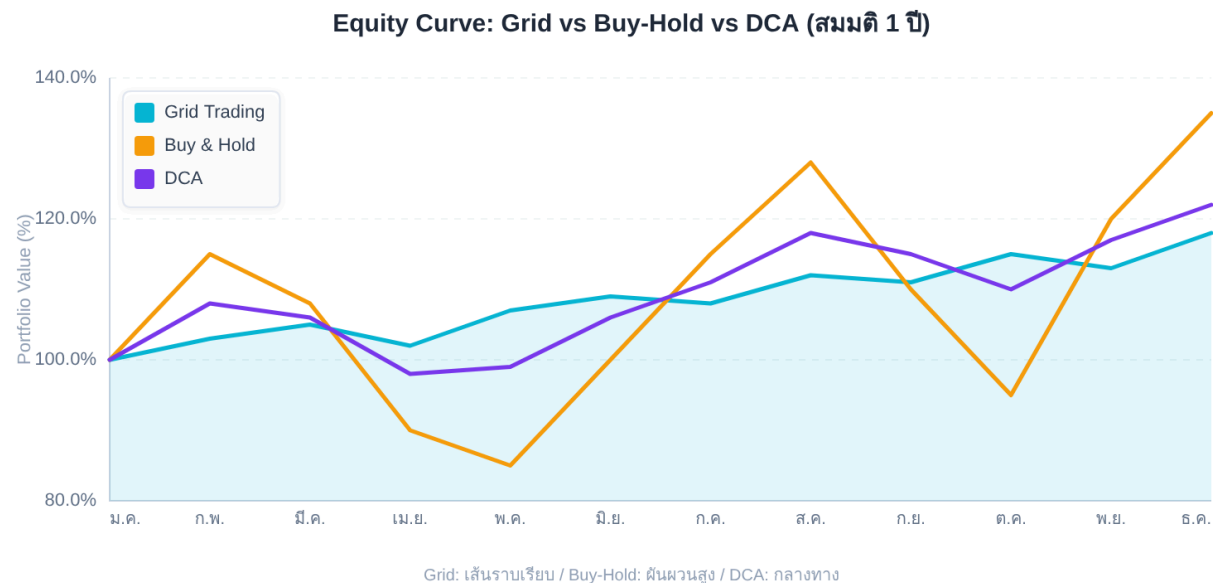
หมายเหตุ: ATR_daily มาจากแท่งรายวัน – เล่มนี้ใช้แท่งรายวันเป็น default สำหรับทุก indicator เว้นแต่ระบุไว้เป็นอย่างอื่น (รายละเอียดดู Part 3 §3.0)

หมายเหตุอีกข้อ: "กำไรต่อรอบ = $Q \times \text{Step}$ " ตั้งอยู่บนสมมติฐานว่าขาย (TP) ที่ level ถัดไปเสมอ (1 step) – นี่คือ default ของเล่มนี้ทั้งเล่ม ไม่ใช่ทางเลือกเดียว (ทางเลือกอื่นดู Part 2 §2.6.1)

Step เล็กหรือใหญ่ – Tradeoff สำคัญ

- **Step เล็ก:** รอบเยอะ → cashflow สูง – แต่ inventory ค้างมาก ถ้าราคาลงแรง
- **Step ใหญ่:** รอบน้อย → cashflow ต่ำ – แต่ fill เร็ว ไม่ค้าง inventory นาน
- **Rule of thumb:** กำไรต่อรอบ $\geq 3 \times \text{fee}$ เพื่อให้กำไรสุทธิเป็นบวกทุกรอบพร้อมมี margin กัน ความคลาดเคลื่อน (ดูรายละเอียดใน Part 3)

0.3 Grid เทียบกับ Buy-Hold และ DCA (Dollar-Cost Averaging — ซื้อสม่ำเสมอทีละน้อยไม่สนราคา)



Grid ให้ equity curve ราบเรียบที่สุดและ drawdown ต่ำสุด — แต่ในปีสมมตินี้ที่ตลาดเป็นขาขึ้นแรง Buy-Hold กลับทำ total return ได้สูงกว่า (+35% vs Grid +18%) เพราะจับ trend ได้เต็มที่ ขณะที่ Grid เก็บกำไรได้แค่จากการแกว่ง (oscillation) เท่านั้น — นี่คือ trade-off ของ Grid: แลก upside ตอนตลาด trend แรง กับความราบเรียบ/DD ต่ำตอนตลาด sideways

มิติ	Grid Trading	Buy & Hold	DCA
กำไรหลัก	Cashflow จากการแกว่ง	Capital gain (ราคาขึ้น)	ซื้อถัวเฉลี่ย ขายเมื่อพร้อม
ต้องการ trend?	ไม่ต้อง (ต้องการ sideways)	ต้องการ (uptrend)	ได้ประโยชน์ถ้า uptrend
Drawdown	น้อย (ถ้า zone ออกแบบดี)‡	สูงมาก (−30% ถึง −70%)	ปานกลาง
ความซับซ้อน	กลาง (ต้องออกแบบ zone+step)	ต่ำ (ซื้อและรอ)	ต่ำ-กลาง
ดีที่สุดเมื่อ	ตลาด sideways-choppy (H < 0.5)†	Bull market ยาว	ตลาดผันผวน ระยะยาว
แย่ที่สุดเมื่อ	ตลาด trending แรง (H > 0.6)†	Bear market ยาว	Sideways แบบไม่ขึ้น

† Hurst Exponent (H) คืออะไร

H คือตัววัดว่าราคา "มีแนวโน้มกลับสู่ค่ากลาง" มากน้อยแค่ไหน: $H < 0.5$ = mean-reverting (ขึ้นแล้วมักลง ลงแล้วมักขึ้น — grid ทำงานดี) · $H = 0.5$ = random walk · $H > 0.5$ = trending (ขึ้นแล้วมักขึ้นต่อ — grid ระวัง)

วิธีคำนวณ H และการ integrate เข้า state machine อยู่ใน [Part 4: Regime Detection](#)

‡ เมื่อไหร่ Grid ล้มเหลว — ทำไม "Drawdown น้อย" ในตารางข้างบนไม่ใช่ทั้งหมด

Grid สมมติว่าตลาดแกว่ง (mean-reverting) — ถ้าราคา **ดิ่งลงไม่กลับ** หรือ **ทะยานขึ้นไม่หยุด** grid จะขาดทุนจาก inventory ที่ค้างอยู่

ตัวอย่าง: BTC \$100k → \$60k โดยไม่กลับ → grid ซื้อตลอดทาง → inventory ค้าง → unrealized loss สูง แต่ถ้า **Close System** ออกแบบดี capital ไม่หมดจนกว่าจะถึง floor (ดู Part 1A)

"Drawdown น้อย" ในตารางเปรียบเทียบด้านบนวัดจาก equity curve ที่ **realized** (grid ปิดกำไรเป็น step ๆ ตลอด) แต่ไม่ได้รวม unrealized loss ของ inventory ที่ยังค้างอยู่ตอนราคาต่ำ — ถ้าตลาด trend ลงแรง ($H > 0.6$) drawdown ที่แท้จริงอาจไม่น้อยเลย เพราะ capital ถูกทยอยดึงเข้าไปซื้อของถูกลงเรื่อยๆ (fully deployed) ต่างจาก buy & hold ที่แค่ถือ position เดียวคงที่

0.4 ทฤษฎีเบื้องหลัง — ทำไม Grid ถึงมีค่าจริง (Random Walk vs เกมของ Shannon)

ส่วนนี้คือข้อสรุปที่สำคัญที่สุดของทั้งเล่ม อธิบายว่าทำไมของ grid มาจากไหนจริงๆ — ควรอ่านให้จบก่อนไปต่อ

ทฤษฎีที่ต้องรู้: ใน Random Walk กำไรคาดหวังของ Grid = 0

ลองตัดเรื่อง mean-reversion ออกไปก่อน สมมติราคา "เดินสุ่ม" แท้ๆ — ขึ้นหรือลงทีละ 1 step ด้วยโอกาสเท่ากันเป๊ะ 50:50 ไม่มีแนวโน้ม ไม่มีความจำ คำถามตรงๆ คือ: grid ที่ตั้งไว้แล้วปล่อยให้ทำงานไปเรื่อยๆ จนกว่าราคาจะหลุดขอบ zone จะมีกำไรคาดหวังเท่าไร?

คำตอบที่พิสูจน์ได้ทางคณิตศาสตร์คือ **ศูนย์พอดี** — และติดลบทันทีที่รวมค่า fee เหตุผลมาจากแรงสองแรงที่หักล้างกันเป๊ะ:

- **แรงบวก:** ทุกครั้งที่ราคาแกว่งผ่านคู่ level → กำไร 1 step (นี่คือสิ่งที่เรามองเห็นและชอบ)
- **แรงลบ:** โครงสร้าง grid ขายของทั้งเร็วตอนขาขึ้น (จำกัด upside) และทยอยซื้อสะสมตอนขาลง (เพิ่ม exposure ให้ฝั่งขาลง) — วันที่ราคาเดินทางเดียวจนหลุดขอบ ความเสียหายก่อนนั้นมีค่าเฉลี่ยเท่ากับกำไร step เล็กๆ ที่เก็บมาได้ทั้งหมดพอดี

Random walk 50:50, หุ่น M, จำนวน grid n ซอง, ไม่คิด fee:

ความเสียหายคาดหวังจากการเดินทางเดียวจนหลุดขอบ
$$= (M/n) \times (n^2/8 - n/4)$$

จำนวนรอบ arbitrage คาดหวังก่อนราคาหลุดขอบ
$$= n^2/8 - n/4 \text{ รอบ} \quad (\text{กำไรรอบละ } M/n)$$

$E(\text{grid}) = \text{กำไรรอบแกว่ง} - \text{ความเสียหายหลุดขอบ} = 0$
เมื่อรวม fee: $E(\text{grid}) < 0$

ตัวเลข $n^2/8 - n/4$ ใช้เป็น **break-even check** ก่อนตั้ง grid ได้เลย: grid ต้องเก็บรอบให้ได้เกินจำนวนนี้ ก่อนราคาหลุด zone จึงจะชนะการถือเงินเฉยๆ

n (จำนวนช่อง grid)	6	8	10	12	16	20
รอบ break-even ที่ต้องเก็บให้ได้	3	6	10	15	28	45

วิธีใช้จริง: ประมาณรอบ/วัน $\approx \text{ATR/Step}$ (สูตรใน 0.2) คุณจำนวนวันที่คาดว่าจะอยู่ใน zone แล้วเทียบกับตาราง — เช่น grid 10 ช่อง เก็บได้ 1.5 รอบ/วัน ต้องอยู่ใน zone เกิน ~7 วันถึงคุ้ม ถ้าประเมินแล้วไม่ถึง แปลว่า grid นั้นตั้งมาเพื่อแพ้ random walk

ข้อสรุปที่เปลี่ยนวิธีคิดทั้งเล่ม

Grid **ไม่ได้สร้างกำไรจากโครงสร้างของตนเอง** — ใน random walk มันแค่แลก "กำไรเล็กๆ หลายครั้ง" กับ "เสียหายก้อนใหญ่ครั้งเดียว" แบบเจ้ากันพอดี Edge ที่แท้จริงต้องมาจากอย่างใดอย่างหนึ่ง: **(1)** ตลาด mean-revert จริง ($H < 0.5$ — ทำให้จำนวนรอบจริงมากกว่าที่ random walk ให้ $\rightarrow$ Part 4 คือเครื่องมือคัดตลาดแบบนี้) หรือ **(2)** ไม่หยุดเมื่อหลุดขอบ แต่ migrate grid ตามราคา (ดู Part 4 §4.5–4.6 เรื่อง Reset/Migration และ Part 1D) — เหตุผลที่ทางที่สองนี้ใช้ได้จริง มาจากเกมง่ายๆ อีกเกมหนึ่ง

แล้วเงินจริงๆ มาจากไหน — เกมเหรียญของ Shannon

ถ้า grid ที่หยุดเมื่อหลุดขอบมีค่าคาดหวังเป็นศูนย์ แล้วคนที่ได้กำไรจาก grid จริงๆ เขาได้มาจากไหน? คำตอบซ่อนอยู่ในเกมง่ายๆ ที่นักคณิตศาสตร์ชื่อ Claude Shannon เคยเล่นให้ลูกศิษย์ดู

คุณมีเงิน \$100 แบ่งเป็นสองกอง: เงินสด \$50 กับเหรียญ \$50 กติกามีข้อเดียว — **ทุกคืน ปรับพอร์ตกลับมาครึ่ง-ครึ่งเสมอ** เหรียญขึ้นก็ขายส่วนเกินออก เหรียญลงก็ซื้อเพิ่มให้กลับมาครึ่งหนึ่ง

สมมติเหรียญนี้บ้าพลัง: วันแรกราคาลดลงครึ่งหนึ่ง วันที่สองเต็งกลับเท่าตัว — จบสองวันราคากลับมาที่เดิม เป๊ะ คนถือเหรียญเฉยๆ ได้กำไร 0%

	เงินสด	เหรียญ	รวม
เริ่ม	\$50	\$50	\$100
เหรียญ $\div 2 \rightarrow$ ปรับสมดุล	\$37.5	\$37.5	\$75
เหรียญ $\times 2 \rightarrow$ ปรับสมดุล	\$56.25	\$56.25	\$112.5

ราคาจบที่เดิม แต่พอร์ตโต 12.5% — และจะโตแบบนี้ไปเรื่อยๆ ทุกรอบแกว่ง กำไรนี้ไม่ได้มาจากการรู้ทิศทาง (เกมนี้ไม่ต้องเดอะไรเลย) มันมาจาก**วินัยการปรับสมดุล**ล้วนๆ: กติกาส่งค้ำให้คุณซื้อตอนถูกและขายตอนแพงโดยอัตโนมัติ เสียงคุ้นๆ ไหม — นี่ก็สิ่งเดียวกับที่ grid ทำ แค่คนละหน้าาก

แล้วอะไรคือความต่างระหว่างเกมของ Shannon (กำไร) กับ grid มีขอบในหัวข้อที่แล้ว (เจ้า)? คำเดียว:

"หยุด" เกมของ Shannon ไม่มีวันเลิกเล่น — ราคาสูงเท่าไรก็ปรับสมดุลต่อ ส่วน grid มีขอบยอมแพ้ที่กัน
เหว ขยายทุกอย่างในวันที่แย่ที่สุดพอดี การ "ขายที่กันเหว" ครั้งเดียวนั้นแหละที่กินกำไรจากการแกว่งทั้งหมด
คืนไปเป๊ะๆ ตามที่พิสูจน์ไว้ข้างบน

เพราะฉะนั้น เส้นทางสู่กำไรของ grid จึงมีสองทาง ไม่มีทางที่สาม:

- เล่นเฉพาะสนามที่แกว่งจริง ($H < 0.5$ — Part 4 จะสอนวิธีดู) เพื่อให้เก็บรอบได้มากกว่าที่ตลาดสู้ให้
- เลิก "หยุด" — เมื่อราคาหลุดขอบ อย่าล้างพอร์ต แต่ย้ายสนามแล้วเล่นต่อ (Part 4 §4.5–4.6 และ Part 1D จะสอนวิธีทำอย่างปลอดภัย)

แต่จำไว้ก่อนดีใจ: การเล่นต่อไม่ใช่ของฟรี เงินของคุณต้องอยู่ในเกมตลอดเวลา — inventory ที่ถือระหว่าง
ทางลงคือค่าตัวของกำไรแบบ Shannon ความเสี่ยงไม่ได้หายไปไหน มันแค่เปลี่ยนชื่อ และหน้าที่ของทั้งเล่ม
นี่คือสอนวิธีจ่ายค่าตัวนั้นให้ถูกที่สุด

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่มนี้): Dempster, Evstigneev & Schenk-Hoppé, "Volatility-Induced Financial Growth" (2007)
· Chen, Chen & Jang, arXiv:2506.11921 (2025)

0.5 ปรัชญา "Grid คือ Grid"

หลักการที่ทำให้ GRID คือ GRID

"Enhancement เสริมคุณภาพ equity curve — ไม่ได้เพิ่ม total P&L อย่างมีนัย"

Grid framework เอง (ไม่มี indicator ใดเลย) ก็มี alpha จากการดักจับ mean-reversion — เฉพาะในตลาดที่ mean-revert จริง ($H < 0.5$) เพราะใน random walk แท้ EV ของ grid = 0 (ดู 0.4)
Indicator ช่วยลด DD, เพิ่ม Sharpe — แต่ P&L รวมใกล้เคียงเดิม

นี่ไม่ได้หมายความว่า Enhancement ไม่มีค่า — ตรงกันข้าม:

Enhancement มีค่า — แต่ไม่ใช่แบบที่คิด

- **ลด DD:** ถ้า DD จาก 20% → 10% คุณนอนหลับได้ดีขึ้น ไม่ panic sell → *นี่คือคุณค่าที่แท้จริง*
- **เพิ่ม Sharpe:** Sharpe จาก 0.8 → 1.2 หมายถึง risk-adjusted return ดีขึ้น → ขยาย capital ได้มากขึ้น
- **ลดเวลาใน DD:** พ้นตัวเร็วขึ้น → ใช้ capital ได้ productive มากขึ้น

สิ่งที่ไม่ควรคาดหวัง: indicator จะทำให้ P&L เพิ่มขึ้น 50% จาก grid ปกติ — นั่นคือ overfitting

หลักฐาน: Coin Flip เป็น Indicator ก็ยังมีกำไร

ลองสมมติ: ทุก 8 ชั่วโมง โยนเหรียญ → หัว = เปิด grid, ก้อย = หยุด grid

ผลที่ได้: กำไร $\approx 50\%$ ของ grid ที่รันตลอด 24 ชั่วโมง (เพราะเปิดแค่ครึ่งเวลา) — แต่ DD ลดลงด้วย

ถ้า ADX filter (ADX = Average Directional Index — ตัววัด "ความแรงของ trend" สเกล 0–100 ยิ่งสูงยิ่ง trend ชัด รายละเอียดเต็มดู Part 4) ทำได้ดีกว่าโยนเหรียญ → มีค่า | ถ้าเท่ากัน → ใช้ random filter ประหยัดเวลา backtest

0.6 ประเภทของ Grid ที่เราจะเรียน

Buy-Only Grid (Close System)

ซื้ออย่างเดียว ขายเพื่อ take profit เท่านั้น — ไม่มี SL ยกเว้น 0 หรือ give-up point

เหมาะกับ: ผู้ถือ long-term view, ต้องการ passive income จากการแกว่ง

→ [Part 1A](#)

Buy-and-Sell Grid (Bidirectional)

มีทั้งฝั่งซื้อ (inventory long) และฝั่งขาย (inventory short) — profit จากทั้งขาขึ้นและขาลง

เหมาะกับ: ตลาด choppy, ผู้ใช้ perpetual futures

→ [Part 1B](#)

Grid Hedge (Spot + Perp)

ถือ Spot BTC ไว้ + short perp เป็น hedge — เก็บ funding rate และกำไรจาก grid

เหมาะกับ: ผู้ถือ spot asset อยู่แล้ว อยากลด risk

→ [Part 1C](#)

Asymmetric Hybrid Grid

Buy-only เป็นแกนหลัก + hedge เฉพาะ inventory จริง + short เป็นชั้นเล็กที่จำกัดขาดทุนไว้ล่วงหน้า

เหมาะกับ: ไม่อยากเลือกสุดขั้วระหว่าง Buy-Only กับ Bidirectional

→ [Part 1D](#)

0.7 สามเสาหลักของ Grid ทุกแบบ

ไม่ว่าจะเป็น Grid แบบไหน ต้องตัดสินใจ 3 อย่าง:

เสาที่ 1: Zone — "เล่นที่ไหน"

กำหนดช่วงราคา [ล่าง, บน] ที่ grid จะทำงาน — Zone กว้างเกินไป = capital ลือมากเกินไป | Zone แคบเกินไป = ราคาออกนอก zone บ่อย

เครื่องมือ: Donchian Channel (price-based), Bollinger Band (σ -based), ATR multiple, Monte Carlo + Block Bootstrap

→ ดูรายละเอียดใน [Part 3](#)

เสาที่ 2: Step — "ห่างแค่ไหน"

กำหนดระยะห่างระหว่าง grid level — Step เล็ก = รอบเยอะ แต่ inventory risk สูง | Step ใหญ่ = รอบน้อย แต่ fill ช้า

เครื่องมือ: ATR-based step ($\text{Step} \approx k \times \text{ATR}$), Keltner Channel, IV-adaptive step

→ ดูรายละเอียดใน [Part 3](#)

เสาที่ 3: Capital — "ใช้เท่าไร อย่างไร"

กำหนดจำนวนเงินที่ต้องใช้ต่อ level และ total capital ที่ reserve ถึง floor — Capital ไม่พอ = margin call ก่อน grid เต็มระบบ

เครื่องมือ: Close System formula, Kelly sizing, Zone Waterfall (Active/Standby/Floor)

→ ดูรายละเอียดใน [Part 1A](#) และ [Part 5](#)

0.8 ลำดับการเรียนรู้และสิ่งที่รอคุณอยู่

Part	เนื้อหา	สิ่งที่จะทำได้หลังอ่าน
1A	Close System + Zone Tricks + Soft Martingale	ออกแบบ grid ที่ไม่ bust ไม่ว่าราคาจะลงแค่ไหน (ถ้า capital พอ)
1B	Buy-and-Sell (Bidirectional)	ทำกำไรทั้งขาขึ้นและขาลง บน perpetual futures
1C	Grid Hedge	ป้องกัน inventory risk ด้วย perp short, เก็บ funding rate ไปด้วย
1D	Asymmetric Hybrid Grid	ผสม buy-only core + hedge + capped short layer เป็นทางสายกลาง แทนการเลือกสุดขั้ว
2	Bloating + Step Pyramid	จัดการ "grid บวม" และเพิ่ม step แทน size ในช่วง trend
3	Volatility-Adaptive Zone+Step	ปรับ zone และ step ตาม ATR, IV, Donchian, Keltner อัตโนมัติ
3B	Indicator Filter + Order Accumulation	ใช้ RSI/Bollinger/Volume/Hurst กรอง entry ผ่าน Composite Score, รวบรวม order ที่พลาดด้วย signal
4	Regime Detection	รู้ว่าตอนนี้ตลาด mean-revert หรือ trend → ตัดสินใจ grid on/off
5	Kelly + Risk Management	คำนวณขนาด position ที่ optimal, ป้องกัน ruin
6	Pair Grid	เทรด spread BTC ↔ ETH หรือ inverse correlation
6B	Grid Snowball + DCA	ทบกำไร 3 แบบ, โอน cashflow ไปสร้าง wealth portfolio
7+7B	Advanced + Framework+Indicator	IV-adaptive, funding-adaptive, Grid เป็น framework + indicator เป็น executor
8	5 Supplementary Strategies	Rebalancing, Stablecoin Grid, Flash Crash, Cross-Exchange Arb, Laddered CSP เสริม Grid
9	Python Implementation	เขียน live bot บน Bybit ตั้งแต่ WebSocket ถึง state machine

0.9 แบบฝึกหัด

แบบฝึกหัด Part 0

- ▶ ข้อ 1: BTC อยู่ที่ \$100k · Grid 10 levels · Step \$2k · $Q = 0.01$ BTC/level — กำไรต่อรอบสุทธิ (fee 0.1%/side) คือเท่าไร?
- ▶ ข้อ 2: ทำไม Grid ถึงทำกำไรได้ในตลาด sideways แต่ Buy-and-Hold ไม่ได้?
- ▶ ข้อ 3: ถ้า Step = \$1,000 (ครั้งหนึ่ง) จะส่งผลต่อ capital และ cashflow อย่างไร?
- ▶ ข้อ 4: Hurst Exponent $H = 0.45$ ของ BTC/USDT บอกอะไรเราเกี่ยวกับ Grid?

GRID TRADING MASTERY · PART 1A

Close System + Zone Tricks + Soft Martingale

ระบบปิด (ไม่มี SL) · Zone Waterfall · Bell Curve Density · Capital Front-Loading · r-parameter

แนวคิดจาก: Close System (พี่ต๋าน Mudley Group)

แก่นของ PART นี้

Close System หมายถึง: วาง capital ทั้งหมดที่ต้องการ *ก่อน* เข้าเล่น จนถึง "floor" หรือ give-up point — ไม่มี stop-loss นอกจากทุนหมด ข้อได้เปรียบ: ไม่โดน stop hunt, ไม่ต้อง top-up กลางทาง, รู้ maximum loss ล่วงหน้า

$$C_{\text{floor}} = \sum_{i=1 \text{ to } N} Q_i \times P_i \text{ [ทุน reserve ถึง floor]}$$

1A.1 Close System คืออะไร

Grid ส่วนใหญ่ที่สอนกันทั่วไปไม่มี stop-loss — ถ้าราคาลงถึงจุดหนึ่งก็ปิด position ยอมขาดทุน

Close System คือแนวคิดตรงกันข้าม:

หลักการ Close System (ระบบปิด)

1. กำหนด **give-up point (floor)** — จุดที่ถ้าราคาลงถึง คุณยอมรับการถือ inventory ทั้งหมด (หรือ capital หมด)
2. วาง **capital ทั้งหมดล่วงหน้า** — คำนวณว่าต้องใช้เงินเท่าไรถ้าราคาลงถึง floor และวาง capital นั้นไว้ก่อน
3. **ไม่มี stop-loss ระหว่างทาง** — ยกเว้น capital หมดถึง floor เท่านั้น
4. **รู้ worst-case ล่วงหน้า** — maximum loss = capital ที่วางไว้ทั้งหมด (ไม่มีเซอร์ไพรส์)

ทำไมไม่มี SL กลาง grid?

ปัญหาของ SL กลาง grid: ถ้า BTC ลงจาก \$100k → \$95k → SL triggered → ขาดทุน \$5k → จากนั้น BTC พุ่งกลับ \$105k — คุณเสียโอกาสทำกำไรทั้งหมด

Close System ไม่มี SL กลาง → ถ้า BTC ลง \$95k ก็รอ → ถ้ากลับ \$105k → กำไร | ถ้าไม่กลับเลย → ขาดทุนเท่า capital ที่ reserve ไว้

Trade-off: ยอมรับ maximum loss ที่ชัดเจน เพื่อแลกกับโอกาสไม่โดน stop hunt

1A.2 Floor Capital — การคำนวณ

Floor = ราคาต่ำสุดที่ยอมรับให้ grid ทำงานถึง (เช่น \$80,000 สำหรับ BTC ที่ \$100,000)

สูตร Capital ที่ต้องการถึง Floor (Equal Weight):
 $C_{\text{floor}} = Q \times \sum(P_i)$ สำหรับทุก i ที่ P_i อยู่ระหว่าง [floor, entry]

ตัวอย่าง: BTC entry = \$100k, floor = \$90k, step = \$2k, $Q = 0.01$ BTC/level
Levels: \$98k, \$96k, \$94k, \$92k, \$90k (5 levels ด้านล่าง)
 $C_{\text{floor}} = 0.01 \times (\$98k + \$96k + \$94k + \$92k + \$90k)$
 $= 0.01 \times \$470k$
 $= \$4,700$

นี่คือทุน "Reserve" ที่ต้องมีก่อนเปิด grid
ถ้าไม่มีครบ \$4,700 → ไม่เปิด grid (ไม่ใช่ลดขนาด order)

สูตร Soft Martingale (ขนาด Q เพิ่มขึ้นตาม level):
 $Q_i = Q_0 \times r^{(i-1)}$ โดย $r \in [1.1, 1.5]$, $i = 1, 2, 3, \dots, N$
 $C_{\text{floor_martingale}} = \sum_{i=1}^N Q_i \times P_i$
 $= \sum (Q_0 \times r^{(i-1)}) \times P_i$ [ต้องคำนวณ case-by-case]

Running Example: คำนวณ Floor Capital

Level	ราคา	Q (BTC)	Capital ที่ใช้	Cumulative
Entry	\$100,000	—	—	—
L1	\$98,000	0.010	\$980	\$980
L2	\$96,000	0.010	\$960	\$1,940
L3	\$94,000	0.010	\$940	\$2,880
L4	\$92,000	0.010	\$920	\$3,800
L5 (floor)	\$90,000	0.010	\$900	\$4,700

สรุป: ต้องมีทุน \$4,700 reserve ก่อนเปิด grid | ถ้า BTC ลงถึง \$90k โดยไม่กลับ → unrealized loss
= $\$4,700 - (0.05 \text{ BTC} \times \$90k) = \$4,700 - \$4,500 = \$200$ เท่านั้น เพราะถือ BTC ไว้

1A.3 Zone Waterfall — Active / Standby / Floor

ปัญหาใหญ่ของ Close System: ต้องวาง capital ทั้งหมดไว้เฉยๆ (idle) รอราคาลงถึงจุดที่ตั้ง order ไว้ — Zone Waterfall แก้ปัญหานี้

ZONE WATERFALL — ACTIVE / STANDBY / FLOOR



ไม่ต้องใช้ทุนทั้งหมดเป็น grid orders — Standby หารดอกเบี้ยระหว่างรอ

วิธีทำงานของ Zone Waterfall

1. เริ่มต้น: Active Zone deploy grid orders ตาม price range ปัจจุบัน
2. เมื่อราคาลง → Active Zone depletes → Standby เลื่อนมาเป็น Active ใหม่
3. เมื่อ Standby depletes → Floor เลื่อนมา (นี่คือ warning signal)
4. เมื่อ Floor depletes → Close System ถึง give-up point → หยุด grid

ข้อดี: Standby Zone ฝากรับดอกเบี้ยได้ขณะรอ → ลด opportunity cost ของ capital ที่ idle

```
def zone_waterfall_capital(total_capital, zone_ratios=(0.5, 0.3, 0.2)):  
    """คำนวณ capital ใน each zone"""  
    active_ratio, standby_ratio, floor_ratio = zone_ratios
```

```

    active_capital    = total_capital * active_ratio    # deploy เป็น
orders
    standby_capital   = total_capital * standby_ratio   # ฝาก
lending/yield
    floor_capital     = total_capital * floor_ratio     # ไม่แตะ

# Standby yield (สมมติ 5% APY)
standby_yield_daily = standby_capital * 0.05 / 365
return {
    "active": active_capital,
    "standby": standby_capital,
    "floor": floor_capital,
    "standby_daily_yield": standby_yield_daily
}

# ตัวอย่าง: total = $10,000
# active=$5,000 · standby=$3,000 · floor=$2,000
# standby_daily_yield = $3,000 × 5%/365 = $0.41/วัน (ลด opportunity
cost)

```

⚠ Counterparty Risk ของ Lending Protocol

Yield 4–8% APY จาก Aave/Compound (lending protocol บน DeFi ที่ผู้ใช้ฝากเหรียญเข้า pool กลางให้คนอื่นกู้ยืม แล้วรับดอกเบี้ยจากผู้กู้) **ไม่ใช่ risk-free**:

- **Smart contract risk:** bug หรือ exploit → เงินอาจหายทั้งหมด (เกิดกับ Euler Finance \$197M ปี 2023)
- **Liquidity risk:** ในช่วง market crash → ถอน USDT ออกไม่ได้ทันที (utilization rate สูง — คือ สัดส่วนเงินใน pool ที่ถูกกู้ไปแล้ว ยิ่งสูงยิ่งเหลือเงินให้ถอนน้อย)
- **เหมาะสำหรับ:** Standby capital ที่รอ > 30 วัน และยอมรับ smart contract risk ได้
- **ทางเลือกที่ safer กว่า:** CEX savings product (Binance Earn, OKX Earn) — counterparty risk ต่ำกว่าแต่ yield ต่ำกว่า

1A.4 Buy-Only Zone Asymmetry — Trick ที่ทรงพลัง

σ (sigma) คืออะไร — ก่อนเข้าเรื่อง

σ (sigma / standard deviation) คือตัวเลขมาตรฐานที่บอกว่า "ราคากระจายตัวห่างจากค่าเฉลี่ยแค่ไหน" ถ้า BTC เฉลี่ยอยู่ที่ \$100k และ $\sigma = \$10k$ แปลว่าราคามักแกว่งอยู่แถวๆ \$90k–\$110k ($\pm 1\sigma$) เป็นส่วนใหญ่ นี่คือกฎทางสถิติทั่วไปสำหรับข้อมูลที่กระจายตัวแบบ "ระฆังคว่ำ" (bell curve/normal distribution): **ประมาณ 68% ของเวลาราคาจะอยู่ใน $\pm 1\sigma$ และ 95% ของเวลาจะอยู่ใน $\pm 2\sigma$ จากค่าเฉลี่ย** — ยิ่งไกลจากค่าเฉลี่ยก็ σ ยิ่งเกิดขึ้นน้อยลงเรื่อยๆ

ทำไมเรื่องนี้สำคัญกับ grid: การรู้ว่าราคา "น่าจะ" อยู่ในช่วงไหนบ่อยแค่ไหน ช่วยตัดสินใจว่าควรวาง zone กว้างแค่ไหน — วางกว้างเกิน $\pm 2\sigma$ เสียทุนไปกับโซนที่ราคาแทบไม่เคยไปถึง

ZONE TRICK #1 — ASYMMETRIC ZONE

Grid ทั่วไป: วาง zone -2σ ถึง $+2\sigma$ (symmetric)

Zone Trick: วาง zone -2σ ถึง $+1\sigma$ (ไม่ถึง $+2\sigma$) สำหรับ Buy-Only Grid

ทำไม: ราคาอยู่เหนือ $+1\sigma$ น้อยมาก — วาง grid สูงเกิน $+1\sigma$ เหมือน "ซื้อบนดอย" ที่ไม่ค่อย fill หรือ fill แล้วราคาไม่กลับลงมา

ข้อดีของ -2σ ถึง $+1\sigma$: ประหยัดทุนได้ ~25% (ตัดส่วนบนออก) + ลด inventory risk บนสุด

สมมติ BTC mean = \$100k, $\sigma = \$10k$ (10%)

Zone ทั่วไป (-2σ ถึง $+2\sigma$): \$80k ถึง \$120k (range \$40k)

Zone Trick (-2σ ถึง $+1\sigma$): \$80k ถึง \$110k (range \$30k)

Capital ที่ประหยัด: Grid levels \$110k–\$120k ไม่ต้องวาง

→ ถ้า step = \$2k → ประหยัด 5 levels × capital ต่อ level

→ $\approx 25\%$ less capital สำหรับ upper zone ที่ "ไม่ค่อย fill"

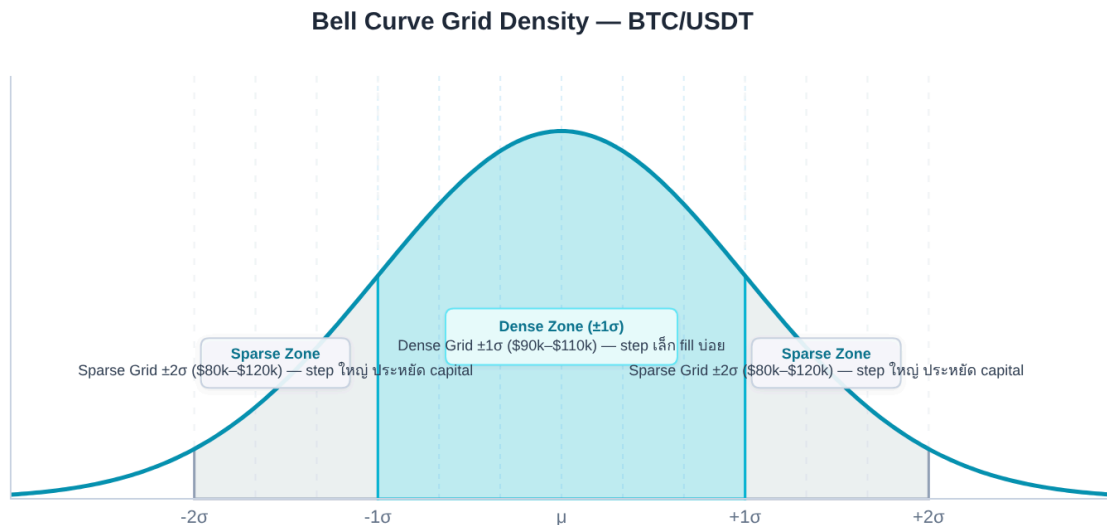
ทดสอบ: ในช่วง 1 ปี BTC อยู่เหนือ $+1\sigma$ เท่าไหร่?

→ ตามทฤษฎีปกติ: ~16% ของเวลา (1 – 84th percentile)

→ จริงๆ BTC: $\approx 15\text{--}20\%$ (ใกล้เคียง) เพราะ crypto skewed right

→ สรุป: 80% ของเวลาราคาอยู่ใน -2σ ถึง $+1\sigma$ = Zone ที่เลือก

1A.5 Bell Curve Grid Density



68% ของเวลา ราคาอยู่ใน $\pm 1\sigma$ / 95% อยู่ใน $\pm 2\sigma$ — จัด grid หนาแน่นตรงกลาง

Grid หนาแน่นที่ mean — Step เล็กในโซนที่ราคาย่อยบ่อย ประหยัด capital ในโซนที่ราคามาน้อย

ZONE TRICK #2 — VARIABLE STEP (BELL CURVE DENSITY)

แทนที่จะใช้ step เท่ากันทุก level ลองปรับ step ตามระยะห่างจาก mean:

- ใกล้ mean ($\pm 0.5\sigma$): step เล็ก → fill บ่อย → cashflow ดี
- ห่าง mean ($1\sigma-2\sigma$): step ใหญ่ขึ้น → fill น้อย → ประหยัด capital

```
def variable_step(price, mean, sigma, base_step):  
    """คำนวณ step size ตามระยะห่างจาก mean (bell curve density)"""  
    z = abs(price - mean) / sigma # z-score ของ price  
    multiplier = 1 + z * 0.5      # step เพิ่ม 50% ต่อ 1σ ที่ห่างออกไป  
    return base_step * multiplier
```

```
# ตัวอย่าง: mean=$100k, σ=$10k, base_step=$2k  
# ที่ $100k (z=0): step = $2,000 × 1.0 = $2,000 (dense)  
# ที่ $95k (z=0.5): step = $2,000 × 1.25 = $2,500  
# ที่ $90k (z=1.0): step = $2,000 × 1.5 = $3,000  
# ที่ $80k (z=2.0): step = $2,000 × 2.0 = $4,000 (sparse)
```

```
# ผลลัพธ์: cashflow จาก center สูง, capital ที่ extremes น้อย
```

ข้อดีของ Bell Curve Density

- Cashflow concentration: 70%+ ของรอบ grid อยู่ใน $\pm 1\sigma$ จาก mean $\rightarrow$ step เล็กในโซนนี้ให้ cashflow สูง
- Capital efficiency: ไม่ต้องใส่ทุนเยอะที่ extreme level ที่ fill น้อย
- Natural risk reduction: ถ้าราคาออกไปที่ extreme, step ใหญ่ $\rightarrow$ ชื่อน้อยกว่าในวิธี equal step

1A.6 Capital Front-Loading — ยืมจาก Upper Zone

ZONE TRICK #3 — CAPITAL FRONT-LOADING

ถ้าราคาอยู่ที่ mean และ upper zone วางเปล่า (ไม่มี sell order fill) — ทำไมต้องปล่อยทุนนั้นนอนเฉยๆ?

Idea: ยืม capital จาก upper zone (ที่ยังไม่ถูกใช้) มาเปิด grid ในราคาปัจจุบัน — เมื่อ sell order fill ที่ upper zone ที่หลัง → คืนทุนอัตโนมัติ

ตัวอย่าง Capital Front-Loading:

Grid zone: \$90k ถึง \$110k · Step \$2k · Q = 0.01 BTC · ราคาปัจจุบัน = \$100k

ปกติ:

- BUY orders ที่ \$98k, \$96k, \$94k, \$92k, \$90k (ด้านล่าง)
- SELL orders ที่ \$102k, \$104k, \$106k, \$108k, \$110k (ด้านบน)
- Capital ใน BUY side = \$4,700 (ตามที่คำนวณ)
- Capital ใน upper zone = 0 (ยังไม่มี inventory)

Front-Loading:

- เปิด BUY orders เพิ่มจาก \$100k ลงมา (aggressive, เต็ม zone)
- จำนวน order เพิ่มขึ้นอีก 1 order (จาก 5 เป็น 6 orders ระหว่าง \$100k-\$90k)
- ใช้ capital จาก "upper zone allocation" มา deploy เพิ่ม
- เมื่อ SELL orders fill ที่ upper zone → คืน capital

ข้อดี: ใช้ทุนได้อย่างมีประสิทธิภาพมากขึ้น (deploy capital productive มากขึ้น) |
ข้อเสีย: ถ้าราคาลงก่อน = inventory เยอะกว่าปกติ

ความเสี่ยงของ Front-Loading

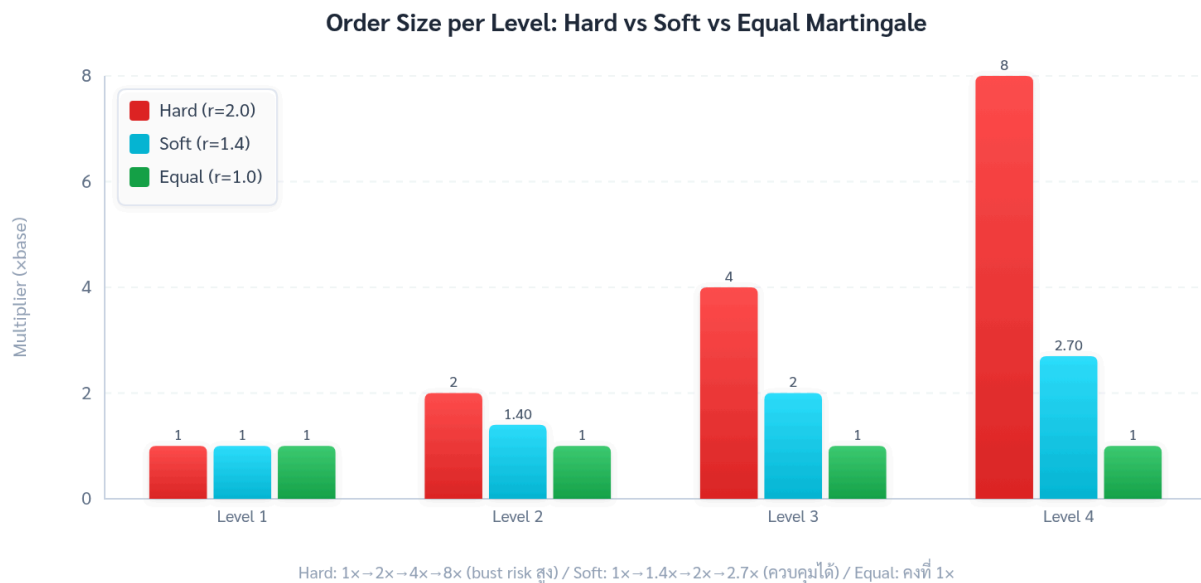
ถ้าราคาลงก่อนที่ upper zone จะ fill → front-loaded capital กลายเป็น "ขาดทุน unrealized" มากกว่าปกติ

วิธีลด risk: Front-load เฉพาะเมื่อ $H < 0.45$ (mean-reverting ชัดเจน) และ ATR ต่ำกว่าค่าเฉลี่ย (ตลาดเงียบ)

1A.7 Soft Martingale — r-parameter

Martingale ดั้งเดิม: เพิ่ม size เป็น 2x, 4x, 8x ทุกครั้งที่ราคาสูง → capital เพิ่มแบบ exponential จนอาจ bust

Soft Martingale: ใช้ $r \in [1.1, 1.5]$ (ปรับได้) แทน $r = 2$ ตายตัว



Soft Martingale ($r=1.4$) เพิ่ม size ช้ากว่า Hard Martingale ($r=2$) — capital requirement ต่างกันมาก

```
def soft_martingale_sizes(n_levels, base_q, r):  
    """คำนวณขนาด order ในแต่ละ level ด้วย soft martingale"""  
    sizes = [base_q * (r ** i) for i in range(n_levels)]  
    return sizes  
  
# Hard Martingale r=2.0 (n=5 levels):  
# [1x, 2x, 4x, 8x, 16x] × base_q  
# Total capital ∝ 1+2+4+8+16 = 31× base_q (สูงมาก)  
  
# Soft Martingale r=1.4 (n=5 levels):  
# [1x, 1.4x, 2.0x, 2.7x, 3.8x] × base_q  
# Total capital ∝ 1+1.4+2.0+2.7+3.8 = 10.9× base_q (ต่ำกว่ามาก)  
  
# Equal Weight r=1.0 (baseline):  
# [1x, 1x, 1x, 1x, 1x] × base_q  
# Total capital ∝ 5× base_q (ต่ำสุด, safe)
```

```
def calculate_capital_requirement(prices, base_q, r):
    """คำนวณ capital ทั้งหมดสำหรับ soft martingale grid"""
    total = 0
    for i, price in enumerate(prices):
        q_i = base_q * (r ** i)
        total += q_i * price
    return total
```

r parameter	ชื่อ	Capital ที่ Level 5 (relative)	เหมาะกับ
1.0	Equal Weight	1x	Conservative, baseline
1.2	Gentle Soft	2.1x	มั่นใจ mean-reversion เล็กน้อย
1.4	Moderate Soft	3.8x	สมดุล (แนะนำ)
1.6	Aggressive Soft ⚠️	6.6x	นอกช่วง [1.1,1.5] ที่แนะนำ — ใช้เฉพาะถ้าทำ stress test แล้วผ่านเท่านั้น (ดู §1A.7 กฎ r>1.5 เสี่ยงมากใน bear market)
2.0	Hard Martingale	16x	ไม่แนะนำ — bust risk สูง

Stress Test ก่อนเลือก r — BTC ลง 40% ต้องใช้ทุนเท่าไร?

ตัวอย่าง: Grid 5 levels · Q=0.001 BTC · r=1.4 · Price \$100k

Level ที่ Fill	Q (Soft Martingale r=1.4)	Capital Deployed	Cumulative BTC
1 (\$98k)	0.001	\$98	0.001
2 (\$96k)	0.0014	\$134	0.0024
3 (\$94k)	0.00196	\$184	0.00436
4 (\$92k)	0.00274	\$252	0.00710
5 (\$90k)	0.00384	\$346	0.01094
รวม	—	\$1,014	0.01094 BTC

ถ้า BTC ลงต่อถึง \$60k (-40% จาก \$100k): Inventory loss = Capital ที่จ่ายจริง - Market value ปัจจุบัน = \$1,014 - (0.01094 × \$60,000 = \$656.4) = **-\$357.6** (ใช้ cost basis จริง ไม่ใช่ราคาอ้างอิง \$100k เพราะ avg cost ที่แท้จริงคือ ~\$92,687/BTC — เทียบวิธีเดียวกับ \$1A.2/\$1A.9)

กฎ: ก่อนเลือก r ให้ทำ stress test: Close System capital ต้อง $\geq$ (cumulative Q ทุก level × worst-case price drop)

$r > 1.5$ เสี่ยงมากใน bear market — Close System อาจหมดก่อนถึง floor

ข้อจำกัดสำคัญของ Soft Martingale

Soft Martingale ไม่ได้ลบ risk ออก — มันแค่ *ลดความเร็ว* ที่ capital เพิ่มขึ้น

ต้องใช้ร่วมกับ Close System เสมอ: คำนวณ capital requirement ทั้งหมดถึง floor ก่อน แล้วค่อยตัดสินใจว่า r เท่าไหร่ที่ capital พอ

Rule: ถ้า capital ที่ต้องการถึง floor $> 50\%$ ของ net worth → ลด r ลงหรือลด N

1A.8 Step Pyramid — เพิ่ม Step แทน Size ในช่วง Trend

IDEA ใหม่ — STEP PYRAMID (ต่างจาก SIZE PYRAMID)

ในช่วง uptrend ชั่วคราว (H ยืนแต่ยังไม่ถึง 0.58 ที่ Part 4 บอกให้ปิด grid): แทนที่จะเพิ่ม size ของ order (martingale) → **เพิ่ม step size** แทน

- **Size Pyramid:** step คงที่, size เพิ่ม → inventory risk เพิ่ม
- **Step Pyramid:** size คงที่, step เพิ่ม → inventory risk เท่าเดิม แต่ fill น้อยลง

ผล: cashflow คล้ายกัน (step ใหญ่ $\times$ fill น้อย $\approx$ step เล็ก $\times$ fill เยอะ) แต่ inventory accumulation ช้าลงในช่วง trend

ตอนนี้จำแค่หลักการพอ: H ยืนขึ้น (เข้าใกล้ 0.58 ที่ Part 4 บอกให้ปิด grid) → step ยิ่งต้องห่างขึ้นเรื่อยๆ ไม่ใช่กระโดดทันที สูตรเต็ม (รวมกับถัวเต็ม% ด้วย) อยู่ใน [Part 2 S2.6](#) — ตอนอ่านถึงตรงนั้นจะมี ถัวเต็ม% ให้ใช้ประกอบแล้ว

1A.9 Running Example รวม — Close System + Soft Martingale

Running Example: BTC/USDT Close System สมบูรณ์

Parameters: Entry = \$100k · Floor = \$90k · Step = \$2k · $r = 1.3$ · Base Q = 0.01 BTC

Level	ราคา	Q (BTC)	Capital (\$)	Cumulative (\$)	กำไรถ้า Sell
L1	\$98,000	0.010	\$980	\$980	+\$20
L2	\$96,000	0.013	\$1,248	\$2,228	+\$26
L3	\$94,000	0.017	\$1,598	\$3,826	+\$34
L4	\$92,000	0.022	\$2,024	\$5,850	+\$44
L5 (floor)	\$90,000	0.029	\$2,610	\$8,460	+\$58

Capital ที่ต้องมี (Close System): \$8,460

Zone Waterfall: Active \$4,230 (ใช้งาน) · Standby \$2,538 (lending ~5% APY → \$0.35/วัน) · Floor \$1,692 (สำรอง)

Unrealized loss ที่จุดแตะ floor: ถ้า BTC ลงถึง \$90k พอดี — BTC สะสมทั้งหมด = 0.091
(0.010+0.013+0.017+0.022+0.029) × \$90,000 = \$8,190 เทียบกับทุนที่ใช้ไป \$8,460 →
unrealized loss = \$270 (ไม่ใช่ "worst case" จริง เพราะ Close System ไม่มี stop-loss — ถ้าราคา
ลงต่ำกว่า \$90k ต่อไป ขาดทุนยังเพิ่มได้อีกไม่มีเพดาน)

Daily cashflow (ประมาณ, 1.5 รอบ/วัน, เฉลี่ย): เฉลี่ยกำไรต่อ level จากตาราง =
(\$20+\$26+\$34+\$44+\$58)/5 = \$36.4 → $\approx 1.5 \times \$36.4 = \$54.6/\text{วัน} = 0.65\%/\text{วัน}$ บน \$8,460

1A.10 แบบฝึกหัด

แบบฝึกหัด Part 1A

- ▶ ข้อ 1: Close System BTC entry \$100k, floor \$85k, step \$2k, $Q = 0.01$ BTC — ต้องการ capital เท่าไหร่?
- ▶ ข้อ 2: ทำไม Buy-Only zone ถึงแนะนำ -2σ ถึง $+1\sigma$ แทน -2σ ถึง $+2\sigma$?
- ▶ ข้อ 3: Soft Martingale $r=1.4$, $base_q=0.01$ BTC, 5 levels — capital รวมมากกว่า Equal Weight กี่%?
- ▶ ข้อ 4: ทำไม Step Pyramid ดีกว่า Size Pyramid ในช่วง uptrend?
- ▶ ข้อ 5 (Challenge): ออกแบบ Close System + Soft Martingale สำหรับ \$20,000 capital — BTC \$100k, floor \$80k, step \$2k

GRID TRADING MASTERY · PART 1B

Buy-and-Sell Grid (Bidirectional)

Long Layer + Short Layer · กำไรทั้งขาขึ้นและขาลง · Capital Allocation · Funding
Rate

ตัวอย่าง: BTC/USDT Perpetual บน Bybit

แก่นของ PART นี้

Buy-and-Sell Grid (Bidirectional) มีทั้ง Long Layer และ Short Layer — Long buy ต่ำ ขายสูง, Short sell สูงซื้อคืนต่ำ → ทำกำไรได้ทั้ง sideways ขาขึ้นและขาลง แต่ต้องการ **Perpetual Futures** สำหรับ Short side

กำไรรวม = กำไร Long Layer + กำไร Short Layer - Funding Rate ที่จ่าย (net)

Futures 101 — พื้นฐานที่ต้องรู้ก่อนเข้าเรื่อง Short Layer

Perpetual Futures (Perp) คือสัญญาซื้อขายล่วงหน้าแบบไม่มีวันหมดอายุ ต่างจากการซื้อ BTC จริง (spot) ตรงที่คุณไม่ได้ถือเหรียญจริง แต่ถือ "สัญญา" ที่ราคาผูกกับ BTC

Short = "ขายก่อน ซื้อคืนทีหลัง" — ยืม BTC มาขายที่ราคาปัจจุบัน แล้วหวังซื้อคืนที่ราคาต่ำกว่าเพื่อทำกำไร (ตรงข้ามกับ Long ที่ซื้อก่อนขายทีหลัง) นี่คือนี่ที่ทำให้ grid ทำกำไรได้แม้ราคาขาลง

Margin กับ Leverage: เวลาเปิด position บน Perp ไม่ต้องวางเงินเต็มมูลค่า (notional) — วางแค่ "เงินค้ำ" (margin) ส่วนหนึ่ง ตามอัตราทด (leverage) ที่เลือก สูตรคือ $\text{margin} = \text{notional} / \text{Leverage}$ เช่น notional \$5,000 ที่ leverage 10x → margin แค่ \$500 ยิ่ง leverage สูง margin ยิ่งน้อย แต่ความเสี่ยงไม่ได้หายไป — แค่นำเงินน้อยลงในการเปิด position ขนาดเท่าเดิม

Liquidation (การถูกบังคับปิด position): ถ้าราคาขยับสวนทางจนขาดทุนเกิน margin ที่วางไว้ exchange จะบังคับปิด position ทันทีเพื่อป้องกันขาดทุนเกินเงินที่มี — ยิ่ง leverage สูง ยิ่งใช้ราคาขยับน้อยก็ liquidate ได้ นี่คือนี่เหตุผลที่เล่นนี้จำกัด leverage ไว้ที่ $\leq 10x$ และมี Short Loss Buffer (ดู \$1B.3) เพื่อไว้เสมอ

1B.1 Buy-Only vs Buy-and-Sell — เปรียบเทียบ

Buy-Only Grid (Part 1A)

- ซื้อเมื่อราคาลง, ขาย (TP) เมื่อขึ้น
- ต้องการ spot หรือ perp long เท่านั้น
- ถ้าไรเมื่อราคา sideways → สูงขึ้น
- ขาดทุน unrealized เมื่อราคาลงต่ำ
- เหมาะ: long-term holder + passive income

Buy-and-Sell Grid (Bidirectional)

- Long layer: ซื้อต่ำขายสูง (ด้านล่าง)
- Short layer: ขายสูงซื้อคืนต่ำ (ด้านบน)
- ต้องการ perpetual futures (margin)
- ถ้าไรทั้งขาขึ้น ขาลง และ sideways
- เหมาะ: choppy market, futures trader

ข้อสำคัญ: Buy-and-Sell Grid **ไม่ได้** "ดีกว่า" Buy-Only เสมอ — ใน bull market ที่ราคาขึ้นตลอด Short layer จะขาดทุนต่อเนื่อง ต้องเลือกให้เหมาะกับ regime ตลาด

1B.2 โครงสร้าง Long Layer + Short Layer

ในระบบ Bidirectional Grid เราแบ่งโซนออกเป็น 2 ส่วน:

ตัวอย่าง: BTC = \$100,000 · Zone \$90k-\$110k · Step \$2k

Long Layer (ด้านล่าง current price):

- L1: BUY \$98k → SELL \$100k (profit \$2k × Q)
- L2: BUY \$96k → SELL \$98k
- L3: BUY \$94k → SELL \$96k
- L4: BUY \$92k → SELL \$94k
- L5: BUY \$90k → SELL \$92k

Short Layer (ด้านบน current price):

- S1: SELL \$102k → BUY \$100k (profit \$2k × Q)
- S2: SELL \$104k → BUY \$102k
- S3: SELL \$106k → BUY \$104k
- S4: SELL \$108k → BUY \$106k
- S5: SELL \$110k → BUY \$108k

$$\begin{aligned}\text{กำไรต่อรอบ (แต่ละ layer)} &= Q \times \text{Step} - \text{Fees} \\ &= Q \times \$2,000 - (Q \times P \times 0.1\% \times 2)\end{aligned}$$

กลไก: ราคาแกว่งผ่าน current price ซ้ำๆ

เมื่อราคาแกว่งขึ้นจาก \$98k → \$102k: Long L1 fill (ซื้อ \$98k ขาย \$100k) + Short S1 fill (ขาย \$102k ซื้อ \$100k) = 2 รอบจากการแกว่งครั้งเดียว

นี่คือ "double capture" ของ bidirectional — แต่ต้องการ margin สำหรับทั้ง 2 layer

1B.3 Capital Allocation ทั้งสองฝั่ง

Capital ที่ต้องการ (ไม่ใช่ Leverage):

Long Layer (ซื้อ BTC ด้วยเงิน):

$$\begin{aligned}\text{Capital_long} &= Q \times \sum(P_i) \text{ สำหรับทุก Long level} \\ &= 0.01 \times (\$98k + \$96k + \$94k + \$92k + \$90k) \\ &= 0.01 \times \$470k = \$4,700\end{aligned}$$

Short Layer (ต้องการ margin สำหรับ short position):

$$\begin{aligned}\text{Capital_short} &= Q \times \sum(P_i) \times \text{margin_rate} \text{ (ใช้ 10x leverage ตามกฎ leverage cap ด้านล่าง)} \\ &= 0.01 \times (\$102k + \$104k + \$106k + \$108k + \$110k) \times 10\% \\ &= 0.01 \times \$530k \times 10\% = \$530 \\ &\text{(ถ้าเสี่ยงกว่านี้ใช้ 20x leverage → margin เหลือแค่ \$265 – แต่ขีดกฎ } \leq 10x \text{ จึงไม่ใช่เป็นเลขหลัก)}\end{aligned}$$

Capital รวม (margin-only – ตัวเลขนี้ยังไม่ใช้ทุนที่ปลอดภัย):

$$\text{Total_margin} = \$4,700 \text{ (long)} + \$530 \text{ (short)} = \$5,230$$

△ margin คือ "เงินค้ำ" ไม่ใช่ "เพดานขาดทุน" – ทุนจริงต้องเผื่อขาดทุนกรณี short โดน SL ทุก order ด้วย:

$$\text{Short Loss Buffer} = \sum Q \times (\text{SL} - \text{Entry}_j)$$

$$\begin{aligned}\text{SL} &= \text{zone_top} + 5\% = \$110k \times 1.05 = \$115.5k \\ \sum(\text{SL} - \text{Entry}) &= \$13.5k + \$11.5k + \$9.5k + \$7.5k + \$5.5k = \$47.5k \\ \text{Buffer} &= 0.01 \times \$47.5k = \$475\end{aligned}$$

$$\begin{aligned}\text{Total (conservative)} &= C_long + M_short + \text{Short Loss Buffer} \\ &\quad + \text{Fee/Funding/Slippage Reserve} \\ &\approx \$4,700 + \$530 + \$475 + \sim\$60 \approx \$5,765\end{aligned}$$

เปรียบกับ Buy-Only: \$4,700 → Bidirectional ต้องการเพิ่ม ~23% (conservative total)

(ไม่ใช่ +11.3% แบบที่เห็นจาก margin-only \$5,230 อย่างเดียว – ต้องรวม Short Loss Buffer + reserve ด้วย)

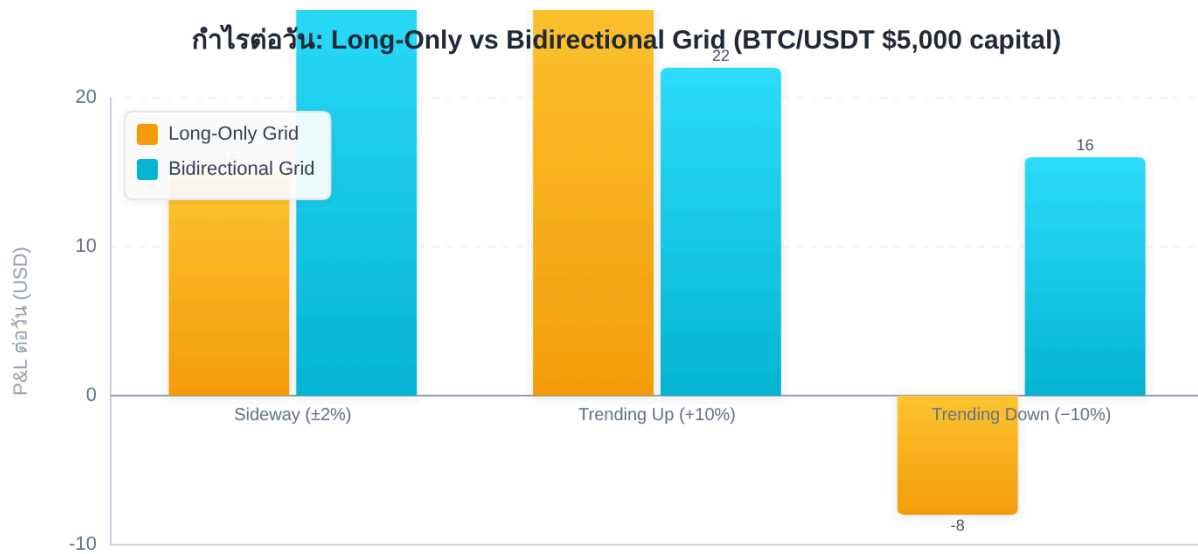
แลกกับรอบเพิ่มอีก 2× ในช่วง sideways

Leverage Warning — Short Layer

Short layer ใช้ leverage — ถ้าราคาพุ่งขึ้นแรง (short squeeze) short position มี unrealized loss สูง

กฎ: Short layer leverage $\leq 10\times$ เสมอ | ตั้ง stop-loss สำหรับ short layer (ต่างจาก long layer ที่ไม่มี SL ใน Close System)

ทำไม short ถึงมี SL แต่ long ไม่มี? เพราะ long สามารถถือ BTC ได้โดยไม่มีต้นทุนเพิ่ม แต่ short position มี funding cost ต่อเนื่องและ unlimited loss ถ้าราคาพุ่ง



Sideway: Bidirectional ดีกว่า / Up Trend: Long-only ได้กำไรเพิ่มจาก inventory / Down: Bidirectional ป้องกันได้ด้วย Short Layer

Bidirectional Grid ทำกำไรได้ทั้ง 3 สถานการณ์ที่ราคายังอยู่ใน zone — Long-only ขาดทุนเมื่อราคาลง แต่ Bidirectional ยังได้กำไรจาก Short Layer (ถ้า trend แรงจนหลุดขอบ ดูตาราง 1B.5 — ทั้งคู่แย่)

1B.4 Funding Rate — ต้นทุนซ้อนของ Short Layer

Perpetual futures มี funding rate ที่จ่ายทุก 8 ชั่วโมง — ถ้า funding rate เป็นบวก ฝั่ง short ได้รับเงิน ถ้าเป็นลบ ฝั่ง short จ่ายเงิน (ทำไม funding ถึงมีและทำไมพลิกกลับได้ — ดูกลไก "ค่าเช่าความห่าง" ใน Part 1C §1C.3; เลข $\times 3$ /วัน ด้านล่างเป็นค่าของ exchange ที่เก็บทุก 8 ชม. ไม่ใช่กฎธรรมชาติ)

Funding Cost/Income (Short Layer):

$$\begin{aligned}\text{funding_pnl} &= \text{short_notional} \times \text{funding_rate} \times (24/8) \text{ per day} \\ &= (0.01 \text{ BTC} \times \$106\text{k avg}) \times \text{rate} \times 3 \quad \# \\ \text{avg}(\$102\text{k} \dots \$110\text{k}) &= \$106\text{k} \\ &= \$1,060 \times \text{rate_per_8h} \times 3\end{aligned}$$

ตัวอย่าง:

$$\begin{aligned}\text{Funding rate} &= +0.05\%/8\text{h} \text{ (บวก} \rightarrow \text{short ได้รับเงิน)} \\ \text{Funding income} &= \$1,060 \times 0.05\% \times 3 = \$1.59/\text{วัน} \checkmark\end{aligned}$$

$$\begin{aligned}\text{Funding rate} &= -0.02\%/8\text{h} \text{ (ลบ} \rightarrow \text{short จ่ายเงิน)} \\ \text{Funding cost} &= \$1,060 \times 0.02\% \times 3 = \$0.636/\text{วัน} \times\end{aligned}$$

$$\begin{aligned}\text{กำไร grid short layer} &= (\text{รอบ} \times \text{profit/รอบ}) - \text{funding_cost} \\ &\approx 1 \text{ รอบ} \times (\$20 - \text{fee}) - \$0.636/\text{วัน} \\ &= \$18 - \$0.64 \approx \$17.36/\text{วัน} \text{ (ยังกำไรอยู่)}\end{aligned}$$

Funding Rate Adaptation (จาก Part 7)

ถ้า funding rate สูงมากเป็นบวก (เช่น $0.3\%/8\text{h}$) $\rightarrow$ Short layer ทำกำไรจาก funding เพิ่ม
นอกจาก grid cycles $\rightarrow$ เพิ่ม size short layer

ถ้า funding rate ลบมาก $\rightarrow$ Short layer เสีย $\rightarrow$ ลด size short layer หรือปิดชั่วคราว

ดูรายละเอียดใน [Part 7: Funding Rate Adaptive Grid](#)

1B.5 เมื่อไหร่ Bidirectional ดีกว่า Buy-Only

สถานการณ์	Buy-Only	Bidirectional	แนะนำ
Sideway choppy ($H < 0.45$)	ดี	ดีกว่า ($2\times$ cycles)	Bidirectional
Bull trend ช้า ($H \approx 0.52$)	ดี (inventory มีมูลค่า)	กลาง (short layer drag)	Buy-Only
Bull trend แรง ($H > 0.58$)	กลาง (grid หยุด)	แย่ (short layer ขาดทุน)	ทั้งคู่ไม่ดี → หยุด grid
Bear trend (ราคาลงต่อเนื่อง)	แย่ (inventory ขาดทุน)	กลาง (short layer ได้)	Bidirectional (short captures)
Funding rate + สูง ($>0.1\%$)	ไม่ได้ประโยชน์	ดีมาก (short gets funding)	Bidirectional

1B.6 Close System สำหรับ Bidirectional

Close System Bidirectional:

Long side: วาง capital ถึง zone_bottom (เหมือน Part 1A)

$C_{\text{long_floor}} = \sum(Q \times P_i)$ สำหรับ Long levels ถึง floor

Short side: ใช้ different approach – short layer มี SL

SL ของ short layer = zone_top + buffer (เช่น zone_top + 5%)

$\text{Capital_short} = Q \times N_{\text{short}} \times P_{\text{avg}} \times \text{margin_rate}$

$\text{Maximum loss (short side)} = (\text{SL_price} - P_{\text{entry}}) \times Q \times N_{\text{short}}$

กฎ Close System Bidirectional:

1. Long layer: ไม่มี SL (Close System ปกติ)
2. Short layer: มี SL ที่ zone_top + 5% (เพื่อป้องกัน short squeeze)
3. Short capital $\leq$ 20% ของ long capital เสมอ
(เพื่อให้ long side dominate ใน bull scenario)

1B.7 Running Example — Bidirectional BTC/USDT

Running Example: BTC Bidirectional Grid

Setup: BTC = \$100k · Zone \$92k–\$108k · Step \$2k · Q = 0.01 BTC · Leverage short = 10×

Layer	ระดับ	ราคา	Action	Capital
Long	L1	\$98k–\$100k	BUY \$98k / SELL \$100k	\$980
	L2	\$96k–\$98k	BUY \$96k / SELL \$98k	\$960
	L3	\$94k–\$96k	BUY \$94k / SELL \$96k	\$940
	L4	\$92k–\$94k	BUY \$92k / SELL \$94k	\$920
Short	S1	\$100k–\$102k	SELL \$102k / BUY \$100k	\$102 (10× margin)
	S2	\$102k–\$104k	SELL \$104k / BUY \$102k	\$104
	S3	\$104k–\$106k	SELL \$106k / BUY \$104k	\$106
	S4	\$106k–\$108k	SELL \$108k / BUY \$106k	\$108

Capital margin-only: Long \$3,800 + Short margin \$420 = **\$4,220** (ตัวเลขนี้ยังไม่ใช้ทุน
ปลอดภัย — ดู §1B.3)

SL ของ Short Layer: $\$108k \times 1.05 = \$113,400$ (ถ้า BTC พุ่งเกินนี้ → ปิด short ทั้งหมด)

Short Loss Buffer: $\Sigma Q \times (SL - \text{Entry}) = 0.01 \times (\$11.4k + \$9.4k + \$7.4k + \$5.4k) = 0.01 \times \$33.6k =$
\$336

Capital รวม (conservative): $\$4,220 + \$336 \approx$ **\$4,560** + fee/funding reserve เล็กน้อย

Daily estimate (mechanical illustration — ไม่ใช่ expected return): 3 รอบ (2 long + 1
short) $\times \$18 \text{ avg} = \$54/\text{วัน} \approx 1.18\%/\text{วัน}$ บน \$4,560

เปรียบ Buy-Only เดิม (4 levels): 1.5 รอบ $\times \$18 = \$27/\text{วัน} = 0.71\%/\text{วัน}$ บน \$3,800

Bidirectional: cashflow +100% (2 เท่า จากรอบที่เพิ่มขึ้นเป็น 3 รอบ) ด้วย capital เพิ่มขึ้น +20% (conservative) — แต่ขึ้นกับ market regime

1B.8 Pseudocode — Bidirectional Grid Manager

```
class BidirectionalGrid:
    def __init__(self, symbol, zone_low, zone_high, step, q_long,
q_short,
                    leverage_short=10, short_sl_buffer=0.05):
        self.zone_low = zone_low
        self.zone_high = zone_high
        self.step = step
        self.q_long = q_long      # BTC per long level
        self.q_short = q_short   # BTC per short level
        self.leverage_short = leverage_short # ต้อง ≤10× ตามกฎ (ดู
§1B.3) – ใช้ตอนคำนวณ margin จริง
        self.short_sl = zone_high * (1 + short_sl_buffer)

    def setup_orders(self, current_price):
        orders = []
        # Long Layer – levels below current price
        level = current_price - self.step
        while level >= self.zone_low:
            orders.append({"side": "BUY", "price": level, "qty":
self.q_long,
                        "tp": level + self.step, "sl": None}) # no
SL (Close System)
            level -= self.step

        # Short Layer – levels above current price
        level = current_price + self.step
        while level <= self.zone_high:
            orders.append({"side": "SELL", "price": level, "qty":
self.q_short,
                        "tp": level - self.step, "sl":
self.short_sl}) # SL required
            level += self.step

        return orders

    def check_short_sl(self, current_price):
        """ตรวจ short squeeze – ถ้าราคาพุ่งเกิน SL ปิดทุก short"""
        if current_price > self.short_sl:
```

```
        return "CLOSE_ALL_SHORTS"  
    return "OK"
```

1B.9 แบบฝึกหัด

แบบฝึกหัด Part 1B

- ▶ ข้อ 1: Bidirectional BTC \$100k, Zone \$95k–\$105k, Step \$2k, $Q=0.01$ BTC — กี่รอบ/วันถ้า $ATR = \$3k$?
- ▶ ข้อ 2: ทำไม Short Layer ถึงต้องมี SL แต่ Long Layer ไม่ต้อง?
- ▶ ข้อ 3: Funding rate = $-0.05\%/8h$ ส่งผลต่อ Short Layer อย่างไร? ควรทำอย่างไร?
- ▶ ข้อ 4: ตลาด Bull Run ชัด ($H=0.65$, $ADX=40$) — ควรทำ Bidirectional หรือ Buy-Only?

Grid Hedge — Spot + Perp Delta-Neutral

ป้องกัน Inventory Risk · Funding Rate Income · Cash & Carry + Grid · Hedge Ratio

อ้างอิง: [Arbitrage Series](#) (Funding Rate, Spot-Perp Basis)

แก่นของ PART นี้

Grid Hedge คือการใช้ Perp Short ป้องกัน inventory risk ของ Spot Grid — ถือ spot BTC เพื่อ grid trading แต่ short perp เท่าๆ กัน $\rightarrow$ net delta ≈ 0 $\rightarrow$ กำไรจาก grid cashflow + funding rate เท่านั้น (ไม่สนใจ BTC price direction)

Net P&L = Grid Cashflow + Funding Income - (Spot P&L + Perp P&L) $\approx$ Grid + Funding [ถ้า hedge สมบูรณ์]

1C.1 ทำไมต้อง Hedge Spot Grid

ปัญหาของ Buy-Only Spot Grid: เมื่อราคาลงมาก grid ซื้อ BTC ทั้งหมด → ถือ BTC ที่มีมูลค่าน้อยกว่าที่ซื้อ
→ unrealized loss สูง

Spot Grid (ไม่ Hedge)

BTC ลง 20%

→ Inventory loss: $-20\% \times \text{Capital}$

→ Grid cashflow: +\$450

Net: ขาดทุน

+

Perp Short Hedge

BTC ลง 20%

→ Short profit: $+20\% \times \text{Notional}$

→ Funding received: +\$7.5/วัน (ดูตัวอย่างเต็มใน §1C.3)

Net: กำไร

=

Hedged Grid

BTC ลง 20%

→ Spot + Short = ~ 0

→ Grid cashflow: +\$450

Net: +\$450 + Funding \$7.5/วัน

1C.2 โครงสร้าง Spot Grid + Perp Short

Setup Delta-Neutral Grid:

Spot Leg (Buy-Only Grid):

ซื้อ BTC ที่ \$98k, \$96k, \$94k ... (Close System)

Capital: \$4,700 (5 levels × avg \$940)

Inventory หลัง fill ทั้งหมด: 0.05 BTC

Perp Leg (Short Hedge):

Short 0.05 BTC perpetual ที่ current price \$100k

Notional: $0.05 \times \$100k = \$5,000$

Margin (10×): \$500

→ Net delta = +0.05 BTC (spot) - 0.05 BTC (perp short) = 0

กำไรจาก Price Movement:

BTC ลง \$20k:

Spot loss = $0.05 \times -\$20k = -\$1,000$

Perp gain = $0.05 \times +\$20k = +\$1,000$

Net price P&L = \$0 ✓

กำไรที่เหลือ = Grid cashflow + Funding received

ปัญหา: Inventory เปลี่ยนตลอดเวลา

Spot grid ไม่ได้ถือ BTC คงที่ — เมื่อ buy order fill, inventory เพิ่มขึ้น; เมื่อ sell order fill, inventory ลด

ดังนั้น hedge perp ต้องปรับตาม inventory: ถ้า inventory = 0.03 BTC → short 0.03 BTC; ถ้า inventory = 0.08 BTC → short 0.08 BTC

Dynamic Hedge Rebalancing: ปรับ perp short ทุกครั้งที่ grid fill (หรือ rebalance ทุก 1 ชั่วโมง)

1C.3 Funding Rate Income จาก Perp Short

เคยสงสัยไหมว่าทำไม perp ถึงมี "funding" ในขณะที่ futures แบบมีวันหมดอายุไม่มี — และทำไมบางที่เราเป็นฝ่ายได้เงิน บางที่ต้องเป็นฝ่ายจ่าย?

Futures ปกติบังคับให้ราคาบรรจบกับ spot เองโดยธรรมชาติ เพราะถึงวันส่งมอบราคาต้องเท่ากันเป๊ะ แต่ perp ไม่มีวันหมดอายุ — exchange เลยต้องประดิษฐ์กลไกทดแทน: ทุกๆ ช่วงเวลาหนึ่ง **ฝั่งที่ผลักราคา perp ให้ห่างจาก spot ต้องจ่าย "ค่าเช่าความห่าง" ให้ฝั่งตรงข้าม** เหมือนเจ้าของบ้านเก็บค่าเช่าเพิ่มจากผู้เช่าที่แย่งกันอยู่ทำเลเดียว

ถ้าตลาดกระทือหรือร้อนอยากซื้อ perp จนราคาสูงกว่า spot ฝั่ง long คือฝ่ายที่ "อยากอยู่ทำเลนั้น" มากกว่า จึงต้องจ่ายให้ฝั่ง short — นี่คือเหตุผลที่ funding มักเป็นบวกในตลาดกระทิง (และทำไม hedge ของเราถึงมักได้ funding แถมมาฟรีๆ)

แต่ถ้าตลาดกลัวจนแห่ไปฝั่ง short แรงจะกลับทิศ: short กลายเป็นฝ่ายต้องจ่าย long แทน hedge ของเราก็จะเริ่มเสียค่าเช่าแทนที่จะได้

Funding Rate Income:

$\text{funding_income} = \text{short_notional} \times \text{funding_rate_per_settlement} \times \text{รอบเก็บ/วัน}$

$$\begin{aligned} &= (0.05 \text{ BTC} \times \$100\text{k}) \times 0.05\% \times 3 \\ &= \$5,000 \times 0.05\% \times 3 \\ &= \$7.50/\text{วัน} \end{aligned}$$

Grid cashflow:

$$1.5 \text{ รอบ/วัน} \times \$18/\text{รอบ} = \$27/\text{วัน}$$

Total daily income:

$$\$27 \text{ (grid)} + \$7.50 \text{ (funding)} = \$34.50/\text{วัน}$$

$$= 34.50 / (4,700 + 500) \times 100\% = 0.66\%/\text{วัน} \text{ บน total capital } \$5,200$$

△ mechanical illustration ภายใต้สมมติฐาน funding บวกคงที่ + fill ครบทุกรอบ
– ไม่ใช่ expected return

เทียบกับ Buy-Only (ไม่ hedge): $\$27/\text{วัน} = 0.57\%/\text{วัน}$ บน $\$4,700$

Grid Hedge เพิ่ม income 28% โดยใช้ capital เพิ่มแค่ 11% (margin)

"3 รอบ/วัน" ในสูตรข้างบนมาจาก Bybit/Binance เก็บ funding ทุก 8 ชั่วโมง — นี่คือการเก็บค่าเช่าของ exchange นั้นไม่ใช่กฎธรรมชาติ บาง exchange เก็บทุก 1 หรือ 4 ชั่วโมง (รอบเก็บ/วัน = $24 \div \text{ชั่วโมงต่อ}$)

รอบ) เช็คว่ามี exchange ที่ใช้จริงก่อนตั้งเลขนี้ในบอท อย่า hardcode 3 ไว้เฉยๆ

สิ่งหนึ่งที่ต้องจำ: ถ้า basis (ส่วนต่างราคา perp กับ spot) เริ่มกว้างผิดปกติ — ไม่ใช่สัญญาณว่ามีเงินฟรีรออยู่ แต่คือสัญญาณว่ากลไก "ค่าเช่า" กำลังไล่ราคาไม่ทัน (มักเกิดตอน volatility พุ่งแรง) ช่วงนั้นควรลด hedge ratio หรือเตรียม de-risk มากกว่าจะรีบเข้าไปเก็บ funding เพิ่ม

1C.4 Hedge Ratio — ไม่ต้อง 1:1 เสมอ

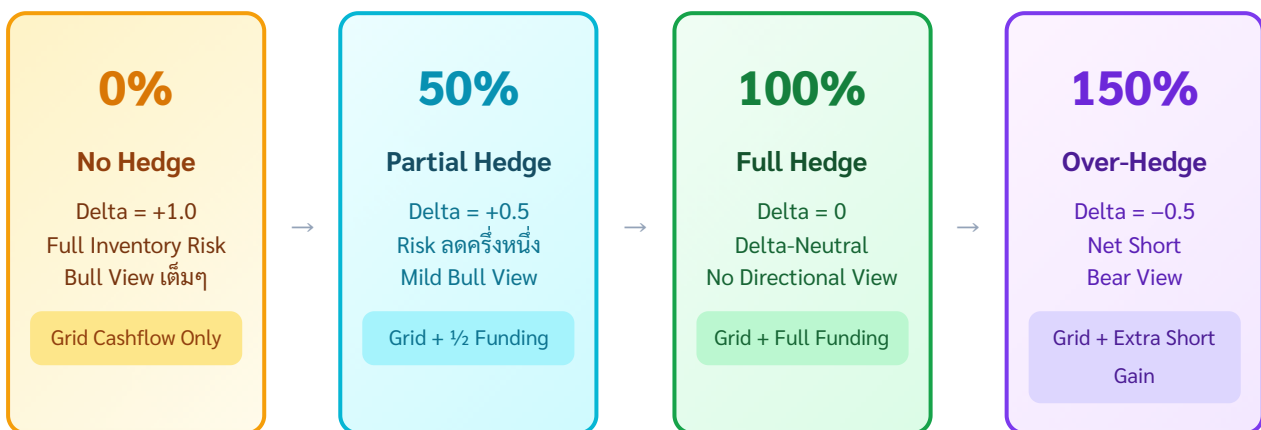
Hedge ratio $\neq 1$ เสมอ — ขึ้นอยู่กับ view และ risk tolerance (ดูตัวเลข Net Delta ของแต่ละระดับได้จาก Hedge Ratio Spectrum ด้านล่าง):

Adaptive Hedge Ratio

ปรับ hedge ratio ตาม signal:

- Funding rate $> 0.1\%/8h$ → เพิ่ม hedge ratio (short มากขึ้น → เก็บ funding มากขึ้น)
- $H > 0.50$ (เข้าเขต CAUTION ของ Part 4) และ ADX ยืนยัน uptrend → ลด hedge ratio (ถือ long มากขึ้น เพื่อ capture trend) — H สูงอย่างเดียวบอกแค่ "กำลัง trend" ไม่บอกทิศทาง ถ้า ADX ชี้ downtrend แทนให้ทำตรงข้าม (เพิ่ม hedge ratio ดูข้อ 4 ด้านล่าง)
- ถึงเต็ม% (สัดส่วน capital ที่ deploy ไปซื้อ inventory แล้วเทียบกับงบสูงสุด — อธิบายเต็มใน Part 2 §2.2) เกิน 70% → เพิ่ม hedge ratio (inventory ค้างมาก ต้องปกป้องมากขึ้น)
- Crash signal → เพิ่ม hedge ratio เป็น 100% หรือมากกว่า

HEDGE RATIO SPECTRUM — DELTA EXPOSURE



ปรับ Hedge Ratio แบบ adaptive ตาม Funding Rate, Hurst, และ Market Signal

การเลือก hedge ratio ข้างต้นแท้จริงคือการทำ arbitrage รูปแบบหนึ่งที่มีชื่อเรียกอยู่แล้ว — มาดูว่าทำไม

1C.5 Cash & Carry + Grid — Combination

Cash & Carry Arbitrage (จาก [Arbitrage Series](#)): ซื้อ Spot + Short Futures Delivery ต่างเดือน → lock in basis spread

Grid Hedge เป็นรูปแบบหนึ่งของ Cash & Carry แต่ใช้ Perpetual แทน Fixed-Term Futures:

Classical Cash & Carry:

ซื้อ 1 BTC Spot @ \$100k

Short 1 BTC Futures Dec @ \$102k (basis = \$2k)

→ ได้ \$2k ที่ expiry ไม่ว่า BTC จะเป็นเท่าไร

Grid Hedge (Perpetual Variation):

ซื้อ BTC Spot (ทยอยผ่าน grid)

Short 1 BTC Perp (dynamic, ตาม inventory)

→ ได้ grid cashflow + funding แทน basis ที่ fixed

ข้อดีของ Perpetual vs Futures:

- + ไม่มีวันหมดอายุ (ไม่ต้อง rollover)
- + Funding rate ปรับตลาด (บางที่ได้มากกว่า basis)
- Funding rate ลบได้ (ถ้า bear sentiment)
- ต้องปรับ hedge ratio อยู่เสมอ

1C.6 Running Example — Complete Grid Hedge

Running Example: BTC Spot Grid + Perp Short Hedge

Setup: Capital \$5,200 · Spot grid \$4,700 (5 levels \$98k–\$90k, $Q=0.01$ BTC $\rightarrow$ 0.05 BTC inventory) · Perp margin \$500 ($10\times = \$5k$ notional, short 0.05 BTC ตรงกับ \$1C.2/\$1C.3)

สถานการณ์	Spot P&L	Perp P&L	Grid Cashflow	Funding	Net
BTC ลง \$5k (sideway)	-\$250	+\$250	+\$40	+\$7.5	+\$47.5
BTC ขึ้น \$5k (sideway)	+\$250	-\$250	+\$35	+\$7.5	+\$42.5
BTC ลง \$20k (crash)	-\$1,000	+\$1,000	+\$80	+\$7.5	+\$87.5
BTC ขึ้น \$20k (pump)	+\$1,000	-\$1,000	+\$25	+\$7.5	+\$32.5

สังเกต: ตารางนี้ทำไว้ทุกแถว ภายใต้สมมติฐานว่า hedge สมบูรณ์ 100% ตลอดเวลา, rebalance หันทันที, funding เป็นบวก, ไม่มี slippage และไม่โดน liquidation — เมื่อเงื่อนไขครบ ผลเชิงทิศทาง (Spot vs Perp) หักล้างกันพอดี เหลือกำไรจาก grid cashflow + funding (Grid Cashflow แต่ละแถวเป็นตัวเลขสมมติแสดงจำนวนรอบที่ต่างกันตามความผันผวนของแต่ละสถานการณ์ ไม่ได้คำนวณจากสูตรรอบ/วันแบบตรงๆ — ดูวิธีคำนวณจริงใน Part 0 §0.2)

Return (mechanical illustration under assumptions — ไม่ใช่ expected return): ~\$32.5–\$87.5/วัน (ช่วง Net ในตารางข้างบน) บน capital \$5,200 = 0.63–1.68%/วัน (ขึ้นอยู่กับ volatility)

Hedge ไม่ได้ "ลบ" ความเสี่ยง — มัน "ย้าย" ความเสี่ยง

สิ่งที่ hedge ทำจริงคือแปลง directional risk ให้กลายเป็นความเสี่ยงรูปอื่น 5 ตัวที่ต้องบริหารแทน:

- **Hedge execution risk** — spot fill แล้วแต่ perp hedge ไม่ fill / fill ช้า $\rightarrow$ ช่วงนั้น delta เปิดโล่ง
- **Funding risk** — funding พลิกเป็นลบ $\rightarrow$ จ่ายทุก 8 ชม. กัด grid cashflow (แถว funding ในตารางกลายเป็นลบ)

- **Basis risk** — spot price กับ perp mark price ไม่เท่ากัน โดยเฉพาะช่วง volatility spike → hedge หักล้างไม่พอดี
- **Margin/liquidation risk** — perp short 10x ถ้า BTC พุ่งแรง margin ไม่พอ → โดน liquidate ทั้งที่ "hedge อยู่"
- **Rebalance cost** — ทุกครั้งที่ inventory เปลี่ยนต้องปรับ hedge = fee + slippage สะสม

Net P&L (ครบทุกเทอม — ใช้แบบนี้เวลาทำบัญชีจริง):

$$\begin{aligned}
 \text{Net P\&L} = & \text{Realized Grid P\&L} \\
 & + \text{Spot Unrealized P\&L} \\
 & + \text{Perp Unrealized P\&L} \\
 & + \text{Funding P\&L} \\
 & - \text{Fees} - \text{Slippage} - \text{Rebalance Cost} - \text{Margin/Borrow Cost}
 \end{aligned}$$

ตารางด้านบนแสดงเฉพาะกรณีที่ 4 เทอมท้ายเป็นศูนย์ — เวอร์ชัน V2 ของบตนี้จะเพิ่ม hedge drift, funding break-even และ liquidation buffer เป็นเครื่องมือคุมความเสี่ยงเหล่านี้

1C.7 แบบฝึกหัด

แบบฝึกหัด Part 1C

- ▶ ข้อ 1: Spot grid inventory = 0.07 BTC · BTC = \$100k · Hedge ratio 80% — Perp short notional เท่าไหร่?
- ▶ ข้อ 2: Funding rate $-0.05\%/8h$ · Short notional \$5,000 · Grid profit \$30/วัน — Net P&L/วัน?
- ▶ ข้อ 3: ทำไม Grid Hedge ถึงดีกว่า Buy-Only ในตลาด Bear?
- ▶ ข้อ 4: เมื่อไหร่ควรเพิ่ม Hedge Ratio เกิน 100%?

Bloating Patterns + Step Pyramid

บวมบน · บวมล่าง · บวมกลาง · Dynamic Sizing · Step Pyramid vs Size Pyramid

จุดหักล้างความเชื่อเดิม: Martingale ทำได้ใน Grid เพราะรู้ทุนล่วงหน้า

แก่นของ PART นี้

Grid บวม หมายถึง inventory สะสมหนาแน่นในโซนใดโซนหนึ่ง — ไม่ใช่สิ่งผิดปกติ แต่ต้องเข้าใจ pattern และออกแบบล่วงหน้า Close System ทำให้เราสามารถ ใช้

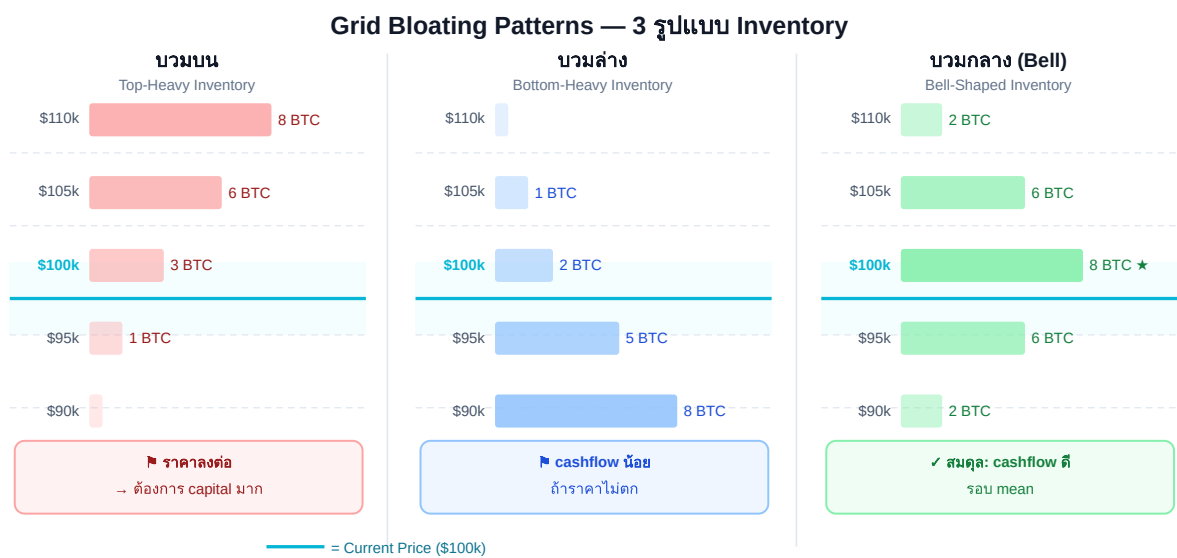
Martingale/Dynamic Sizing ได้อย่างปลอดภัย เพราะรู้ worst-case capital requirement แล้ว

2.1 Grid Bloating คืออะไร

เมื่อราคาอยู่ในโซนหนึ่งนานๆ grid ซื้อสะสม inventory ในโซนนั้น — เรียกว่า **grid บวม** ในโซนนั้น

ตัวอย่าง: BTC ลงจาก \$100k → \$90k แล้วแกว่งที่ \$90k–\$95k นาน 2 สัปดาห์

- Levels \$98k–\$92k ซื้อสะสม แต่ยังไม่ sell (ราคาไม่กลับมา)
- Levels \$92k–\$90k fill และ sell ชั่วๆ (cashflow ดี)
- ผลลัพธ์: inventory "บวม" ที่ระดับ \$90k–\$98k มี BTC ค้างอยู่มาก



ทั้ง 3 แบบมี P&L ใกล้เคียงกัน — ความต่างคือ risk distribution และ smoothness ของ cashflow

3 รูปแบบ bloating — P&L รวมใกล้เคียงกัน แต่ risk distribution และ cashflow timing ต่างกัน

2.2 บวมบน (Top-Heavy) — Inventory สะสมสูง

บวมบน (Top-Heavy)

inventory มากในโซนบน

ราคาสูง → ซื้อสูง รอขาย

เกิดเมื่อ: ราคา เคยสูง แล้วลงมา — grid ซื้อระหว่างทางลง แต่ราคาไม่กลับขึ้นไปขาย

สาเหตุ Top-Heavy:

1. เปิด grid ที่ราคาสูงกว่า mean ปัจจุบัน
2. BTC ดิ่งลง → grid ซื้อทุก level
3. ราคาหยุดแกว่งที่ต่ำกว่า zone กลาง

ผลกระทบ:

- + Grid levels ต่ำยังทำ cashflow ได้ปกติ
- Upper inventory ขาดทุน unrealized สูง
- ถ้าราคาพุ่งกลับ → ขายได้ทันที + กำไรมาก
- ถ้าราคาลงต่อ → inventory เพิ่มต่อ

แนวทางจัดการ:

- Option A: รอ (Close System — ถือไว้)
- Option B: ลด Q ของ upper levels ลง 50%
(ใช้ทุนน้อยลงในส่วนที่ "ดอย")
- Option C: เพิ่ม hedge (short perp ส่วนที่ upper inventory)

แม่ค้าผลไม้กับ grid ที่บวม

สามตัวเลือกข้างบนใช้ได้จริงทั้งสามทาง แต่ยังติดคำถามหนึ่งข้อ: **เมื่อไหร่ควรใช้ตัวไหน?**

ลองดูวิธีของคนเจอปัญหานี้มาก่อน grid trader หลายพันปี: แม่ค้าผลไม้ เวลาส้มในแผงค้างสต็อกเยอะ แม่ค้าไม่เคยปิดร้านหนี และไม่เคยรับซื้อส้มเพิ่มในราคาเดิมด้วย — เธอทำสองอย่างพร้อมกัน: **กดราคารับซื้อลงอีกหน่อย** (อยากได้ของเพิ่มก็ต้องถูกจริงๆ) และ **ยอมขายออกไว้อีกหน่อย** ยิ่งสต็อกหนัก ยิ่งกด-ยิ่งยอม พอสต็อกกลับมาปกติ ราคารับซื้อ-ขายก็เลื่อนกลับที่เดิม

นั่นแหละคือคำตอบทั้งหมด และมันบีบเลือกกฎข้อเดียว:

ยิ่งถือของเต็มมือ จุดซื้อไม้ถัดไปยิ่งต้องไกลขึ้น

วัดความ "เต็มมือ" ด้วยสัดส่วนที่มีอยู่แล้วจาก Part 1A: ทุนที่ใช้ซื้อสะสมไปแล้ว (cumulative cost ในตาราง 1A.9) เทียบกับ C_floor (ทุนเต็มถ้าซื้อครบทุก level ถึง floor) — เรียกสั้นๆ ว่า "ถึงเต็มกี่ %" แล้วสามตัวเลือกข้างบนก็เลยเป็นทางแยก กลายเป็นสเกลเดียว:

ถึงเต็ม (cumulative cost / C_floor)	ทำอะไร	ตรงกับ
ต่ำกว่า 30%	เล่นตามปกติ — ยังมีพื้นที่รับของ	Option A: รอ
30–70%	ขยับ step ฝั่งซื้อให้ห่างขึ้นตามสัดส่วนถึง (สูตรในกล่องด้านล่าง)	Option B: ลด Q / ขยาย step
เกิน 70%	หยุดรับของเพิ่ม แล้วปกป้องของที่มี	Option C: hedge (Part 1C)

สังเกตว่า **Step Pyramid** ในหัวข้อ 2.6 ใช้กฎข้อนี้จริงๆ — รวม Hurst regime (ตลาดเริ่ม trend) กับถึงเต็ม% (จากสูตรด้านบน) เป็นสูตรเดียว มันคือคำตอบเดียวกับที่งานวิจัยด้าน market making มืออาชีพคำนวณออกมาได้ (ดูอ่านต่อท้ายหัวข้อ) — จะเรียกมันว่า inventory skew หรือ "วิธีของแม่ค้าผลไม้" ก็ความหมายเดียวกัน

► สูตรสำหรับคนอยากตั้งเป็นตัวเลข (กดเปิด)

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่นนี้): Avellaneda & Stoikov, "High-Frequency Trading in a Limit Order Book" (2008) — ต้นตำรับ inventory skew ของ market maker มืออาชีพ

2.3 บวมล่าง (Bottom-Heavy) — Inventory สะสมต่ำ

บวมล่าง (Bottom-Heavy)

inventory มากในโซนล่าง
ราคาลงมาก่อน กลับมา

เกิดเมื่อ: ราคา *ดิ่งลงไกล* สะสม inventory มาก แล้วค่อยๆ *ฟื้น* — levels ล่างๆ fill ซ้ำๆ บ่อย

สาเหตุ Bottom-Heavy:

1. BTC crash ลง 30%+ จาก entry
2. Grid ซื้อ level ล่างสุดทั้งหมด (Close System สำเร็จ)
3. ราคาฟื้นช้า แกว่งที่ bottom zone

ผลกระทบ:

- + Cashflow จาก lower zone ดี (รอบเยอะ)
- + ถ้าราคาพุ่งกลับ → ขายได้ทั้งหมด กำไรใหญ่
- Capital lock ในโซนล่างมาก
- Upper zone ว่าง → ไม่ได้ใช้ชั้นบน

แนวทางจัดการ:

- Option A: เพิ่ม grid levels ใหม่ที่โซนบน
(Front-load capital จาก Zone ที่ว่าง)
- Option B: นำ inventory ล่างขาย แล้วซื้อใหม่สูงกว่า
(Grid Reset — ดูใน Part 4)

2.4 บวมกลาง (Center-Heavy / Bell) — Ideal Pattern

บวมกลาง (Bell)

inventory หนาแน่น $\pm 1\sigma$

cashflow ดีที่สุด

เกิดเมื่อ: ราคาแกว่งรอบ mean อย่างสมดุล — Bell Curve Density (Part 1A) ออกแบบมาให้ได้ pattern นี้

ทำไม Bell Curve Pattern ดีที่สุด:

1. Inventory กระจายรอบ mean
2. Fill บ่อยสุดในโซนที่ราคาอยู่นานสุด ($\pm 1\sigma$)
3. Capital ไม่กระจุกตัวที่ extreme
4. ถ้าราคาออกนอก zone $\rightarrow$ inventory น้อย (step ใหญ่ไม่ fill บ่อย)

วิธีรักษา Bell Pattern:

- ใช้ Variable Step (bell curve density จาก Part 1A)
- Rebalance ทุกสัปดาห์ถ้า inventory เบี้ยวมาก
- ดู Hurst — ถ้า H ต่ำ (mean-reverting) pattern นี้ stable

2.5 Dynamic Sizing — จุดหักล้างความเชื่อเดิม

ทำไม Martingale ปลอดภัยใน Close System

ความเชื่อเดิม: Martingale อันตราย — เพิ่ม size เรื่อยๆ จนทุนหมด

ใน Close System: เราคำนวณ capital ทั้งหมดที่ต้องใช้ถึง floor ก่อน เปิด grid — Martingale แค่กำหนดว่าจะ allocate capital อย่างไรใน range ที่คำนวณไว้แล้ว

ตัวอย่าง: มี \$10,000 · Close System บอกว่า worst case ใช้ \$8,000 | Soft Martingale กำหนดว่าจาก \$8,000 นั้นจะใช้มากขึ้นที่ level ล่าง — แต่ไม่เกิน \$8,000 รวม

Dynamic Sizing Pseudocode:

```
def allocate_grid_capital(total_capital, n_levels, style="bell"):
    """
    Allocate capital across grid levels (all styles stay within
    total_capital)
    """
    if style == "equal":
        weights = [1.0 / n_levels] * n_levels

    elif style == "top_heavy":      # บวมบน: น้ำหนักมากที่บน (i=0 = level
    ใกล้ราคาปัจจุบันที่สุด)
        weights = [1.0 / (i + 1) for i in range(n_levels)]

    elif style == "bottom_heavy":  # บวมล่าง: น้ำหนักมากที่ล่าง (i มากขึ้น =
    ลีกลงไปถึง zone_low)
        weights = [1.0 / (n_levels - i) for i in range(n_levels)]

    elif style == "bell":          # บวมกลาง: Gaussian weights
        center = n_levels // 2
        weights = [math.exp(-0.5 * ((i - center) / (n_levels/4))**2)
                    for i in range(n_levels)]

    # Normalize weights to sum to 1
    total = sum(weights)
    weights = [w / total for w in weights]
```

```
# Allocate capital
capital_per_level = [total_capital * w for w in weights]
return capital_per_level

# ตัวอย่าง: $10,000 · 5 levels · bell style (center=2, n_levels/4=1.25)
# raw weights ก่อน normalize: [0.278, 0.726, 1.000, 0.726, 0.278] ← หน้า
#                             แนนกลาง, รวม = 3.008
# หลัง normalize (หารด้วย 3.008 แล้วคูณ $10,000):
#   → [$924, $2,414, $3,324, $2,414, $924] ✓ รวม = $10,000
```

2.5.1 Overlap Guard — ระยะห่างขั้นต่ำกันไม้กระจุก

Dynamic Sizing ในหัวข้อที่แล้วคุณว่าแต่ละ level ได้ทุนเท่าไร — แต่ยังมีอีกแกนหนึ่งที่คุมได้: ไม้แต่ละไม้ อยู่ห่างกันพอไหม ปกติเรื่องนี้ไม่ต้องคิดเลย เพราะ step ที่ผูกกับ ATR (Part 3) การันตีระยะห่างให้อยู่แล้ว — แต่มีสถานการณ์หนึ่งที่เราเริ่มต้นนั้นพึงเจียบๆ: **ตอนย้าย zone ทั้งกริดโดยมี inventory ค้าง** (Grid Reset / Zone Migration — Part 4 §4.5–4.6) กริดชุดใหม่ไม่รู้จักไม้เก่า มันอาจวาง level ใหม่ทับโซนราคาที่เราไม้ ค้างอยู่แล้วพอดี ผลคือ exposure สองก้อนมากองที่ราคาเดียวกันโดยไม่ตั้งใจ

ทำไมเรื่องนี้สำคัญ: โมเดลความเสี่ยงของกริดตั้งอยู่บนการกระจายไม้ทั่ว zone — แต่ละ level คือเดิมพัน ย่อยที่อิสระว่าราคาจะแวะมาแถวนั้นแล้วดึง ไม้ที่กองกันที่ราคาเดียวคือเดิมพันก้อนใหญ่จุดเดียว ซึ่งขัดกับ สมมติฐานตั้งต้นของระบบเราเอง (เหตุผลเดียวกับกฎ 5% max notional ต่อ level ใน Part 5 — กฎนั้นคุม ขนาดต่อไม้ กฎนี้คุมความหนาแน่นเชิงพื้นที่ของไม้)

Overlap Guard: ก้อนวาง order ใหม่ทุกครั้ง

ถ้า $|P_{\text{new}} - P_{\text{nearest_open}}| < m \times \text{ATR}$ →ข้าม (skip) level นั้น ไม่วาง

P_{new} = ราคาของ level ใหม่ที่กำลังจะวาง

$P_{\text{nearest_open}}$ = ราคาของไม้ค้าง (open position) ที่อยู่ใกล้ P_{new} ที่สุด

m = สัดส่วนระยะห่างขั้นต่ำ แนะนำ 0.5–1.0

(ใช้หน่วย ATR ไม่ใช่ดอลลาร์ตายตัว เพราะ "ใกล้เกินไป"

เป็นแนวคิดสัมพันธ์กับความผันผวน — ต่อกะเดียวกับ step)

ทุนที่ไม่ได้วางเพราะโดน skip → คงเป็น cash ใน Active Zone รอวางที่ level ถัดไป ที่พ้นระยะ — ไม่ใช่เอาไปเพิ่มขนาดไม้อื่น (จะกลายเป็นย้ายปัญหากระจุกไปอีกจุด)

ต้นทุนของ guard นี้คือรอบเทรดที่หายไปเล็กน้อยตรงรอยต่อของ zone เก่า-ใหม่ ผลตอบแทนคือ margin ที่ ถูกเก็บไว้รอ level ที่ได้เปรียบจริง — ใน asset ที่ mean-revert การได้ซื้อกระจายตัวดีกว่าการได้ซื้อที่จุด เดียวเสมอ

2.6 Step Pyramid vs Size Pyramid

เมื่อตลาดมีแนวโน้ม (H เพิ่มขึ้น) — มีทางเลือก 2 แบบในการ "respond":

Strategy	วิธี	ผลต่อ Cashflow	ผลต่อ Inventory Risk
Size Pyramid	เพิ่ม Q ที่ level ลึก (martingale)	คงที่	สูงขึ้น (ซื้อมากถ้าลง)
Step Pyramid	เพิ่ม Step size แทน	ลดลงเล็กน้อย	ต่ำกว่า (fill น้อยกว่า)
Equal Weight	ไม่เปลี่ยนอะไร (baseline)	ลดลงตาม trend	สะสมเร็ว

Step Pyramid Implementation (สูตรรวม – canonical, ที่อื่นในเล่มอ้างมาที่นี่):

```
def compute_step(base_step, hurst, tank_fullness,
                 h_normal=0.50, h_max=0.58, trend_factor=0.5):
    """
    รวม 2 สัญญาณที่ทำให้ step ต้องห่างขึ้น:
    - Hurst สูง (trending, ดู Part 4) → เพิ่ม step
    - ถึงเต็ม% สูง (inventory skew, สูตรจาก §2.2) → เพิ่ม step อีกชั้น
    แทนที่ depth_multiplier แบบเดา (เวอร์ชันก่อนหน้า) ด้วยถึงเต็ม% จริง
    """
    hurst_ratio = min(1.0, max(0, (hurst - h_normal) / (h_max -
h_normal)))
    hurst_multiplier = 1 + hurst_ratio * trend_factor
    tank_multiplier = 1 + tank_fullness # ตรงกับสูตรใน §2.2: step × (1
+ ถึงเต็ม%)
    return base_step * hurst_multiplier * tank_multiplier

# ตัวอย่าง (ต่อจากตาราง 1A.9): base_step = $2,000 · Hurst = 0.55
# ถึงเต็มที่ level 3 = cumulative $3,826 / C_floor $8,460 = 0.452 (45.2%)
# hurst_ratio = (0.55-0.50)/(0.58-0.50) = 0.625
# hurst_multiplier = 1 + 0.625×0.5 = 1.3125
# tank_multiplier = 1 + 0.452 = 1.452
# step = $2,000 × 1.3125 × 1.452 ≈ $3,812 (ใหญ่ขึ้น → fill น้อยลง)
```

Step Pyramid: กำไรเท่าเดิม สบายใจกว่า

ทำไม Step Pyramid ดี: ในช่วง trend

- Size Pyramid: ซื้อมากขึ้นที่ราคาต่ำ → ถ้า trend กลับ กำไรมาก แต่ถ้า trend ต่อ → inventory massive
- Step Pyramid: ซื้อช้าลง (fill น้อยกว่า) → inventory สะสมช้า → ถ้า trend ต่อ ไม่ติดสต็อกมาก
- P&L รวมใกล้เคียงกัน แต่ **drawdown** ต่างกันมาก

2.6.1 TP ไม่ได้มีแบบเดียว — เข็มหมุดฝั่งขาย (ปรับระยะ TP ได้เหมือนปรับ step ฝั่งซื้อ)

หัวข้อที่แล้วสอนขยับ step ฝั่งซื้อเมื่อตลาดเริ่ม trend — คำถามที่ควรถามต่อทันทีคือ: แล้วฝั่งขายล่ะ? ทำไม TP ต้องอยู่ที่ level ถัดไปเสมอ? ตลอดเล่มนี้ TP = "ขายที่ level ถัดไป" (1 step) เป็นค่าที่ตั้งตายตัวมาโดยตลอด — มันคือ default ที่ดี แต่ไม่ใช่ทางเลือกเดียว:

TP Policy	กลไก	เหมาะกับ Regime	Trade-off
1-Step (default)	ขายที่ level ถัดไปเสมอ (entry + step)	H ต่ำ (<0.50 , mean-reverting) — ราคาแกว่งกลับบ่อย	รอบถึง cashflow สม่่าเสมอ กำไรต่อรอบเล็ก
TP กว้าง (Sell-Side Pyramid)	ขยาย TP distance ตาม H — สัมมาตรงกับ Step Pyramid ฝั่งซื้อใน §2.6	H เริ่มขึ้น ($0.50-0.58$, CAUTION)	รอบน้อยลง กำไรต่อรอบใหญ่ขึ้น แต่ inventory ค้างรอนานกว่า
Partial TP	ขายบางส่วนที่ level ถัดไป ปล่อยที่เหลือรอ level ถัดไปอีก	ไม่ผูก regime — เป็นทางเลือกลดความเสี่ยง all-or-nothing ล้วนๆ	ลดความเสี่ยงขายหมดเร็วเกินไป แต่ต้อง track quantity ที่เหลือต่อ level (ดูกล่องเตือน)
Trailing / Infinity	ไม่ตั้ง TP ตายตัว เลื่อนตามราคา — ดู Infinity Grid ใน Part 1D §1D.3	H สูงมาก เทรนด์ชัดเจน	เก็บ upside เทรนด์ได้เต็มที่ แต่เสีย cashflow สม่่าเสมอที่ 1-step ให้
Basket TP (รวมทุกไม้)	TP เดียวคำนวณจากต้นทุนเฉลี่ยถ่วงน้ำหนักของทุกไม้ที่ถือ + $k \times \text{ATR}$ แล้วปิดทุกไม้พร้อมกัน (รายละเอียดด้านล่าง)	โหมดักสถานการณ์หลังสะสมไม้ลึก — เฉพาะ asset ที่ mean-revert จริง ($H < 0.50$) และไม่ใช่ default	ออกจากไม้ค้างได้โดยไม่ต้องรอราคากลับถึงไม้แรก แลกกับ cashflow รายรอบที่หายไป ระหว่างถือ และ risk ที่ย้ายไปกองใน floating drawdown

ข้อควรจำ

ไม่มี TP policy ไหน "กำไรรวมมากกว่า" — ทุกแบบปรับแค่ **รูปทรง**ของ P&L (ความถี่รอบ, ขนาดกำไรต่อรอบ, drawdown, ความเสี่ยง inventory ค้าง) ไม่ใช่เพิ่ม expected value สุทธิ ยังเป็นไปตาม zero-EV theorem ใน Part 0 §0.4 เหมือนเดิม

คำเตือน: Partial TP ต้องมี accounting เพิ่ม

Framework ใน Part 9 §9.2 (GridLevel) เก็บ qty เป็นค่าเดียวต่อ level และ filled เป็น boolean — ออกแบบมาสำหรับ all-or-nothing เท่านั้น ถ้าจะใช้ Partial TP จริง ต้องขยาย GridLevel ให้ track "qty ที่ขายไปแล้ว" กับ "qty ที่เหลือ" แยกกันต่อ level ก่อน — เล่มนี้ยังไม่ implement ส่วนนั้น เป็นงานต่อยอดสำหรับใครอยากทำจริง

Basket TP — ย้ายจุดอ้างอิงจาก "ไม้แรก" มาที่ "ต้นทุนเฉลี่ยของสิ่งที่ถืออยู่จริง"

เมื่อราคาลงลึกจนสะสมไม้หลาย level TP รายไม้แบบ default มีสมมติฐานซ่อนอยู่โดยที่เราไม่ทันสังเกต: ไม้บนสุดจะปิดได้ก็ต่อเมื่อราคากลับขึ้นไปถึง entry + step ของมัน — แต่ mean reversion สัญญาแค่ว่าราคาจะกลับหาค่าเฉลี่ย ไม่เคยสัญญาว่าจะกลับไปถึงไม้แรกที่เราซื้อไว้สูงสุด ยิ่งลงลึก ไม้แรกยิ่งไกลจากค่าเฉลี่ยปัจจุบัน และความน่าจะเป็นที่จะได้ปิดมันยิ่งต่ำลงเรื่อยๆ

Basket TP แก้ที่จุดอ้างอิง: สิ่งที่เราพอร์ตถือจริงไม่ใช่ไม้หลายไม้แยกกัน แต่คือ position เดียวที่มีต้นทุนเฉลี่ยถ่วงน้ำหนัก (VWAP — Volume-Weighted Average Price) ซึ่งอยู่ต่ำกว่าไม้แรกเสมอเพราะถ่วงลงมา — ตั้ง TP เดียวจากต้นทุนนั้นแล้วปิดทุกไม้พร้อมกัน:

$$P_{\text{basket_TP}} = \frac{\sum (Q_i \times P_i)}{\sum Q_i} + k \times \text{ATR}$$

P_i, Q_i = ราคาและขนาดของไม้ที่ i ที่ถืออยู่ (นับเฉพาะไม้ที่ fill แล้ว)

เทอมแรก = ต้นทุนเฉลี่ยถ่วงน้ำหนัก (VWAP) — คำนวณใหม่ทุกครั้งที่มีไม้ fill เพิ่ม

k = ระยะกำไรเป้าหมายเป็นสัดส่วนของ ATR (ค่าเริ่มต้น 0.5 — ตรงๆเดียวกับ step ผังซื้อใน Part 3)

เหตุผลที่เรียกรอแค่ $k \times \text{ATR}$ จากจุดปัจจุบัน: ATR คือนิยามของระยะที่ราคาขยับ "เป็นปกติ" ต่อแท่งอยู่แล้ว — การตั้ง $0.5 \times \text{ATR}$ จึงเป็นเหตุการณ์ความน่าจะเป็นสูง ต่างจากการรอราคากลับไปถึงไม้แรกซึ่งเป็นเหตุการณ์ความน่าจะเป็นต่ำลงเรื่อยๆ ตามความลึกที่ลงมา

ตัวอย่าง (ตัวเลขจาก Running Example Part 3 §3.9: $\text{ATR} = \$2,800 \cdot \text{step} = \$1,500 \cdot Q = 0.009 \text{ BTC/ไม้}$):

ราคาลงจาก \$101k → \$96.5k เกี้ยวไม้ BUY 3 ไม้:

ไม้ 1: \$100,000 · ไม้ 2: \$98,500 · ไม้ 3: \$97,000 (Q เท่ากันทุกไม้)

$$\text{VWAP} = (100,000 + 98,500 + 97,000) / 3 = \$98,500$$

$$P_{\text{basket_TP}} = \$98,500 + 0.5 \times \$2,800 = \$99,900$$

เทียบ: TP รายไม้ของไม้ 1 อยู่ที่ \$100,000 + \$1,500 = \$101,500

Basket TP ปิดทุกไม้ได้ที่ \$99,900 – ต่ำกว่านั้น \$1,600

P&L ถ้าราคาได้ถึง \$99,900:

ไม้ 1: $(\$99,900 - \$100,000) \times 0.009 = -\0.90 ← ยอมขาดทุนเล็กน้อยที่ไม้บนสุด
ไม้ 2: $(\$99,900 - \$98,500) \times 0.009 = +\12.60
ไม้ 3: $(\$99,900 - \$97,000) \times 0.009 = +\26.10
รวม $= +\$37.80 \quad \checkmark (= 3 \times 0.009 \times \$1,400)$

สังเกตจากตัวอย่าง: ไม้บนสุดปิดขาดทุนนิดหน่อย — นี่ไม่ใช่ข้อบกพร่องแต่คือ trade-off ที่ตั้งใจ: แลกกำไรเต็มเม็ดของไม้บน (ที่ต้องรอราคาวิ่งอีก \$1,600 และอาจไม่มาเลย) กับการปลด inventory ทั้งหมดออกจากความเสี่ยงทันทีที่ราคาถึงในระยะ "ปกติ" ของมัน แล้วเริ่มกริดรอบใหม่จากสภาพพอร์ตสะอาด

คำเตือน: Basket TP คือถูกผสม Down-Averaging — อ่าน metric ให้เป็น

กริดที่ใช้ Basket TP โดยเนื้อแท้คือการถัวต้นทุนขาลง (down averaging) รูปแบบหนึ่ง มันใช้สมมติฐาน mean-revert *ข้าสองชั้น* — ชั้นแรกตอนทางกริด ชั้นสองตอนออก — จึงห้ามใช้กับ asset ที่ $H > 0.50$ เด็ดขาด และต้องอยู่ใต้ Close System (Part 1A) + Drawdown Ladder (Part 5) เสมอ เพราะถ้า mean ไม่กลับมา (regime เปลี่ยนถาวร) Basket TP ไม่มีคำตอบให้ — kill switch เท่านั้นที่มี

ผลข้างเคียงเชิงสถิติที่ต้องรู้: ระบบแบบนี้ realize กำไรเล็กน้อยครั้ง ส่วนขาดทุนซ่อนอยู่ใน floating drawdown จนกว่า kill switch จะทำงาน — **Profit Factor ใน backtest จึงสูงเวอร์โดยโครงสร้าง ไม่ใช่โดยฝีมือ** (ดูหมายเหตุ PF ใน Appendix และเช็คลิสต์อ่าน backtest ใน Part 9 §9.4) เวลาประเมินระบบแบบนี้ให้ดู Sharpe/Calmar และ Max Drawdown แทน

ในกรอบ zero-EV (Part 0 §0.4): Basket TP ไม่ได้เพิ่ม expected value — มันแลก "โอกาสกำไรเต็มถั่วถึงไม้แรก" กับ "กำไรเล็กที่ความน่าจะเป็นสูงและออกจากความเสี่ยงเร็วกว่า" คือปรับรูปทรงและจังหวะเวลาของ P&L เท่านั้น เหมือน TP policy อื่นๆแบบในตารางข้างบน

2.7 Running Example — Comparing 3 Bloating Patterns

 **Running Example: BTC ลงจาก \$100k → \$85k แล้วแกว่ง**

Setup: Zone \$80k–\$110k · Step \$2k · Capital \$15,000 · 3 strategies เปรียบกัน

Strategy	Inventory ที่ \$85k	Capital ที่ ใช้	Cashflow/วัน (ที่ \$83k–\$88k)	Profit ถ้า BTC กลับ \$100k
Equal Weight	0.08 BTC	\$7,200	\$36	+\$1,200
Bottom-Heavy (บวมล่าง)	0.14 BTC	\$12,320	\$54	+\$2,100
Bell Curve	0.06 BTC	\$5,400	\$28	+\$900

สังเกต: Bottom-Heavy ได้ทั้ง cashflow สูงสุดและ profit กลับมากที่สุด แต่ใช้ capital มากสุดด้วย

ถ้า BTC ลงต่อไป \$70k: Bottom-Heavy มี max loss สูงสุด | Bell Curve มี max loss ต่ำสุด

สรุป: เลือก style ตาม *conviction* — ถ้ามั่นใจ BTC จะกลับ → Bottom-Heavy | ถ้าไม่แน่ใจ → Bell Curve

2.8 แบบฝึกหัด

แบบฝึกหัด Part 2

- ▶ ข้อ 1: Grid บวมบน (Top-Heavy) มักเกิดใน market condition แบบไหน?
- ▶ ข้อ 2: ทำไม Size Pyramid ถึงเสี่ยงกว่า Step Pyramid ใน trending market?
- ▶ ข้อ 3: Bell Curve Grid density ให้ $Q_{center} = 0.02 \text{ BTC}$ · $Q_{extreme} = 0.005 \text{ BTC}$ · Step เท่ากัน \$2k — เปรียบ cashflow vs Equal Weight $Q=0.01 \text{ BTC}$ ทุก level
- ▶ ข้อ 4: ถ้า inventory ค้าง "บวมล่าง" มาก ควร Reset Grid หรือรอ?
- ▶ ข้อ 5: Hurst ปัจจุบัน $H = 0.53$ — ควรเลือก TP policy แบบไหนจาก §2.6.1 และทำไม?

Zone Sizing — ATR, Donchian, Keltner & Bootstrap

วิธีกำหนด Zone และ Step จากข้อมูลตลาด · Donchian Channel · Keltner Channel

Block Bootstrap Monte Carlo · GARCH Volatility Forecast

อ้างอิง: [Vol Series Part 2](#) (ATR/Volatility), [Statarb Ch.3](#) (OU Process)

แก่นของ PART นี้

Zone และ Step ไม่ควรตั้งแบบ "ลองดู" — ควรมาจากข้อมูล volatility ของตลาดจริง ATR บอก "ราคาเดินกี่ USD/วัน" → ใช้ตั้ง Step | Donchian Channel บอก "range ของตลาดในช่วงนี้" → ใช้ตั้ง Zone | Keltner ให้ตัวเลข step แบบเดียวกับ ATR แต่มองเป็น "แถบ" รอบราคา | Block Bootstrap (§3.5) และ GARCH (§3.6) เป็นทางเลือกขั้นสูงกว่า ใช้เมื่ออยากได้ zone/step ที่ปรับตาม volatility clustering จริง ไม่ใช่แค่ range/ค่าเฉลี่ยย้อนหลัง — ถ้ายังไม่เหมาะกับเรื่องนี้ ใช้ ATR+Donchian (§3.1–3.4) พอเพียงแล้ว

$$\text{Step} \approx 0.5 \times \text{ATR}(14) \cdot \text{Zone} = \text{Donchian}(30) \cdot \text{Keltner} = \text{EMA} \pm \text{ATR} \times k$$

3.0 ก่อนวัดอะไร — นาฬิกา 3 เรือนของ Grid

คำถามที่มีมือใหม่ทุกคนถามก่อนเปิด grid แรก: "ควรดูกราฟ timeframe ไหน?" คำตอบตรงที่สุดอาจฟังแปลก — grid ไม่ได้ดูกราฟเลย มันตื่นเมื่อราคาแตะ level ไม่ว่าจะแตะตอนไหน ไม่มีแท่งเทียนเกี่ยวข้องกับตอนที่ order จะ fill แต่รอบตัว grid มีนาฬิกาอยู่สามเรือนที่ต้องตั้งให้ถูก และเรือนแรกตั้งผิดแล้วอันตรายกว่าที่คิด:

- **เรือนวัด (measurement clock):** แท่งเทียนที่ใช้คำนวณ ATR/Donchian/Hurst — เล่มนี้ใช้ **แท่งรายวัน (1D)** เป็น default ทุกจุด เว้นแต่ระบุไว้เป็นอย่างอื่น
- **เรือนตัดสินใจ (decision clock):** ความถี่ที่ใช้ regime ซ้ำ — Part 4 §4.7 สอนไว้แล้วว่าตรวจทุก 1 ชั่วโมง (แค่ไม่เคยเรียกมันว่า "นาฬิกา" มาก่อน)
- **เรือนปฏิบัติ (execution clock):** ตอนที่ order จริงๆ ทำงาน — real-time เสมอ ไม่มี timeframe เพราะ level คือสิ่งที่ trigger ไม่ใช่แท่งเทียน

เรือนที่อันตรายที่สุดถ้าตั้งผิดคือเรือนแรก ลองดูว่าทำไม: ATR(14) บนแท่งรายวัน หมายถึง "ราคาเดินเฉลี่ยเท่าไรต่อวัน ในช่วง 14 วันที่ผ่านมา" แต่ถ้าเปิดกราฟค้ำที่ 1H แล้วกด ATR(14) ตัวเลขที่ได้คือ "ราคาเดินเฉลี่ยเท่าไรต่อ 14 ชั่วโมง" — เล็กกว่ากันประมาณ $\sqrt{24} \approx 5$ เท่า (ตัวเลขจริงในแบบฝึกหัดข้อ 5 ท้ายบทนี้) เอาไปตั้ง Step ตรงๆ (§3.1) จะได้ step เล็กผิดปกติ → grid fill ถี่เกินจริง → fee กินกำไรทั้งหมดโดยไม่รู้ตัวว่าพลาดตรงไหน เรื่องเดียวกันเกิดกับ Donchian (§3.2) และ Hurst (Part 4 §4.1) — สูตรถูก แต่ input ผิด timeframe

ปัญหาไม่ได้จบแค่ fee: ต่อให้ไม่สนใจต้นทุนเลย ตัวเลข indicator เองก็ "เพี้ยน" ขึ้นถ้าสุมตัวอย่างถี่เกินไป เพราะ noise จากกลไกตลาด (bid-ask bounce, latency) เข้าไปปนกับสัญญาณจริงมากขึ้นเรื่อยๆ ยิ่งช่วงเวลาสั้น — นี่คือเหตุผลที่ "แท่งรายวัน" ไม่ใช่ default ที่เลือกมาลอยๆ แต่มีงานวิจัยด้าน volatility estimation รองรับตรงๆ (ดูอ่านต่อท้ายหัวข้อ)

Zone Horizon	Candle Interval แนะนำ	ข้อควรระวัง
Zone หลัก (~30 วัน, Donchian(30))	1D (รายวัน)	default ของเล่นนี้ทั้งเล่ม
Micro-grid รายวัน (scalp ภายในวัน)	1H	ต้อง derive สูตร ATR/Step ใหม่เอง — ห้ามเอาตัวเลขจาก daily มาใช้ตรงๆ, เช็ค fee ให้ดีเพราะ fill ถี่กว่ามาก
Swing grid (สัปดาห์-เดือน)	4H หรือ 1D	Zone กว้างขึ้น step ต้องกว้างตาม

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่นนี้): Zhang, Mykland & Ait-Sahalia, "A Tale of Two Time Scales: Determining Integrated Volatility with Noisy High-Frequency Data," *Journal of the American Statistical Association* (2005) — พิสูจน์ว่าทำไมการวัด volatility จากข้อมูลที่สุ่มถี่เกินไปถึงเพี้ยน

สิ่งที่นาฬิกา 3 เรือนนี้ไม่ได้สอน

นี่ไม่ใช่ multi-timeframe entry timing (ดูกราฟใหญ่ยืนยันกราฟเล็กก่อนเข้าไม้) — grid ไม่ time entries อยู่แล้ว level ทำหน้าที่นั้นแทน สิ่งที่น่าฬิกา 3 เรือนตอบคือ "ตัวเลขที่ป้อนเข้าสู่สูตรมาจากช่วงเวลาไหน" เท่านั้น

3.1 ATR — ตัววัด Volatility สำหรับ Step

ATR (Average True Range) วัด "ขนาดเฉลี่ยของการเคลื่อนไหวต่อวัน" — เหมาะมากสำหรับกำหนด Step ของ grid:

$$ATR(n) = \frac{1}{n} \sum_{i=1}^n TR_i \quad \text{โดย} \quad TR_i = \max(H_i - L_i, |H_i - C_{i-1}|, |L_i - C_{i-1}|)$$

หลักการ: Step ควรเป็น "หนึ่งหน่วยของความผันผวน"

BTC/USDT — $ATR(14) = \$2,500/\text{วัน}$

Step ที่แนะนำ:

Step_tight = $0.25 \times ATR = \$625$ → fill บ่อย แต่ fee สูง

Step_moderate = $0.50 \times ATR = \$1,250$ → สมดุล (แนะนำ)

Step_wide = $1.00 \times ATR = \$2,500$ → fill น้อย cashflow ต่ำ แต่ตาม volatility พอดี

Rule of Thumb: ตั้ง Step $\approx 0.5 \times ATR(14)$

→ ราคาต้อง "เดินครึ่งวัน" เพื่อ complete 1 grid cycle

→ ไม่ fill ถี่เกินไป (fee ไม่กิน profit)

→ ไม่ช้าเกินไปจนไม่ได้ทำเงิน

ตรวจสอบ: กำไรต่อรอบ $> 3 \times$ ค่า Fee ต่อรอบ

$Q \times \text{Step} > 3 \times (Q \times P \times 0.1\% \times 2)$

$\text{Step} > 3 \times P \times 0.002 = 3 \times \$100k \times 0.002 = \$600$

$\therefore \text{Step} \geq \600 สำหรับ BTC ที่ \$100k

ทำไม ATR(14) ไม่ใช่ ATR(7) หรือ ATR(30)?

ATR(7) = ตอบสนองเร็วเกินไป — volatility spike 2 วันทำให้ Step พุ่งแล้วหัด grid ปรับถี่เกิน

ATR(14) = 2 สัปดาห์ (บนแท่งรายวัน — ดู \$3.0) = ค่ากลางที่ดี: ตาม regime ปัจจุบัน แต่ไม่ noisy

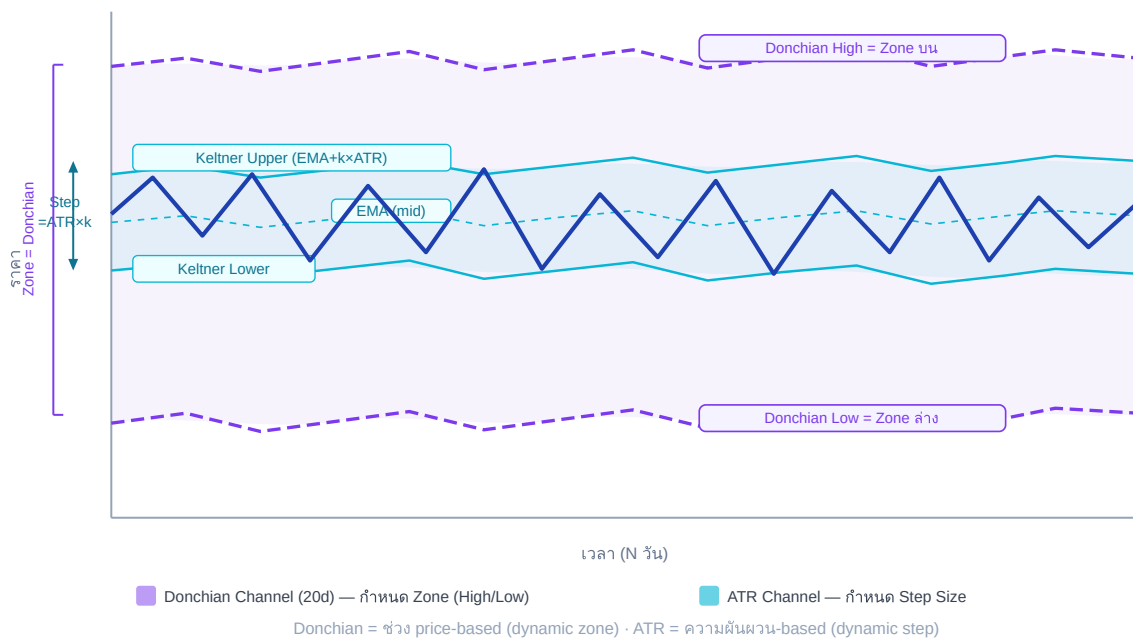
ATR(30) = ตามช้าเกินไป — ใน volatility cluster ที่ ATR พุ่งแล้ว ATR(30) ยังต่ำ = step เล็กเกิน

3.2 Donchian Channel — กำหนด Zone จาก Price Range

Donchian Channel(N) = highest high – lowest low ของ N periods ที่ผ่านมา:

$$\text{Donchian}_{\text{high}}(N) = \max_{i=0}^{N-1}(H_{t-i}) \quad \text{Donchian}_{\text{low}}(N) = \min_{i=0}^{N-1}(L_{t-i})$$

Donchian Channel (Zone) + ATR-Based Step สำหรับ Grid



Donchian Channel (กว้าง, สีม่วง) กำหนด Zone · Keltner Channel (แคบ, สีฟ้า) กำหนด Step · ราคาแกว่งภายใน Keltner มากกว่า 68% ของเวลา

Donchian Zone Sizing:

```
# Zone = Donchian(30) – range ของ 30 วันที่ผ่านมา (หน้าต่างแนะนำ ดูกล่องด้านล่าง)
zone_high = max(high[-30:]) # highest high 30 วัน
zone_low = min(low[-30:]) # lowest low 30 วัน
```

ตัวอย่าง BTC:

```
30-day high = $108,000
30-day low = $91,000
Zone width = $17,000
```

```
# เพิ่ม buffer 5% ต่อข้าง (ไม่ใช่ 10%) เพื่อรองรับ breakout
zone_high_buffered = zone_high * 1.05 # $113,400
```

```
zone_low_buffered = zone_low * 0.95 # $86,450
```

```
# ทดสอบ: zone กว้างพอไหม?
```

```
min_zone_width = ATR(14) * 6 # อย่างน้อย 6 ATR = 6 × $2,500 = $15,000
```

```
if zone_width < min_zone_width:
```

```
    zone_high = current_price * 1.08 # ขยายเป็น ±8%
```

```
    zone_low = current_price * 0.92
```

Donchian(20) vs Donchian(30) vs Donchian(60)

- **Donchian(20)** — $\approx$ 1 เดือน trading: ตาม regime ปัจจุบัน ปรับตัวเร็ว เหมาะ swing grid
- **Donchian(30)** — $\approx$ 6 สัปดาห์: สมดุล หน้าต่างยอดเยี่ยม (แนะนำ)
- **Donchian(60)** — $\approx$ 3 เดือน: zone กว้าง grid อยู่ได้นาน แต่ capital ใช้มาก

เลือกตาม holding period ที่วางแผน: grid 1 เดือน → Donchian(20); grid 3 เดือน → Donchian(60)

3.3 Keltner Channel — Step จาก ATR Bands

Keltner Channel ใช้ EMA เป็น midline + ATR เป็น band width — เหมาะสำหรับการกำหนด Step ที่ "ตาม volatility อย่าง adaptive":

$$KC_{upper} = EMA(20) + k \times ATR(14) \quad KC_{lower} = EMA(20) - k \times ATR(14)$$

Keltner-Based Step Sizing:

$k = 1.0$ (standard Keltner bandwidth = 1 ATR each side)

$KC_{upper} = EMA(20) + 1.0 \times ATR(14) = \$100k + \$2,500 = \$102,500$

$KC_{lower} = EMA(20) - 1.0 \times ATR(14) = \$100k - \$2,500 = \$97,500$

$KC_{width} = \$5,000$ (total 2 ATR range)

Step = Keltner half-width / desired levels per ATR

desired_levels_per_atr = 2 # 2 grid levels ต่อ 1 ATR ฝั่ง

step = $ATR(14) / \text{desired_levels_per_atr} = \$2,500 / 2 = \$1,250$

หรือ: $\text{step} = KC_{width} / \text{total_levels_in_keltner_zone}$
 $= \$5,000 / 4 \text{ levels} = \$1,250$

ข้อดี Keltner Step:

- + Step ปรับตาม volatility อัตโนมัติ
- + ช่วง volatility สูง: ATR สูง → step ใหญ่ → fill ช้าลง (ป้องกัน overtrading)
- + ช่วง volatility ต่ำ: ATR ต่ำ → step เล็ก → fill บ่อยขึ้น (เก็บ cashflow)

3.3.1 Adaptive Step Implementation

```
def adaptive_step(atr14, current_price, fee_rate=0.001):  
    """  
    คำนวณ step ที่ปรับตาม ATR โดยรับประกันว่ากำไรต่อรอบ > 3× fee  
    """  
    # Step พื้นฐาน = 0.5 × ATR  
    step_base = 0.5 * atr14  
  
    # ตรวจสอบ fee threshold
```

```
fee_per_cycle = current_price * fee_rate * 2 # buy + sell
min_step = 3 * fee_per_cycle # ถ้าไร ≥ 3× fee
```

```
step = max(step_base, min_step)
```

```
# Round ให้สวยงาม (nearest 100 USD สำหรับ BTC)
```

```
step = round(step / 100) * 100
```

```
return step
```

```
# ตัวอย่าง:
```

```
# ATR(14) = $2,500, P = $100k, fee = 0.1%
```

```
# step_base = $1,250
```

```
# fee_per_cycle = $100k × 0.001 × 2 = $200
```

```
# min_step = $600
```

```
# step = max($1,250, $600) = $1,250 → round(12.5)*100 = $1,200
```

```
# (Python round() ใช้ banker's rounding: round(12.5)=12 เพราะ 12 เป็นเลขคู่  
ไม่ใช่ปัด 13)
```

3.4 Combining Donchian + Keltner — Dual Channel

Method

วิธีที่ดีที่สุดในกลุ่ม ATR/Donchian/Keltner (ไม่นับ Bootstrap/GARCH ที่เป็นแนวทางขั้นสูงกว่าคนละแบบ — ดู §3.5–3.6): ใช้ทั้งสองร่วมกัน — Donchian กำหนด Zone (wide), Keltner กำหนด Step (inner) นี่คือนิยามวิธีที่ตัวอย่าง §3.9 และ validation checklist §3.8 ใช้จริงตลอดเล่ม เพราะคำนวณง่ายและเพียงพอสำหรับส่วนใหญ่ — Bootstrap/GARCH ให้ผลแม่นยำกว่าในตลาดที่ volatility เปลี่ยนเร็ว แต่ต้องแลกกับความซับซ้อนที่เพิ่มขึ้น

Parameter	วิธีคำนวณ	ตัวอย่าง BTC
Zone High	Donchian(30) high $\times$ 1.05	$\$108k \times 1.05 = \$113,400$
Zone Low	Donchian(30) low $\times$ 0.95	$\$91k \times 0.95 = \$86,450$
Step	ATR(14) $\times$ 0.5 (Keltner-based)	$\$2,500 \times 0.5 = \$1,250$
N (levels)	$\text{int}((\text{Zone_high} - \text{Zone_low}) / \text{Step})$	$(\$113,400 - \$86,450) / \$1,250 = 21.56 \rightarrow \text{int}()$ ปัดลง = 21 levels
Q (per level)	$\text{grid_capital} / (N \times \text{avg_price})$	$\$20k / (21 \times \$100k) = 0.0095 \text{ BTC} \approx 0.010$

```
def dual_channel_sizing(prices, highs, lows, current_price,
                        dc_period=30, atr_period=14,
                        grid_capital=20000):
    """
    Zone จาก Donchian(30), Step จาก ATR(14)
    """
    # Donchian Zone
    dc_high = max(highs[-dc_period:]) * 1.05
    dc_low  = min(lows[-dc_period:]) * 0.95

    # ATR Step
    true_ranges = []
    for i in range(1, atr_period + 1):
        tr = max(highs[-i] - lows[-i],
                  abs(highs[-i] - prices[-i-1])),
```

```
        abs( lows[-i] - prices[-i-1]))
    true_ranges.append(tr)
atr = sum(true_ranges) / len(true_ranges)
step = round(0.5 * atr / 100) * 100 # round to $100

# Grid Parameters
n_levels = int((dc_high - dc_low) / step)
avg_price = (dc_high + dc_low) / 2
q_per_level = grid_capital / (n_levels * avg_price)

return {
    "zone_high": dc_high,
    "zone_low": dc_low,
    "step": step,
    "n_levels": n_levels,
    "q_per_level": round(q_per_level, 4)
}
```

3.5 Block Bootstrap — Zone Sizing ที่แข็งแกร่ง

Donchian ดูแค่ high/low ในช่วงเวลา แต่ไม่บอกว่า "ความน่าจะเป็นที่ราคาจะอยู่ในช่วงนี้" คือเท่าไร
Block Bootstrap ช่วยตอบคำถามนี้

⚠ Bootstrap วัด Path Excursion ไม่ใช่ Final Price

สำหรับ Grid Trading: สิ่งที่สำคัญคือ **ราคาต่ำสุดระหว่างทาง** (max path excursion) ไม่ใช่ราคาสุดท้าย — grid ต้องอยู่รอดตลอด path ไม่ใช่แค่ตอนสิ้นสุด

ฟังก์ชันด้านล่างคำนวณทั้ง final price และ min_price_in_path — ใช้ p5_min_path (5th percentile ของราคาต่ำสุด) เป็นตัวกำหนด zone_low

แนวคิด Block Bootstrap Monte Carlo:

1. เก็บ daily return ของ BTC ย้อนหลัง 90 วัน
2. สุ่ม "blocks" ของ return (ขนาด block = 5 วัน) → เพื่อรักษา autocorrelation (ความสัมพันธ์ระหว่างวัน — วันผันผวนแรงมักตามด้วยวันผันผวนแรงต่อ ไม่ใช่สุ่มอิสระแยกวันต่อวัน สุ่มเป็นก้อนๆ จึงรักษาลักษณะนี้ไว้ได้ ต่างจากสุ่มทีละวันที่จะทำลายมันไป)
3. ประกอบ synthetic price path จาก blocks ที่สุ่มได้
4. ทำซ้ำ 1,000 ครั้ง → ได้ distribution ของ "ราคาจะไปถึงไหนใน 30 วัน"
5. ใช้ percentile 5 และ 95 เป็น zone boundaries
(ใช้ min_price_in_path สำหรับ zone_low — grid ต้องอยู่รอดตลอด path)

```
def block_bootstrap_zone(daily_returns, n_sim=1000, horizon=30,
                        block_size=5, start_price=100000):
    """
    Bootstrap zone boundaries ด้วย 95% confidence interval
    คำนวณทั้ง final price และ min_price_in_path (path excursion metric)
    สำหรับ grid trading: zone_low ควรใช้ p5_min_path ไม่ใช่ p5_final
    เพราะ grid ต้องอยู่รอดตลอด path — ไม่ใช่แค่ตอนสิ้นสุด
    """
    import random
    final_prices = []
    min_prices_in_path = [] # path excursion metric: ราคาต่ำสุดระหว่างทาง

    for _ in range(n_sim):
        price = start_price
```

```

min_price = start_price # ติดตามราคาต่ำสุดตลอด path
days = 0
while days < horizon:
    # สุ่ม block ของ returns
    start_idx = random.randint(0, len(daily_returns) -
block_size)
    block = daily_returns[start_idx : start_idx + block_size]
    for r in block:
        price *= (1 + r)
        min_price = min(min_price, price) # บันทึก worst
excursion
        days += 1
        if days >= horizon:
            break
    final_prices.append(price)
    min_prices_in_path.append(min_price) # max drawdown from start

# Zone จาก final price (95% CI ของ final distribution)
final_prices.sort()
p5_final = final_prices[int(0.05 * n_sim)] # 5th percentile final
p95_final = final_prices[int(0.95 * n_sim)] # 95th percentile
final

# Path excursion metric: zone_low ควรใช้ค่านี้แทน p5_final
# เพราะ grid ต้องอยู่รอดตลอด path ไม่ใช่แค่ตอนสิ้นสุด
min_prices_in_path.sort()
p5_min_path = min_prices_in_path[int(0.05 * n_sim)] # worst 5%
path low

return {
    "zone_low":    p5_min_path,    # ใช้ path excursion เป็น zone_low
    "zone_high":   p95_final,      # 95th percentile final price
    "p5_final":    p5_final,       # final price 5th pctl
(reference)
    "p5_min_path": p5_min_path,    # path excursion 5th pctl
(primary)
}

```

ตัวอย่าง BTC:

1,000 simulations, 30-day horizon

p5_final = \$87,500 (ราคาสุดท้าย 5th pctl – ไม่พอสำหรับ grid)

p5_min_path = \$84,200 (ราคาต่ำสุดระหว่างทาง 5th pctl – ใช้เป็น zone_low)

p95_final = \$119,800 (95th pctlile)
→ Zone: \$84k-\$120k (grid ต้องอยู่รอดถึง \$84k ระหว่างทาง)

การเลือก block_size

block_size=5 (วัน) เป็นค่าเริ่มต้นที่สมเหตุสมผลสำหรับ BTC แต่ควร test หลายค่า:

- block_size เล็ก (3-5): ล้อมรับ autocorrelation สั้น (ความผันผวนรายวัน)
- block_size ใหญ่ (10-20): ล้อมรับ momentum / trend ยาวกว่า
- วิธีที่ดีกว่า: Politis-Romano Stationary Bootstrap (ไม่ต้องกำหนด block_size ตายตัว) — ใช้ arch library: `from arch.bootstrap import StationaryBootstrap`

ทำไม Block Bootstrap ดีกว่า Simple Historical Range?

- **Simple range:** max – min ของ 30 วัน — อาจกว้างเกิน (ถ้ามี spike) หรือแคบเกิน (ถ้าช่วง low-vol)
- **Block Bootstrap:** preserve autocorrelation (volatility cluster), ให้ probabilistic range ที่ ถูกกว่า
- Zone จาก 95% CI หมายความว่า: "90% ของ scenarios ราคาอยู่ใน zone นี้" — เข้าใจง่าย จัดการ capital ได้ตาม probability

3.6 GARCH(1,1) สำหรับ Volatility-Adaptive Step

GARCH(1,1) จาก [Vol Series](#) ให้ค่า volatility forecast ที่ดีกว่า ATR ธรรมดาในช่วง volatility cluster:

ความหมายของตัวแปรในสูตร (ไม่ต้องจำได้ แค่รู้ว่าแต่ละตัวทำหน้าที่อะไร): ω (omega) = ระดับความผันผวนพื้นฐานระยะยาว (baseline) · α (alpha) = น้ำหนักที่ให้กับ "ช็อก" ของเมื่อวาน (ยิ่งราคาผันผวนแรงเมื่อวาน ยิ่งดันค่าพยากรณ์วันนี้ขึ้นมาก) · β (beta) = น้ำหนักที่ให้กับค่าพยากรณ์ของเมื่อวานเอง (ความ "จำ" ของโมเดล) — ผลรวม $\alpha + \beta$ ใกล้ 1 ยิ่งมาก แปลว่า volatility ที่พุ่งขึ้นจะค่อยๆ ลดลงช้า (persistent)

$$\sigma_t^2 = \omega + \alpha \cdot \epsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2$$

GARCH Step Sizing:

```
from vol_series_tools import fit_garch, forecast_vol

# Fit GARCH(1,1) บน 90 วันย้อนหลัง:  $\omega=1e-6$ ,  $\alpha=0.08$ ,  $\beta=0.91$ 
garch_params = fit_garch(daily_returns[-90:])

# Forecast vol สำหรับ 7 วันข้างหน้า, ใช้ day-1 forecast เป็น ATR
vol_forecast_7d = forecast_vol(garch_params, horizon=7)
sigma_daily = vol_forecast_7d[0] # = 0.025 (2.5% daily vol) ในสภาวะปกติ
price = 100000

# ประมาณ 1-day price move จาก  $\sigma$  (แบบง่าย ไม่ใช่ VaR ทางการที่มี confidence level กำหนดชัด)
atr_garch = sigma_daily * price # = $2,500

# Step = 0.5 × GARCH-ATR (เหมือน ATR method แต่ forward-looking)
step = 0.5 * atr_garch # = $1,250

# ข้อดีของ GARCH vs ATR:
# ATR: backward-looking (เฉลี่ย 14 วันที่แล้ว)
# GARCH: forward-looking (forecast vol วันพรุ่งนี้จาก cluster ปัจจุบัน)
# ช่วง volatility เฝิง spike → GARCH ตอบสนองเร็วกว่า ATR(14)
```

GARCH ตอบสนองเร็วแค่ไหน — ดูตามเวลาจริง:

วันปกติ ($\sigma=2.5\%$): $\text{step_garch} = 0.5 \times 2.5\% \times \$100k = \$1,250$
หลัง flash crash ($\sigma=6\%$): $\text{step_garch} = 0.5 \times 6\% \times \$100k = \$3,000$
สัปดาห์ต่อมา (σ ลดเหลือ 3.5%): $\text{step_garch} = \$1,750$

→ GARCH ปรับ step เร็วกว่า ATR(14) ที่ยังติด MA ของ 14 วันก่อน

```
# Practical: ถ้าไม่ fit GARCH เอง → ใช้ implied vol จาก options
# IV ของ BTC 7-day ATM option = 65% annualized
#  $\sigma_{\text{daily}} = 65\% / \sqrt{365} = 3.4\%/\text{day} \rightarrow \text{ATR}_{\text{implied}} = \$3,400$ 
```

3.6.1 ทางเลือกขั้นสูง: ตั้ง step จากคิวจริงในตลาด (ข้ามได้ถ้ายังไม่มีข้อมูล fill)

ทุกวิธีข้างบน (ATR, Donchian, Keltner, GARCH) ตอบคำถามเดียวกัน: "ตลาดกว้างแค่ไหน" แล้วตั้ง step ตามความกว้างนั้น — เหมือนวัดความกว้างถนนแล้วตั้งระยะห่างจุดจอดรถตามนั้น ถ้ามีข้อมูล order fill จริงในมือ (ไม่ใช่แค่ราคา OHLC) มีวิธีที่ตรงจุดกว่า: ตั้ง step จาก "คิวลูกค้าจริง" — ดูว่าคนตั้งราคาซื้อขายห่างจาก mid price เท่าไหร่ และโอกาสถูก fill ลดลงเร็วแค่ไหนเมื่อตั้งราคาไกลขึ้น

แนวคิดสั้นๆ

step ที่ optimal ไม่ได้ขึ้นกับแค่ "ตลาดกว้างแค่ไหน" (volatility) แต่ขึ้นกับ "ถ้าตั้งราคาไกลขึ้นอีกหน่อย โอกาสถูก fill ลดลงเร็วแค่ไหน" ด้วย — ตลาดที่ order ยัง fill ไวแม้มั้งไกล ใช้ step แคบได้โดยไม่เสียรอบ ส่วนตลาดที่ fill ช้าลงเร็วเมื่อตั้งไกล ต้องปรับ step ให้เหมาะเพื่อไม่ให้พลาดรอบ — สองปัจจัยนี้ calibrate ได้จากข้อมูล fill จริงเท่านั้น ดูราคาอย่างเดียวไม่พอ

เส้นทางนี้เหมาะกับคนที่มี tick-level data และพร้อม calibrate พารามิเตอร์เอง — ถ้ายังไม่มี ข้ามไปที่ 3.7 ได้เลยโดยไม่เสียอะไร ATR/GARCH ข้างบนใช้ได้ดีสำหรับคนส่วนใหญ่อยู่แล้ว

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่มนี้): Guéant, Lehalle & Fernandez-Tapia, "Dealing with the Inventory Risk" (2013) — สูตร closed-form สำหรับ step จาก order-arrival intensity ที่ calibrate ได้จริงบนข้อมูล crypto

3.7 Volatility Regime — ปรับ Zone/Step ตาม Regime

Zone และ Step ไม่ใช่ fixed — ควรปรับตาม volatility regime ปัจจุบัน:

Regime	ATR (BTC ตัวอย่าง)	Step แนะนำ (= step_factor x ATR ด้านล่าง)	Zone Width	N Levels
Low Vol (Consolidation)	\$1,000–\$1,500	\$350–\$525	\$8k–\$12k	16–24
Normal Vol	\$1,500–\$3,000	\$750–\$1,500	\$15k–\$25k	16–22
High Vol	\$3,000–\$5,000	\$1,800–\$3,000	\$25k–\$40k	13–14
Extreme Vol (Crash/Pump)	>\$5,000	\$3,500+	\$40k+	≤ 16

```
def vol_regime_step(atr14, current_price):  
    """  
    กำหนด step ตาม vol regime — N อยู่ในช่วง 13-24 levels แล้วแต่ regime (ดู  
    ตารางด้านบน)  
    """  
    atr_pct = atr14 / current_price  
  
    if atr_pct < 0.015:      # Low vol: ATR < 1.5%  
        regime = "low"  
        step_factor = 0.35  
    elif atr_pct < 0.030:   # Normal: ATR 1.5-3%  
        regime = "normal"  
        step_factor = 0.50  
    elif atr_pct < 0.050:   # High: ATR 3-5%  
        regime = "high"  
        step_factor = 0.60  
    else:                   # Extreme: ATR > 5%  
        regime = "extreme"  
        step_factor = 0.70
```

```
step = round(step_factor * atr14 / 100) * 100  
return regime, step
```

3.8 Practical Zone Validation Checklist

ก่อน deploy grid ทุกครั้ง ตรวจสอบ 5 ข้อนี้:

```
def validate_zone(zone_high, zone_low, step, current_price, atr14,
                  grid_capital, q_per_level, fee_rate=0.001):
    """
    ตรวจสอบ zone ก่อน deploy
    Returns: list of (check_name, passed, message)
    """
    checks = []

    # 1. Current price อยู่ใน zone
    in_zone = zone_low < current_price < zone_high
    checks.append(("Price in Zone", in_zone,
                  f"P={current_price:}, Zone=[{zone_low:}, {zone_high:}]"))

    # 2. Zone กว้างพอ ( $\geq 6$  ATR)
    zone_width = zone_high - zone_low
    wide_enough = zone_width  $\geq 6 * atr14$ 
    checks.append(("Zone Width", wide_enough,
                  f"Width={zone_width:,.0f} Min={6*atr14:,.0f} (6×ATR)"))

    # 3. Step  $\geq 3 \times$  fee per cycle
    profit_per_cycle = q_per_level * step
    fee_per_cycle = q_per_level * current_price * fee_rate * 2
    profitable = profit_per_cycle > 3 * fee_per_cycle
    checks.append(("Profitability", profitable,
                  f"Profit/cycle=${profit_per_cycle:.1f} Fee=${fee_per_cycle:.1f}"))

    # 4. Capital พอสำหรับ worst case
    n_buy_levels = (current_price - zone_low) / step
    worst_case_capital = n_buy_levels * q_per_level * current_price * 0.9
    capital_ok = grid_capital  $\geq$  worst_case_capital
    checks.append(("Capital Sufficient", capital_ok,
                  f"Grid=${grid_capital:},"))
```

```
WorstCase=${worst_case_capital:,.0f}"))
```

```
# 5. N levels  $\leq$  30 (manageable)
```

```
n_total = int(zone_width / step)
```

```
manageable = n_total <= 30
```

```
checks.append(("N Levels OK", manageable, f"N={n_total}"))
```

```
return checks
```

3.9 Running Example — BTC Zone Sizing จากศูนย์

Running Example: คำนวณ Zone + Step สำหรับ BTC/USDT

ข้อมูล: BTC = \$102,000 · ATR(14) = \$2,800 · Donchian(30): high=\$112,000 low=\$89,000

Step 1: Zone จาก Donchian(30) + Buffer

zone_high = \$112,000 × 1.05 = \$117,600 → ปัดเป็น \$118,000 (round to nearest \$1,000)

zone_low = \$89,000 × 0.95 = \$84,550 → ปัดเป็น \$85,000

zone_width = \$118,000 - \$85,000 = \$33,000

Step 2: ตรวจสอบ minimum zone width

min_width = 6 × ATR(14) = 6 × \$2,800 = \$16,800

\$33,000 > \$16,800 ✓

Step 3: Step จาก ATR

step_base = 0.5 × \$2,800 = \$1,400

fee_min = 3 × (\$102k × 0.001 × 2) = \$612

step = max(\$1,400, \$612) = \$1,400 → round to nearest \$500 = \$1,500

Step 4: N และ Q

N = \$33,000 / \$1,500 = 22 levels

grid_capital = \$20,000 (สมมติ)

avg_price = (\$118k + \$85k) / 2 = \$101,500

Q = \$20,000 / (22 × \$101,500) = 0.00896 BTC ≈ 0.009 BTC

Step 5: Validation

✓ Price in zone: \$102k ∈ [\$85k, \$118k]

✓ Zone width: \$33k > \$16.8k

✓ Profitability: 0.009 × \$1,500 = \$13.5 vs fee \$1.84/cycle → 7.3× ✓

✓ N levels: 22 ≤ 30 ✓

Step 6: Distribution ของ BUY orders (Asymmetric Zone Part 1A)

ราคาปัจจุบัน \$102k → ใกล้กลาง zone

Upper zone (\$102k → \$118k): ใส่ SELL รอ หรือไม่วาง order

Lower zone (\$102k → \$85k): วาง BUY orders 11 levels × 0.009 BTC
Capital ที่ต้องใช้ worst-case: $11 \times 0.009 \times \$93k \text{ (avg)} = \$9,207$

3.10 แบบฝึกหัด

แบบฝึกหัด Part 3

- ▶ ข้อ 1: $BTC = \$95,000 \cdot ATR(14) = \$3,200 \cdot Fee = 0.1\%$ — Step ที่แนะนำคือเท่าไร?
- ▶ ข้อ 2: Donchian(30) high=\$108k, low=\$88k · Buffer=5% · Current=\$100k — Zone boundaries คืออะไร? ราคา in-zone ไหม?
- ▶ ข้อ 3: Grid capital \$15,000 · N=20 levels · avg_price=\$100k — Q per level คือเท่าไร? กำไรต่อรอบถ้า Step=\$1,500?
- ▶ ข้อ 4 (Challenge): ATR ของ BTC ขึ้นจาก \$2,000 → \$5,000 ใน 1 สัปดาห์ (volatility spike) — ควรปรับ grid อย่างไร?
- ▶ ข้อ 5 (Challenge — นาฬิกาเรือนวัด): $BTC = \$100,000 \cdot ATR(14)$ บนแท่งรายวัน = \$2,500 (ตัวอย่างใน §3.1) แต่เมื่อเปิดกราฟค้ำที่ 1H แล้วกด ATR(14) ตรงนั้นแทน (นับจาก 14 แท่ง 1H = 14 ชั่วโมงย้อนหลัง ไม่ใช่ 14 วัน) — ถ้าราคาเดินแบบสุ่ม (ATR สเกลตาม $\sqrt{\text{เวลา}}$) ATR(14, 1H) จะได้ประมาณเท่าไร และถ้าเอาไปตั้ง Step ตรงๆ จะเกิดอะไรขึ้น?

Indicator Filter + Order Accumulation

RSI · Bollinger Band · Volume Filter · Composite Score 0–100

Order Accumulation Strategy · TWAP Grid · Partial Deploy

อ้างอิง: [Vol Part 2](#) (BB/ATR), [Statarb Ch.11](#) (State Machine)

แก่นของ PART นี้

Grid math กำหนด "ขนาด" — Indicator filter กำหนด "เวลา" ไม่ใช่ทุก indicator ที่เหมาะสม
ต้องเลือกเฉพาะ indicator ที่บอก "mean-reversion opportunity" ไม่ใช่ trend-following |
Order Accumulation ช่วยลด timing risk เมื่อ entry ไม่แน่ใจ

$$\text{Composite Score} = 30 \times \text{RSI_score} + 30 \times \text{BB_score} + 20 \times \text{Vol_score} + 20 \times \text{Hurst_score} \in [0, 100]$$

3B.1 ทำไมต้อง Indicator Filter

Grid trading ทำงานได้ดีใน "mean-reverting" market — แต่ไม่ใช่ทุกเวลาที่ตลาดเป็น mean-reverting
Indicator filter ช่วย:

- **Gate keeper:** บอกว่า "ตอนนี้ควรเปิด grid หรือไม่"
- **Size adjuster:** บอกว่า "ควร deploy เต็มหรือ partial"
- **Entry timing:** บอกว่า "เปิด grid ตอนนี้หรือรอ dip ก่อน"

สิ่งที่ Indicator Filter ไม่ควรทำ

ไม่ใช่ indicator เพื่อ "predict direction" — grid ไม่ต้องรู้ว่าราคาจะขึ้นหรือลง

ใช้ indicator เพื่อตอบคำถาม: **"ตลาดเป็น mean-reverting ระดับไหน?"** เท่านั้น

ถ้าใช้ MACD crossover หรือ Moving Average crossover เพื่อ "signal" — คุณกำลัง overlay trend-following บน grid ซึ่งขัดกับ grid philosophy

3B.2 RSI Filter — Mean-Reversion Opportunity

RSI (Relative Strength Index) วัด "overbought/oversold" — สำหรับ grid เราต้องการรู้ว่า "ราคาเบี่ยงจาก mean มากแค่ไหน":

$$RSI = 100 - \frac{100}{1 + RS} \quad RS = \frac{\text{avg gain (14 periods)}}{\text{avg loss (14 periods)}}$$

RSI Score สำหรับ Grid (ใช้ RSI ใน Mean-Reversion Context):

```
def rsi_score_for_grid(rsi_value):
    """
    Score สูง = โอกาส mean-reversion สูง (ดีสำหรับ grid entry)
    Score ต่ำ = ตลาด momentum สูง (ไม่เหมาะ grid)
    """
    # RSI extremes (< 30 หรือ > 70) = mean-reversion opportunity
    if rsi_value < 30:
        score = 70 + 100 * (30 - rsi_value) / 30 # ยิ่ง oversold ยิ่งสูง
    (70-100)
    elif rsi_value > 70:
        score = 70 + 100 * (rsi_value - 70) / 30 # ยิ่ง overbought ยิ่งสูง
    (70-100)
    else:
        # RSI 30-70 = middle zone
        # 50 = neutral (score ปานกลาง), ยิ่งห่างจาก 50 ยิ่งสูง
        distance_from_50 = abs(rsi_value - 50)
        score = 50 + distance_from_50 # 50 → score=50, 30→score=70,
        70→score=70

    return min(100, max(0, score))

# ตัวอย่าง:
# RSI=25 → score = 70 + 100×(30-25)/30 = 86.7 (oversold มาก → grid
entry ดี)
# RSI=31 → score = 50 + |31-50| = 69 ≈ 70 (ใกล้ oversold → โอกาสดี)
# RSI=50 → score = 50 (neutral → ปานกลาง)
# RSI=72 → score = 70 + 100×(72-70)/30 = 76.7 (overbought → grid sell
ดี)
```

RSI สำหรับ Entry Timing

นอกจาก filter แล้ว RSI ยังใช้กำหนด "เปิด grid ที่ขึ้นไหนก่อน" — ใช้ threshold 35/65 ที่นี้ (ต่างจาก 30/70 ใน `rsi_score_for_grid()` ด้านบน เพราะเป็นคนละงาน: 30/70 คือจุดที่ถือว่า "extreme" สำหรับให้คะแนน filter ส่วน 35/65 คือจุดตัดสินใจสำหรับ level-weighting ซึ่งควรไวกว่า/กว้างกว่าเพื่อเริ่มเอียง allocation ก่อนที่ RSI จะถึงขั้น extreme):

- RSI < 35 (oversold): เปิด lower zone ก่อน (ราคาน่าจะฟื้น) → deploy 70% ของ capital ที่ lower levels
- RSI 35–65 (neutral): deploy equal weight ตามปกติ
- RSI > 65 (overbought): เน้น upper zone / รอ pull-back ก่อน deploy lower zone

3B.3 Bollinger Band Score — Volatility Gate

Bollinger Band (BB) ให้ข้อมูล 2 อย่าง: ตำแหน่งของราคาเทียบกับ band และ width ของ band (BB Width):

$$\begin{aligned} \text{BB}_{\text{upper}} &= \text{SMA}(20) + 2\sigma_{20} & \text{BB}_{\text{lower}} &= \text{SMA}(20) - 2\sigma_{20} \\ \%B &= \frac{P - \text{BB}_{\text{lower}}}{\text{BB}_{\text{upper}} - \text{BB}_{\text{lower}}} & \text{BB Width} &= \frac{\text{BB}_{\text{upper}} - \text{BB}_{\text{lower}}}{\text{SMA}(20)} \end{aligned}$$

```
def bb_score_for_grid(price, sma20, upper, lower):
    """
    Score สูง = ราคาอยู่ใกล้ band (ใกล้ extreme = mean-reversion โอกาส)
    Score ต่ำ = ราคาอยู่กลาง (ไม่มี extreme signal)
    BB Width ต่ำ (squeeze) = ดีสำหรับ grid เพราะ vol ต่ำ fill บ่อย
    """
    # %B: ตำแหน่งของราคาใน band
    if upper == lower:
        return 50
    pct_b = (price - lower) / (upper - lower)

    # ถ้าอยู่นอก band (%B < 0 หรือ > 1) = extreme = โอกาสสูง
    if pct_b < 0:
        position_score = min(100, 60 + (0 - pct_b) * 200)
    elif pct_b > 1:
        position_score = min(100, 60 + (pct_b - 1) * 200)
    else:
        # ภายใน band: ยิ่งใกล้ขอบ ยิ่งดี
        distance_from_edge = min(pct_b, 1 - pct_b) # 0 = ที่ band edge,
        0.5 = กลาง
        position_score = 60 - distance_from_edge * 80 # 60 at edge, 20
        at center

    # BB Width score: narrow band = low vol = grid-friendly
    bb_width = (upper - lower) / sma20
    if bb_width < 0.02: # squeeze (<2%)
        width_score = 100
    elif bb_width < 0.04: # narrow
        width_score = 75
    elif bb_width < 0.08: # normal
        width_score = 50
```

```
else:                                # wide (high vol)
    width_score = 25

return 0.6 * position_score + 0.4 * width_score
```

3B.4 Volume Filter — ยืนยัน Mean-Reversion

Volume สูงผิดปกติมักเกิดพร้อม trend / breakout — volume ต่ำปกติเหมาะกับ grid มากกว่า:

```
def volume_score_for_grid(current_volume, volume_sma20):
    """
    Volume ratio = current_volume / sma_volume_20
    Score สูง = volume ปกติหรือต่ำ (grid-friendly)
    Score ต่ำ = volume spike (breakout risk)
    """
    vol_ratio = current_volume / volume_sma20

    if vol_ratio < 0.7:          # volume ต่ำผิดปกติ (อาจไม่มี liquidity)
        return 40
    elif vol_ratio < 1.0:       # volume ปกติต่ำ = ideal
        return 85
    elif vol_ratio < 1.5:       # volume ปกติสูง
        return 70
    elif vol_ratio < 2.0:       # volume สูง = ระวัง
        return 45
    elif vol_ratio < 3.0:       # volume spike = breakout warning
        return 20
    else:                       # extreme volume spike = หยุด grid
        return 0

# ตัวอย่าง:
# Daily volume ปกติ = 50,000 BTC
# วันนี้ volume = 75,000 BTC (vol_ratio = 1.5)
# → vol_ratio=1.5 ไม่ < 1.5 (เงื่อนไข strict) → ตกไปเงื่อนไขถัดไป → score = 45 (ระวัง)
```

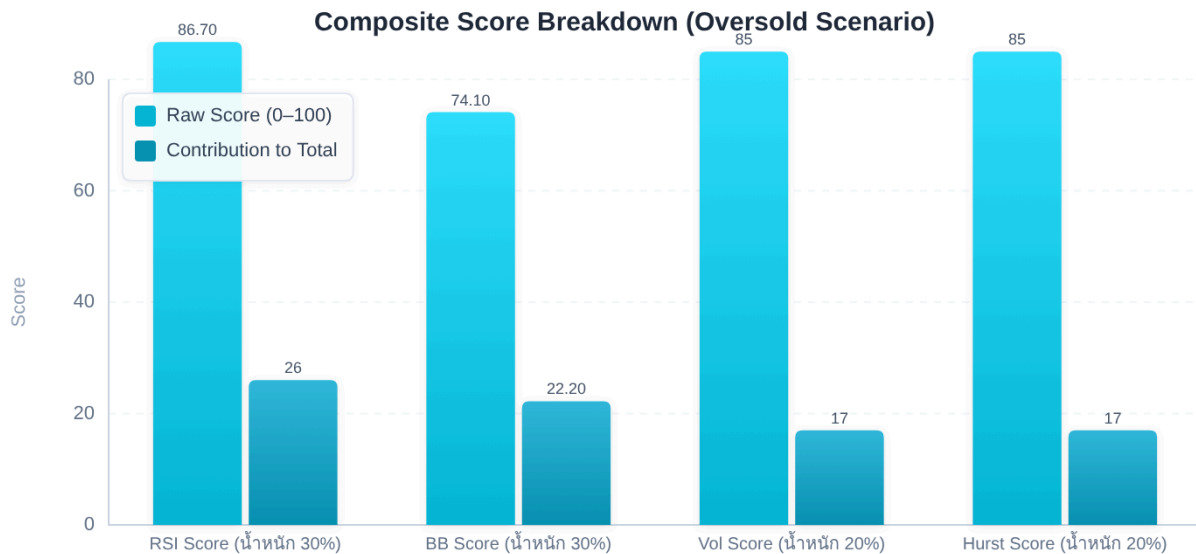
3B.5 Hurst Score — Mean-Reversion Regime Check

Hurst Exponent จาก [Statarb](#) คือ indicator หลักของ grid — ควรเป็น component สำคัญใน composite score:

```
def hurst_score_for_grid(hurst_value):  
    """  
    Hurst < 0.5 = mean-reverting = grid ดี → score สูง  
    Hurst > 0.5 = trending = grid ไม่ดี → score ต่ำ  
    """  
    if hurst_value < 0.40:          # strongly mean-reverting  
        return 100  
    elif hurst_value < 0.45:       # mean-reverting  
        return 85  
    elif hurst_value < 0.50:       # mildly mean-reverting  
        return 65  
    elif hurst_value < 0.55:       # borderline  
        return 40  
    elif hurst_value < 0.58:       # mildly trending  
        return 20  
    else:                          # trending / breakout (Part 4 GRID OFF  
territory)  
        return 0  
  
# Hurst = 0.43 → score = 85 (เปิด grid ได้ดี)  
# Hurst = 0.47 → score = 65 (เปิดได้ mean-revert อ่อน)  
# Hurst = 0.52 → score = 40 (เปิดได้ แต่ลด size)  
# Hurst = 0.62 → score = 0 (หยุด grid)
```

3B.6 Composite Score 0–100 — Grid Readiness Index

รวม 4 indicators เป็น Composite Score เพื่อดัดสินใจ:



Raw Score × น้ำหนัก = Contribution → รวม 4 contributions = Composite Score 82.2 → FULL DEPLOY (≥ 75)

Running Example: RSI=25 (oversold), BB=%B=0.04 + width 1.5% (squeeze), Vol ratio=0.85× (ปกติต่ำ), Hurst=0.43 → Composite Score = 82.2 → FULL DEPLOY

```
WEIGHTS = {
    "rsi": 0.30,    # 30% – mean-reversion extremity
    "bb": 0.30,    # 30% – position + volatility
    "vol": 0.20,   # 20% – liquidity/breakout check
    "hurst": 0.20, # 20% – regime check
}

def composite_grid_score(price, rsi, bb_upper, bb_lower, sma20,
                        volume, volume_sma20, hurst):
    scores = {
        "rsi": rsi_score_for_grid(rsi),
        "bb": bb_score_for_grid(price, sma20, bb_upper, bb_lower),
        "vol": volume_score_for_grid(volume, volume_sma20),
        "hurst": hurst_score_for_grid(hurst),
    }
    composite = sum(scores[k] * WEIGHTS[k] for k in WEIGHTS)
    return composite, scores
```

```
ACTION_TABLE = {
```

```

(75, 100): ("FULL_DEPLOY", "deploy 100% grid capital"),
(55, 75): ("PARTIAL_DEPLOY", "deploy 60% grid capital, hold 40%
standby"),
(35, 55): ("MINIMAL_DEPLOY", "deploy 30% grid capital, observe"),
(0, 35): ("WAIT", "รอ score ดีขึ้น ไม่เปิด grid"),
}

```

```
def resolve_composite_action(score):
```

```

    """single source of truth สำหรับ tier 75/55/35 – ที่อื่นในเล่ม (เช่น Part
    7B §7B.5) เรียกฟังก์ชันนี้แทนที่จะ hardcode tier ซ้ำ"""

```

```

    for (lo, hi), (name, desc) in ACTION_TABLE.items():
        if lo <= score < hi or (hi == 100 and score == 100):
            return name, desc
    return "WAIT", "รอ score ดีขึ้น ไม่เปิด grid"

```

ตัวอย่างผล:

```

RSI=32 (oversold)                → rsi_score    = 68
%B=0.1 (near lower) + width 1.5% (squeeze) → bb_score = 71.2
Vol ratio=0.9                    → vol_score   = 85
Hurst=0.46                       → hurst_score = 65   # 0.46 ไม่ < 0.45
→ ตกไปช่วง <0.50

```

```

Composite = 0.30×68 + 0.30×71.2 + 0.20×85 + 0.20×65
           = 20.4 + 21.36 + 17 + 13 = 71.8

```

Action: PARTIAL_DEPLOY (deploy 60%, hold 40% standby) – Hurst อยู่
borderline

เลยไม่ถึง FULL_DEPLOY (≥75) ทั้งที่ RSI/BB/Vol ดีทั้งหมด

3B.7 Order Accumulation — เปิด Grid แบบ TWAP

แทนที่จะเปิด grid order ทุก level ทันที ("all-in") — Order Accumulation ค่อยๆ วาง order ที่ละชั้นตามเวลา (TWAP-like — TWAP ย่อจาก Time-Weighted Average Price เทคนิคการทยอยวาง order เท่าๆ กันที่ละช่วงเวลา แทนที่จะยิงทีเดียวหมด เพื่อให้ราคาเฉลี่ยที่ได้ไม่เอียงไปทางใดทางหนึ่งมากเกินไป)

กลยุทธ์ Order Accumulation:

แบบ 1: Time-Spread (TWAP Grid)

วาง 20% ของ orders ทุก 1 ชั่วโมง → ใช้เวลา 5 ชั่วโมงกว่าจะ full deploy

ข้อดี: ถ้าราคาลงต่อหลัง open → ชั้นล่างซื้อได้ราคาดีกว่า

ข้อเสีย: เสียโอกาสถ้าราคากลับขึ้นทันที

แบบ 2: Price-Level Triggered

เปิด order ที่ level ถัดไปก็ต่อเมื่อ level ปัจจุบัน fill แล้ว

ข้อดี: ไม่ lock capital ใน pending orders

ข้อเสีย: ถ้าราคากระโดดข้าม levels → พลาด fill

แบบ 3: Score-Triggered Deployment

ตรวจ composite score ทุก 1 ชั่วโมง:

Score > 75: deploy 20% เพิ่ม (จน 100%)

Score 55-75: คง size ปัจจุบัน

Score < 35: หยุด deploy เพิ่ม

```
def order_accumulation_plan(total_capital, composite_score,
current_deployed):
```

```
    """
```

```
    คำนวณว่าควร deploy เพิ่มเท่าไร – เรียก resolve_composite_action()
    เพื่อไม่ให้ boundary (75/55/35) เปลี่ยนจาก single source of truth
    """
```

```
    action_name, _ = resolve_composite_action(composite_score)
```

```
    target_deploy = {"FULL_DEPLOY": 1.00, "PARTIAL_DEPLOY": 0.60,
                     "MINIMAL_DEPLOY": 0.30, "WAIT": 0.0}[action_name]
```

```
    additional = max(0, target_deploy - current_deployed)
```

```
    return additional * total_capital # จำนวน capital ที่ deploy เพิ่ม
```

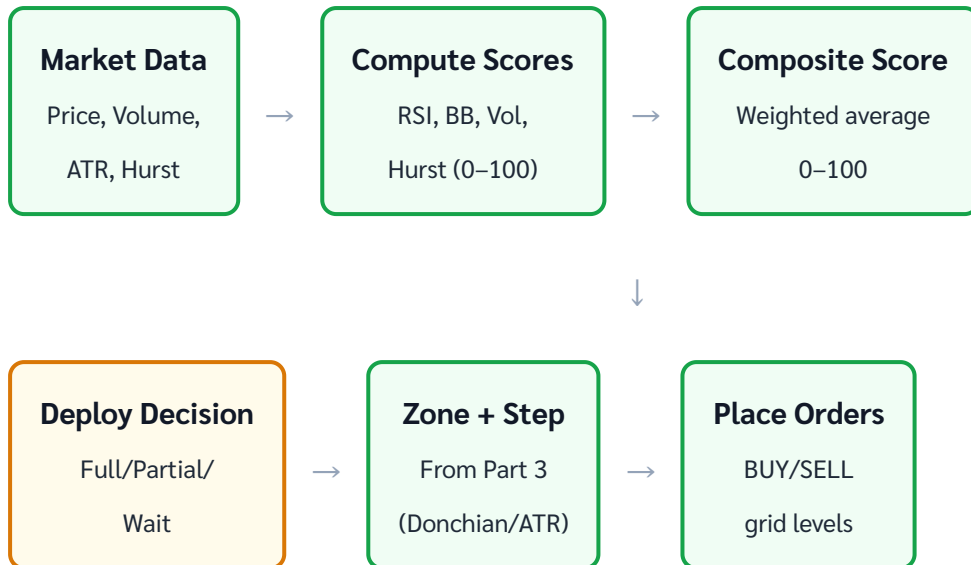
Order Accumulation ช่วยลด "Timing Risk"

ปัญหา: เปิด grid ทั้งหมดวันเดียว แต่ราคาลงต่ออีก 10% วันถัดไป → ทุก BUY order fill ที่ราคาสูงกว่าที่ควร

Order Accumulation: วาง 30% แรก → ถ้าราคาลง → วางอีก 30% ที่ระดับต่ำกว่า → average entry ดีกว่า

คล้ายกับ DCA แต่ structured: มี systematic trigger (score/time/price level) แทนที่จะเป็น random

3B.8 Filter Pipeline — Flow สมบูรณ์



```
class GridFilterPipeline:
    def __init__(self, zone_high, zone_low, step, grid_capital):
        self.zone_high = zone_high
        self.zone_low = zone_low
        self.step = step
        self.grid_capital = grid_capital
        self.deployed_pct = 0.0

    def evaluate(self, market_data):
        """
        market_data: dict ของ indicators
        Returns: action + capital_to_deploy
        """
        score, sub_scores = composite_grid_score(
            price=market_data["price"],
            rsi=market_data["rsi14"],
            bb_upper=market_data["bb_upper"],
            bb_lower=market_data["bb_lower"],
            sma20=market_data["sma20"],
            volume=market_data["volume"],
            volume_sma20=market_data["volume_sma20"],
            hurst=market_data["hurst30"]
        )
```

```
additional = order_accumulation_plan(  
    self.grid_capital, score, self.deployed_pct  
)  
  
action = "DEPLOY" if additional > 0 else "HOLD"  
self.deployed_pct += additional / self.grid_capital  
  
return {  
    "score": round(score, 1),  
    "sub_scores": sub_scores,  
    "action": action,  
    "deploy_amount": additional,  
    "total_deployed_pct": self.deployed_pct  
}
```

3B.9 Running Example — Full Pipeline

Running Example: Filter Pipeline ตัดสินใจ Deploy Grid

Setup: BTC = \$98,500 · Grid capital = \$20,000 · Zone \$85k–\$118k (เดียวกับ Part 3 §3.9) · Step \$1,500

Indicator	ค่า	Score (0–100)	น้ำหนัก	Contribution
RSI(14)	31 (oversold)	69	30%	20.7
BB Position + Width	%B=0.08, Width=3.2%	62.2	30%	18.6
Volume Ratio	0.85x (ต่ำกว่าปกติ)	85	20%	17.0
Hurst(30)	0.44 (mean-reverting)	85	20%	17.0
Composite Score			73.3 → PARTIAL DEPLOY	

Action: PARTIAL_DEPLOY (score 73.3, อยู่ในช่วง 55–75 —ไม่ถึง 75 เพราะ BB score

โดนลดจาก width 3.2% ที่เป็นแค่ "narrow" ไม่ใช่ "squeeze")

Order Accumulation Plan (deploy 60% ตาม PARTIAL_DEPLOY):

 ชั่วโมงที่ 0: deploy 60% ทันที = \$12,000

 Standby ที่เหลือ \$8,000: รอ re-evaluate score ทุกชั่วโมง

 ถ้า BB width ลดลงต่ำกว่า squeeze (<2%) หรือ Hurst ลดลง → composite ขยับเข้า

 FULL_DEPLOY (≥75) ค่อย deploy standby ที่เหลือเพิ่ม

Entry Zone Analysis:

 ราคาปัจจุบัน \$98,500 (ใกล้ lower half ของ zone)

 BUY orders เต็ม grid (reference sizing): \$97k, \$95.5k, \$94k, ..., \$85k (9 levels)

 Capital เต็มถ้า deploy 100% ทุก level (worst case all fill): $9 \times 0.009 \times \$91k \text{ avg} = \$7,371$

แต่ composite บอก PARTIAL_DEPLOY (60%) → deploy จริงแค่ 60% ×
\$7,371 ≈ \$4,423

(เปิดเฉพาะ level บนๆ ก่อน ที่เหลือรอ standby)

Capital remaining ใน Standby: \$20,000 - \$4,423 = \$15,577 (78%)

Score Report:

- ✓ RSI oversold → BTC น่าจะ rebound → lower zone เปิดก่อน
- ✓ BB near lower band → mean-reversion signal ชัด แต่ width แค
"narrow" ไม่ใช่ squeeze
- ✓ Volume ต่ำ → ไม่มี breakout risk
- ✓ Hurst 0.44 → mean-reverting (ยังไม่ถึงระดับ "strongly", ต้อง <0.40)
→ เหมาะสำหรับ grid entry แต่ deploy บางส่วนก่อน ไม่ใช่เต็ม 100%

3B.10 แบบฝึกหัด

แบบฝึกหัด Part 3B

- ▶ ข้อ 1: RSI=72, BB %B=0.95 + Width=3% (normal), Volume ratio=2.5x, Hurst=0.56 — Composite Score และ Action คืออะไร?
- ▶ ข้อ 2: ทำไม Volume spike ถึงเป็น signal ลบสำหรับ grid (ทั้งที่ volume สูงแสดง "activity")?
- ▶ ข้อ 3: Composite Score = 62 · Grid capital = \$10,000 · ปัจจุบัน deployed = 0% — ควร deploy เท่าไหร่?
- ▶ ข้อ 4 (Challenge): Order Accumulation แบบ Price-Level Triggered เหมาะกับ market condition แบบไหนที่สุด?

GRID TRADING MASTERY · PART 4

Regime Detection — เมื่อไหร่เปิด/หยุด Grid

Hurst Exponent · ADX · Bollinger Band Width · State Machine (GRID ON / CAUTION
/ GRID OFF)

Grid Reset + Zone Migration · Practical Watchdog System

อ้างอิง: [Statarb Ch.3](#) (Hurst, OU Process), [Statarb Ch.11](#) (State Machine)

แก่นของ PART นี้

Grid ทำงานได้ดีใน **mean-reverting regime** ($H < 0.5$) เท่านั้น — การรู้ว่า regime เปลี่ยนแปลงและตอบสนองทันทีคือความแตกต่างระหว่าง "grid ที่กำไรต่อเนื่อง" กับ "grid ที่ติดหล่ม" State Machine ช่วยเปลี่ยน grid behavior อัตโนมัติตาม regime

GRID ON ($H < 0.50$, $ADX < 40$ สำหรับ BTC) → **CAUTION** ($H 0.50-0.58$, $ADX 40-50$) → **GRID OFF** ($H > 0.58$, $ADX > 50$) — **BTC-calibrated** (Forex threshold 25/35 สูงเกินไปสำหรับ BTC)

ตัวเลขข้างบนคือค่าตัดสินใจแบบง่าย — ค่าที่ใช้จริงใน state machine (§4.4) มี hysteresis (จุดเข้า/ออกไม่เท่ากัน) เพื่อป้องกันการสลับสถานะถี่เกินไป

4.1 Hurst Exponent — ตัววัด Mean-Reversion

Hurst Exponent H วัดว่าราคาเป็น mean-reverting ($H < 0.5$), random walk ($H = 0.5$), หรือ trending ($H > 0.5$):

$$H = \frac{\log(R/S)}{\log(n)} \quad \text{โดย } R/S = \text{Rescaled Range}$$

Hurst Exponent สำหรับ Grid:

คำนวณด้วย R/S Analysis (rolling 30 วัน):

1. เก็บ daily return $r_i = \log(P_i / P_{i-1})$
2. R/S ของ n วัน = $(\max \text{ cumulative dev} - \min \text{ cumulative dev}) / \text{std}(\text{returns})$
3. Plot $\log(R/S)$ vs $\log(n) \rightarrow \text{slope} = H$

```
def hurst_exponent(prices, min_lag=10, max_lag=100):
    """
    คำนวณ Hurst จาก price series (R/S method)
    """
    import numpy as np

    returns = np.diff(np.log(prices))
    lags = range(min_lag, min(max_lag, len(returns) // 2))

    rs_values = []
    for lag in lags:
        # คำนวณ R/S สำหรับแต่ละ lag
        sub_series = [returns[i:i+lag] for i in range(0, len(returns)-lag, lag)]
        rs_per_sub = []
        for sub in sub_series:
            mean_sub = np.mean(sub)
            dev = np.cumsum(sub - mean_sub)
            R = np.max(dev) - np.min(dev)
            S = np.std(sub, ddof=1)
            if S > 0:
                rs_per_sub.append(R / S)
        if rs_per_sub:
            rs_values.append((lag, np.mean(rs_per_sub)))
```

```

if len(rs_values) < 2:
    return 0.5 # default random walk

log_lags = np.log([x[0] for x in rs_values])
log_rs    = np.log([x[1] for x in rs_values])
H = np.polyfit(log_lags, log_rs, 1)[0]
return H

```

ตัวอย่าง:

H = 0.42 → mean-reverting → GRID ON

H = 0.51 → random walk → CAUTION

H = 0.63 → trending → GRID OFF

⚠ คำ H ที่คำนวณได้มีความคลาดเคลื่อน — อย่าเชื่อตัวเลขนี้ด้วยแบบเป๊ะๆ

ฟังก์ชัน `hurst_exponent` ด้านบนใช้วิธี R/S (Rescaled Range) ซึ่งเมื่อข้อมูลมีแค่ 90 วัน (สั้น) ค่า H ที่ได้มักสูงเกินจริงไปประมาณ 5–10% และมีความคลาดเคลื่อนสูง (Standard Error) ประมาณ ± 0.05 – 0.10 — เท่ากับว่าตัวเลข H ที่แสดง เช่น 0.52 จริงๆ อาจอยู่ที่ไหนก็ได้ตั้งแต่ ~ 0.42 ถึง ~ 0.62 นี่คือสาเหตุที่ threshold $H=0.50/0.58$ ใน Part นี้ต้องมี hysteresis (§4.4) กันไว้ ไม่ใช่ตัดสินใจจากตัวเลขเดียวแบบทันที

กฎปฏิบัติ: $H < 0.45$ หรือ $H > 0.60$ ถือว่าชัดเจนพอเชื่อถือได้ — ช่วง 0.45–0.60 (กรอบ CAUTION จริง 0.50–0.58 กันความคลาดเคลื่อนไว้ทั้งสองข้าง) ควรรอสัญญาณเพิ่มจาก ADX/BB ก่อนตัดสินใจ ไม่ใช่เชื่อ H ตัวเดียว

สำหรับคนที่อยากได้ค่า H แม่นกว่า R/S: มีวิธี DFA (Detrended Fluctuation Analysis) ที่ bias ต่ำกว่า ใช้ library `nolds` (`pip install nolds` แล้ว `import nolds; H = nolds.dfa(log_returns)`) — ไม่จำเป็นสำหรับการเริ่มต้น แต่ถ้าจะทำจริงจึงแนะนำให้เปลี่ยนมาใช้

REGIME DETECTION — STATE MACHINE

GRID ON

mean-reverting

$H < 0.50$ (Hurst)

$ADX < 40$ (BTC)

BB width ปกติ

Full Deploy

H rising



H drops

CAUTION

borderline regime

$H = 0.50$ – 0.58

$ADX = 40$ – 50

Vol spike เริ่มขึ้น

Reduce 50% size

ADX spike



ADX drops

GRID OFF

trending / breakout

$H > 0.58$

$ADX > 50$

ราคาออกนอก Donchian

Pause — Cancel all

ตัวอย่าง Hurst timeline — สี = regime ที่ active



$H < 0.50$ = grid เต็ม · $0.50-0.58$ = ลด size · $H > 0.58$ = หยุด grid (ADX threshold ปรับสำหรับ BTC)

4.2 ADX — ยืนยัน Trend Strength

ADX (Average Directional Index) วัดความแรงของ trend โดยไม่บอกทิศทาง — ใช้ยืนยัน Hurst signal:

ADX = Smoothed MA of DX

$$DX = \frac{|+DI - -DI|}{+DI + -DI} \times 100$$

ADX Threshold สำหรับ BTC ≠ Forex/Equity

ADX 25 = "trend" ใน textbook ถูก calibrate สำหรับ Forex และ Equity — BTC มี volatility สูงกว่า 3–5x

BTC ADX มักอยู่ที่ 30–50 แม้ใน sideways market — threshold ที่เหมาะสมกว่า:

ADX Range	Forex/Equity	BTC (แนะนำ)
<25	Sideways	Sideways / Weak trend
25–40	Trend emerging	ยังคือ Sideways สำหรับ BTC
40–50	Strong trend	Trend emerging (CAUTION)
>50	Very strong	Strong trend (GRID_OFF candidate)

Backtest บน BTC data จริง (2020–2024) สนับสนุน threshold BTC-calibrated: CAUTION เริ่มที่ 40, GRID OFF ที่ 50

ADX Value	ความหมาย	Grid Action
< 15	ไม่มี trend (choppy)	GRID ON — ideal condition
15–40	trend อ่อน-ปานกลาง (BTC: ยัง sideways)	GRID ON (ระวัง upper zone)
40–50	trend เริ่มชัด	CAUTION — ลด size 50%
> 50	trend แรง	GRID OFF — pause ทั้งหมด

แนวคิดเบื้องหลังโค้ดด้านล่าง: **+DI** วัดว่าราคาขยับขึ้นมากแค่ไหนเทียบกับการขยับทั้งหมด (สัดส่วนของ "แรงขาขึ้น") ส่วน **−DI** วัดแรงขาลงในทำนองเดียวกัน — ถ้า +DI สูงกว่า −DI มากแปลว่าแรงขึ้นเหนือแรงลง (แนวโน้มขาขึ้น) ADX คือค่าเฉลี่ยของ "ส่วนต่างระหว่าง

+DI กับ -DI" ที่ทำให้เรียบขึ้น จึงบอกแค่ *ความแรงของ trend* (ไม่ว่าขึ้นหรือลง) ไม่บอกทิศทาง — ต้องดู +DI vs -DI แยกต่างหากถ้า
อยากรู้ทิศทาง

```
def compute_adx(highs, lows, closes, period=14):
    """
    คำนวณ ADX จาก OHLC data
    """
    import numpy as np

    tr_list, dm_plus, dm_minus = [], [], []
    for i in range(1, len(closes)):
        high_diff = highs[i] - highs[i-1]
        low_diff = lows[i-1] - lows[i]
        dm_plus.append(max(high_diff, 0) if high_diff > low_diff else
0)
        dm_minus.append(max(low_diff, 0) if low_diff > high_diff else
0)
        tr = max(highs[i] - lows[i],
                  abs(highs[i] - closes[i-1]),
                  abs(lows[i] - closes[i-1]))
        tr_list.append(tr)

    # Smooth ด้วย Wilder MA
    def wilder_ma(data, n):
        result = [sum(data[:n]) / n]
        for i in range(n, len(data)):
            result.append(result[-1] * (n-1)/n + data[i] / n)
        return result

    atr14 = wilder_ma(tr_list, period)
    dmp14 = wilder_ma(dm_plus, period)
    dmm14 = wilder_ma(dm_minus, period)

    di_plus = [100 * p / a for p, a in zip(dmp14, atr14)]
    di_minus = [100 * m / a for m, a in zip(dmm14, atr14)]
    dx_list = [100 * abs(p - m) / (p + m) for p, m in zip(di_plus,
di_minus)]
    adx = wilder_ma(dx_list, period)

    return adx[-1]
```

4.2.1 Normalized Slope — ทิศทางที่ Hurst กับ ADX ไม่บอก (Optional Overlay)

สัญญาณสองตัวที่ผ่านมามีจุดบอดร่วมกันหนึ่งจุด: **ไม่มีตัวไหนบอกทิศทาง** Hurst บอก *นิสัย* ของราคา (mean-revert หรือ trend) ADX บอก *ความแรง* ของ trend แบบไร้ทิศ — สถานการณ์ที่อันตรายที่สุดสำหรับ กริดฝั่ง Long คือ "ราคากำลังตั้งแรงต่อเนื่อง" ซึ่งทั้งคู่จับได้ช้า: Hurst ต้องรอข้อมูลสะสมพอจะเปลี่ยนนิสัย ADX ขึ้นแล้วก็ยังต้องไปเปิด +DI/-DI คู่อีกชั้นว่าแรงนั้นขึ้นหรือลง ระหว่างรอ กริดยังคงรับไม้สวนขาลงไปเรื่อยๆ

ทำไมขาลงถึงต้องมีสัญญาณเฉพาะ: ช่วงที่ราคาลงแรง ผลตอบแทนรายแท่งมักมี autocorrelation ชั่วคราว — ลงแล้วมีแนวโน้มลงต่อ (panic ไม่มีลิมิต ต่างจาก euphoria ที่ทยอยซื้อ) ช่วงนั้นราคาไม่ใช่ random walk แต่เป็น walk ที่มี *drift* ติดลบ และไม้ Long ทุกไม้ที่รับระหว่างนั้นมี expected value ติดลบทันที ไม่ใช่ศูนย์ — สัญญาณนี้จึงออกแบบเป็น **veto** ฝั่งซื้อฝั่งเดียว ไม่ใช่ filter สมมาตร

```
slope_norm = slope(LinReg(EMA, lookback)) / ATR
```

`slope(LinReg(...))` = ความชันของเส้นตรงที่ fit ค่า EMA ย้อนหลัง `lookback` แท่ง ด้วย Ordinary Least Squares — เครื่องมือเดียวกับที่ Part 6 ใช้หา hedge ratio แต่เปลี่ยนจากถาม spread มาถามราคา หน่วยของ slope: \$/แท่ง = ค่าประมาณของ drift เฉพาะหน้า

ATR = ความผันผวนปกติต่อแท่ง (\$/แท่ง — Part 3)

หารกันแล้วได้ค่าไร้หน่วย: "drift แรงแค่ไหนเทียบกับ noise ปกติ"

- ตีรณะเดียวกับ z-score ใน Part 6 (สัญญาณ ÷ ความผันผวนของมันเอง)
- ใช้ EMA แทนราคาดิบเพราะ EMA กรอง noise ชั้นแรกให้แล้ว LinReg เป็นชั้นที่สอง
- เกณฑ์เดียวใช้ได้ทุก asset ทุก timeframe เพราะ normalize แล้ว (slope ดิบ -\$500/แท่ง คือหายนะของเหรียญ \$100 แต่คือเรื่องจั่วของ BTC)

slope_norm	ความหมาย	Action
< -0.15	drift ลงแรงเกิน noise ชัดเจน — "มีดกำลังร่วง"	Veto ฟังซื้อ: หยุดกางกริด Long ใหม่ชั่วคราว (ไม้เดิม + TP เดิมคงไว้ ไม่แตะ)
-0.15 ... +0.15	drift เล็กเทียบ noise — ยังถือว่าแกว่งปกติ	Grid ทำงานตาม State Machine ปกติ
> +0.15	drift ขึ้นแรง	ไม่ต้อง veto — แต่พิจารณา Zone Migration (§4.6) เพราะราคาอาจกำลังหนีขึ้นจาก zone

threshold ± 0.15 เป็นจุดตั้งต้นที่ calibrate กับ BTC รายวัน (สอดคล้องกับ threshold BTC ใน §4.2) — asset อื่น/timeframe อื่นควร backtest หาเกณฑ์ของตัวเอง หลักคือเลือกค่าที่ veto เฉพาะช่วง crash จริง ไม่ veto ถิ่น grid ไม่ได้ทำงาน

```
def slope_veto(ema_values, atr, lookback=5, threshold=-0.15):
    """
    Veto ฟังซื้อเมื่อ drift ลงแรงเกิน noise — ใช้เป็น overlay เสริม
    state machine (§4.4) ไม่ใช่ vote ที่ 4
    """
    import numpy as np
    y = np.array(ema_values[-lookback:], dtype=float)
    x = np.arange(lookback, dtype=float)
    slope = np.polyfit(x, y, 1)[0]      # $/แท่ง
    slope_norm = slope / atr            # ไร่หน่วย
    return slope_norm < threshold, slope_norm

# ตัวอย่าง (ATR = $2,800 จาก Running Example Part 3 §3.9):
# EMA(20) 5 วันล่าสุด: [104,000, 103,200, 102,100, 100,800, 99,400] — ดังต่อไปนี้
# slope = -$1,160/แท่ง → slope_norm = -1160/2800 = -0.41 < -0.15 → VETO ✓
# เทียบกับ: [100,000, 99,800, 99,700, 99,500, 99,400] — ลงเหมือนกันแต่ช้า
# slope = -$150/แท่ง → slope_norm = -150/2800 = -0.05 → ไม่ veto
# (ราคาลงจริง แต่ drift เล็กเทียบ noise — นี่คือสภาพที่กริดเก็บเกี่ยวได้ปกติ)
```

ตำแหน่งของ overlay นี้ในระบบ — และสิ่งที่มันไม่ได้ทำ

ลำดับการใช้: คำนวณหลัง State Machine (§4.4) ตัดสิน state แล้ว — ถ้า state = GRID_ON แต่ slope_norm veto → ปฏิบัติเหมือน CAUTION เฉพาะฟังซื้อใหม่ (ไม่กางไม้ Long เพิ่ม) จนกว่า slope_norm จะกลับเข้าโซน neutral เราจึงใจไม่รวมมันเป็น vote ที่ 4 ใน majority vote เพราะสัญญาณ 3 ตัวเดิมสมมาตร (วัดสภาพตลาด) แต่ตัวนี้ทำงานฝั่งเดียว (คุ้มครองฟังซื้อ) — ผสมกันจะ

ทำให้ logic การ vote ตีความยากขึ้นโดยไม่จำเป็น ใครจะต่อเข้า framework ใน Part 9 ให้แทรกใน `run_cycle()` หลังขั้น regime detection ก่อนขั้น order placement

สิ่งที่มันไม่ได้ทำ: filter นี้ไม่ได้สร้างกำไร — ในกรอบ zero-EV (Part 0 §0.4) มันแค่หยุดเล่นในหน้าต่างเวลาที่ EV ของไม้ใหม่ติดลบชัด แล้วกลับมาเล่นเมื่อ EV กลับมา ≈ 0 และเหลือ volatility ให้เก็บเกี่ยว — เหมือนกรรมการที่เป่านกหวีดพักเกม ไม่ใช่กองหน้าที่ทำประตูเพิ่ม

4.3 Bollinger Band Width — Squeeze Detection

BB Width ต่ำ (squeeze) = ราคากำลัง consolidate ณ ปัจจุบัน → เหมาะ deploy grid ตอนนี้ BB Width พุ่งสูง = volatility expansion = breakout → grid อาจติดหล่ม

ข้อควรระวัง: squeeze เป็นภาพ ณ จุดเวลาเดียว ไม่ใช่การรับประกันว่าจะไม่ breakout — ในทางเทคนิคอล squeeze มักถูกมองเป็นสัญญาณเตือนก่อน volatility expansion (ราคาสะสมพลังงานก่อนวิ่งแรง) ดังนั้น คะแนน "ideal" ด้านล่างใช้ได้เฉพาะ ณ รอบที่ประเมิน — ต้องอาศัย State Machine (§4.4) ตรวจสอบซ้ำทุก ชั่วโมงเพื่อจับจังหวะที่ BB Width เริ่มขยายและปรับ state ทันที ไม่ใช่ deploy แล้ววางใจว่า squeeze จะอยู่แบบนั้นตลอด

```
BB Width = (BB_upper - BB_lower) / SMA(20)
```

Thresholds สำหรับ Grid:

BB_width < 0.02 (2%)	→ Squeeze — GRID ON (ideal ณ จุดนี้)
BB_width 0.02-0.04	→ Normal — GRID ON
BB_width 0.04-0.08	→ Expanding — CAUTION
BB_width > 0.08 (8%)	→ Wide — GRID OFF (high vol)

ตัวอย่าง BTC ที่ \$100k:

```
BB_upper = $105,000, BB_lower = $95,000  
BB_width = ($105k - $95k) / $100k = 10% → GRID OFF (เกิน 8% threshold)
```

```
BB_upper = $101,500, BB_lower = $98,500  
BB_width = $3k / $100k = 3% → Normal → GRID ON
```

4.4 State Machine — GRID ON / CAUTION / GRID OFF

GRID ON	CAUTION	GRID OFF
Full Deploy 100%	Reduce 50% size	Pause / Cancel all
$H < 0.50$	$H \text{ } 0.50\text{--}0.58$	$H > 0.58$
$ADX < 40 \text{ (BTC)}$	$ADX \text{ } 40\text{--}50 \text{ (BTC)}$	$ADX > 50 \text{ (BTC)}$
$BB_width < 4\%$	$BB_width \text{ } 4\text{--}8\%$	$BB_width > 8\%$

```
class GridStateMachine:
    STATES = ["GRID_ON", "CAUTION", "GRID_OFF"]

    def __init__(self):
        self.state = "GRID_ON"
        self.state_history = []

    def evaluate(self, hurst, adx, bb_width):
        """
        Determine grid state from 3 indicators.
        Requires 2-of-3 indicators to agree before state change (noise
        filter).
        Hurst ใช้ hysteresis (dead band) กันการ flap ตอนแกว่งใกล้ 0.50: ถ้า
        ตอนนี้
        ยังเป็น GRID_ON ต้อง  $H \geq 0.52$  ถึงจะขยับเป็น CAUTION (ไม่ใช่ 0.50 พอดี) |
        ถ้าตอนนี้เป็น CAUTION/GRID_OFF อยู่แล้ว ต้อง  $H < 0.47$  ถึงจะกลับมา ON
        """
        if hurst > 0.58:
            hurst_sig = "OFF"
        elif self.state == "GRID_ON":
            hurst_sig = "CAUTION" if hurst >= 0.52 else "ON"
        else:
            hurst_sig = "ON" if hurst < 0.47 else "CAUTION"

        signals = {
            "hurst": hurst_sig,
            "adx": "OFF" if adx > 50 else ("CAUTION" if adx > 40 else
            "ON"), # BTC-calibrated: CAUTION=40, OFF=50 (textbook Forex
            threshold=25/35 ไม่ถูกสำหรับ BTC)
```

```

        "bb":      "OFF" if bb_width > 0.08 else ("CAUTION" if
bb_width > 0.04 else "ON"),
    }

    counts = {"ON": 0, "CAUTION": 0, "OFF": 0}
    for sig in signals.values():
        counts[sig] += 1

    # Majority vote (2 of 3) – สัญญาณ OFF แม้แค่ 1 ตัวก็ยกระดับเป็นอย่าง
น้อย
    # CAUTION เสมอ (เดิมถ้า ON=2/OFF=1/CAUTION=0 จะตกไป GRID_ON เนี่ย)
ทั้งที่
    # มีสัญญาณหนึ่งกำลังเตือนอันตราย – เช่น BB width ซึ่งไวสุดในสาม ตัว)
    if counts["OFF"] >= 2:
        new_state = "GRID_OFF"
    elif counts["OFF"] >= 1 or counts["CAUTION"] >= 2:
        new_state = "CAUTION"
    else:
        new_state = "GRID_ON"

    if new_state != self.state:
        self.state_history.append((self.state, new_state))
        self.state = new_state

    return self.state, signals

def get_size_multiplier(self):
    return {"GRID_ON": 1.0, "CAUTION": 0.5, "GRID_OFF": 0.0}
[self.state]

```

Hysteresis — ป้องกัน State Flip ที่เกินไป

ปัญหา: ถ้า Hurst แกว่งรอบ 0.50 (บางครั้ง 0.49 บางครั้ง 0.51) → state เปลี่ยนทุกชั่วโมง = สั่ง cancel/open orders บ่อยเกิน = fee เสียเยอะ

Solution: Hysteresis (Dead Band)

- เปลี่ยนจาก GRID_ON → CAUTION ต้องมี $H \geq 0.52$ (ไม่ใช่ 0.50)
- เปลี่ยนกลับจาก CAUTION → GRID_ON ต้องมี $H < 0.47$ (ต่ำกว่า threshold)
- Dead band = 0.05 รอบ threshold ทำให้ state stable กว่า

สังเกต: dead band นี้ implement อยู่ใน `evaluate()` ด้านบนแล้ว (เช็ค `self.state` ก่อนเลือก threshold) ไม่ใช่แค่แนวคิดลอยๆ

4.5 Grid Reset — เริ่ม Zone ใหม่ทั้งหมดเมื่อจำเป็น

เมื่อ Grid OFF แล้ว regime กลับมาเป็น mean-reverting: ราคาอาจเปลี่ยนไปมากจนต้อง reset zone ใหม่

Grid Reset Decision:

สถานการณ์ที่ต้อง Reset (ไม่ใช่แค่ restart):

1. ราคาหลุดออกนอก original zone (ต่ำกว่า zone_low หรือสูงกว่า zone_high)
2. ราคา mean เปลี่ยนไป > 15% จากตอนตั้ง grid เดิม
3. ผ่านไปนานกว่า 30 วัน (regime อาจเปลี่ยนโครงสร้าง)

```
def should_reset_zone(original_zone_mid, current_price,
original_zone_low,
                        original_zone_high, days_since_open):
    """
    Returns True ถ้าควร reset zone ใหม่
    """
    price_drift = abs(current_price - original_zone_mid) /
original_zone_mid

    out_of_zone = current_price < original_zone_low or current_price >
original_zone_high
    large_drift = price_drift > 0.15
    stale      = days_since_open > 30

    return out_of_zone or large_drift or stale
```

Grid Reset Steps:

1. Cancel orders ทั้งหมดที่ pending
2. ขาย inventory บางส่วน (ถ้า unrealized loss < 15%)
หรือ รักษา inventory ไว้ถ้า BTC (ถือได้)
3. คำนวณ zone ใหม่จาก Donchian + ATR ปัจจุบัน
4. คำนวณ Q ใหม่ตาม capital ที่เหลือ
5. วาง orders ใหม่ในชั่วโมงที่ score > 55

4.6 Zone Migration — เลื่อน Zone ตาม Trend แบบนุ่มนวล (ไม่ใช่ Reset)

แทนที่จะ reset ทั้งหมด บางครั้งแค่ "เลื่อน zone" ขึ้นหรือลงตาม EMA (Exponential Moving Average — ค่าเฉลี่ยเคลื่อนที่ที่ให้น้ำหนักราคาล่าสุดมากกว่าราคาเก่า จึงตอบสนองไวกว่าค่าเฉลี่ยธรรมดา) ก็เพียงพอ วิธีนี้ต่างจาก 4.5 ตรงที่ **ไม่ปิด order เดิม ไม่ขาย inventory ทิ้ง** — แค่อัปเดต ปรับขอบ zone ตามราคา que เคลื่อนไป inventory เก่ายังรอ TP ตามปกติ:

Zone Migration (Trailing Zone):

ทุก 24 ชั่วโมง:

```
new_zone_mid = EMA(20) ของ current price
zone_high = new_zone_mid × (1 + zone_half_width_pct)
zone_low = new_zone_mid × (1 - zone_half_width_pct)
```

ตัวอย่าง: BTC ขยับจาก \$100k → \$110k ใน 1 สัปดาห์

Old zone: \$90k-\$115k (mid=\$102.5k)

EMA(20) = \$107k

zone_half_width = 15% (ตาม Donchian width)

New zone: $\$107k \times 0.85 = \$90.95k \rightarrow \$107k \times 1.15 = \$123.05k$

การย้าย:

- + Cancel orders ที่อยู่นอก new zone
- + เพิ่ม orders ใน upper zone ที่เพิ่งเข้ามา
- + inventory ที่ BUY ไปแล้วในชั้นเก่า → ยังคง TP เดิม

Zone Migration ดีกว่า Grid Reset เมื่อ:

- ราคา drift ช้า (trending แต่ไม่เร็ว)
- Inventory ยังไม่ได้ขาดทุนมาก
- ไม่อยากเสีย inventory ที่ BUY ไปแล้ว

ข้อควรระวังตอน migrate: กริดชุดใหม่ไม่รู้จำไม้เก่าที่ค้างอยู่ — level ใหม่อาจวางทับโซนที่มี inventory เดิมพอดี ทำให้ exposure กระจุกที่ราคาเดียวโดยไม่ตั้งใจ ก่อนวาง order ชุดใหม่ให้รัน Overlap Guard ([Part 2 §2.5.1](#): skip level ที่ห่างไม้ค้างไม่ถึง $m \times ATR$)


```

        elif new_state == "GRID_ON":
            current_size = self.grid.get_deployed_size()
            if current_size < 1.0:
                actions.append({"action": "INCREASE_SIZE",
                                "target": 1.0, "reason": "regime
cleared"})

        # ตรวจสอบ zone migration
        if should_reset_zone(self.grid.zone_mid,
market_data["current_price"],
                                self.grid.zone_low, self.grid.zone_high,
                                self.grid.days_running):
            actions.append({"action": "ZONE_MIGRATION", "reason":
"price drifted"})

        return {"state": new_state, "signals": signals, "actions":
actions}

```

Watchdog Alert Types:

1. REGIME_CHANGE → ส่ง LINE/Telegram notify
2. ZONE_BREACH → ราคาหลุด zone boundary
3. DD_WARNING → Drawdown > 4% (ดู Part 5)
4. DD_HALT → Drawdown > 8% → cancel all
5. RESET_REQUIRED → Zone stale > 30 วัน

4.8 Running Example — Regime Change Sequence

 Running Example: BTC Regime เปลี่ยนจาก Mean-Reverting → Trending

Timeline: 3 สัปดาห์ · Grid เปิดที่ \$95k–\$115k · Step \$2k · Capital \$25,000

วันที่	BTC Price	Hurst (30d)	ADX (14d)	BB Width	State	Action
Day 1	\$103k	0.44	18	3.2%	GRID ON	Deploy 100%
Day 5	\$106k	0.47	21	3.8%	GRID ON	ทำงานปกติ
Day 10	\$112k	0.52	27	5.1%	CAUTION	Reduce to 50%
Day 12	\$118k	0.59	44	8.2%	GRID OFF	Cancel all orders
Day 15	\$125k	0.61	38	9.2%	GRID OFF	รอ regime กลับ
Day 18	\$121k	0.55	29	6.1%	CAUTION	รอสัญญาณส่วนใหญ่ (ADX/BB) กลับ ON
Day 21	\$118k	0.47	22	4.0%	GRID ON	Zone Migration → \$105k–\$135k

สรุปผลระหว่าง Day 1–21:

Grid ON (Day 1–9): +\$180 cashflow (ทำงานปกติ)
CAUTION (Day 10–11): +\$20 cashflow (ลด size 50%)
GRID OFF (Day 12–17): \$0 cashflow (ถือ inventory, รอ)
Zone Migration (Day 21): เปิด grid ใหม่ที่ \$105k–\$135k

Inventory ที่ถือ: BUY ที่ \$98k, \$100k, \$102k, \$104k (ตอน grid เปิด)
ราคาปัจจุบัน \$118k → avg buy price = $(\$98k + \$100k + \$102k + \$104k) / 4 = \$101k$

inventory กำไร unrealized = $4 \times 0.01 \text{ BTC} \times (\$118\text{k} - \$101\text{k}) = 4 \times 0.01 \times \$17\text{k} = \$680$

ถ้าไม่มี state machine:

- grid ยังเปิด order ที่ \$120k, \$122k, \$124k ระหว่าง uptrend
- ขาย inventory เร็วเกินไป (SELL TP ที่ \$100k, \$102k หรือต่ำกว่าราคาตลาด)
- พลาด upside gain ของ BTC

2-of-3 Vote Breakdown (Day 12):

$H=0.59 > 0.58 \rightarrow \text{OFF}$ · $\text{ADX}=44 \in [40,50] \rightarrow \text{CAUTION}$ · $\text{BB}=8.2\% > 8\% \rightarrow \text{OFF}$
counts: $\text{OFF}=2$, $\text{CAUTION}=1 \rightarrow \text{majority OFF} \rightarrow \text{GRID OFF} \checkmark$

2-of-3 Vote Breakdown (Day 21) – state ก่อนหน้าคือ CAUTION:

$H=0.47$ ไม่ < 0.47 (hysteresis exit ต้อง < 0.47 เป๊ะ ไม่ใช่ $\leq$) → ยังนับเป็น CAUTION
 $\text{ADX}=22 < 40 \rightarrow \text{ON}$ · $\text{BB}=4.0\%$ ไม่ $> 4\% \rightarrow \text{ON}$
counts: $\text{OFF}=0$, $\text{CAUTION}=1 \rightarrow$ ไม่ถึงเกณฑ์ CAUTION (ต้อง ≥ 2) → GRID ON
 $\checkmark$

หมายเหตุ: รอบนี้ ADX/BB โหวตชนะเป็น ON แม้ Hurst เองยังไม่ผ่าน hysteresis exit พอดี – Hurst ต้องรอรอบถัดไปที่ $H < 0.47$ จริงๆ ถึงจะ "ON" ในตัวเอง แต่ระบบ 2-of-3 vote ไม่ต้องรอให้ทุกสัญญาณ ON พร้อมกัน

4.9 แบบฝึกหัด

แบบฝึกหัด Part 4

- ▶ ข้อ 1: Hurst=0.46, ADX=28, BB_width=3.5% — State Machine ส่งผลให้ state เป็นอะไร? (2-of-3 majority vote)
- ▶ ข้อ 2: State เปลี่ยนจาก GRID_ON → GRID_OFF โดยตรง (ข้าม CAUTION) — เป็นไปได้ไหม? เกิดอย่างไร?
- ▶ ข้อ 3: Grid run 35 วัน · Original zone_mid = \$100k · Current price = \$118k · Zone \$88k–\$115k — ควร reset หรือ migrate?
- ▶ ข้อ 4 (Challenge): ต่อยอด Hysteresis rules — \$4.4 ทำ ON ↔ CAUTION แล้ว ขอบ CAUTION ↔ OFF ควรเป็นแบบไหน?
- ▶ ข้อ 5: EMA(20) 5 วันล่าสุด = [98,000, 97,600, 97,000, 96,200, 95,200] · ATR = \$2,800 · threshold = -0.15 — slope_norm เท่าไหร่? veto ฝั่งซื้อไหม? แล้วไม้ Long ที่ถืออยู่ กับ TP ของมันต้องทำอะไร?

Sizing & Risk — Kelly, Position Limit, Drawdown Control

Kelly Criterion บน Grid · Fractional Kelly · Max Notional ต่อ Level · DD Halt

อ้างอิง: [Statarb Ch.12](#) (Kelly, Position Sizing)

แก่นของ PART นี้

Sizing ที่ดีไม่ได้ทำให้กำไรเพิ่มมากขึ้น แต่ป้องกัน **ruin** — grid ที่กำไรได้ต่อเนื่องจะหยุดถ้า capital หมดก่อน Kelly ช่วยตอบ "ควร deploy กี่% ของ capital ต่อกลยุทธ์ grid รวม (ไม่ใช่ต่อ level)" เพื่อ maximise long-run geometric growth

$$\text{Kelly } f^* = (p \times b - q) / b \cdot \text{Fractional} = 0.25 \times f^* \cdot Q \\ = f^* \times \text{capital} / (N \times P)$$

5.1 Kelly Criterion บน Grid — แนวคิด

Kelly Criterion ดั้งเดิม (จาก [Statarb Ch.12](#)) ให้ optimal fraction ของ capital ที่ควร risk ต่อ trade โดยหลักการคือ **maximise $E[\log(\text{wealth})]$** — ไม่ใช่ maximize expected profit ต่อรอบ แต่ maximize long-run geometric growth rate:

$$f^* = \frac{p \cdot b - q}{b}$$

โดย: p = ความน่าจะเป็นที่ trade นั้นกำไร, b = อัตราส่วน win/loss, $q = 1 - p$

สำหรับ Grid Trading: "1 trade" = 1 grid cycle (BUY fill → SELL fill ที่ level ถัดไป)

การประมาณ Kelly สำหรับ Grid:

กรณี BTC/USDT Bybit:

$P(\text{cycle completes}) = p$ ← ความน่าจะเป็นที่ราคาขึ้นถึง TP หลัง BUY fill

ประมาณ: สำหรับ mean-reverting asset ($H < 0.5$), $p \approx 0.55-0.65$

Win amount per cycle = $Q \times \text{Step} - \text{Fees} = \18

Loss amount per cycle = "ไม่มี" "loss" ต่อ cycle ใน Close System
(loss = opportunity cost ของ capital ที่ lock)

ปัญหา: Kelly classic ไม่ fit grid โดยตรง เพราะ grid ไม่มี "cut loss"

แนวทางประยุกต์:

→ ใช้ Kelly กำหนด "ขนาด grid รวม" vs total capital (ไม่ใช่ต่อ cycle)

→ f^* = optimal fraction ของ net worth ที่ควรใส่ใน grid portfolio

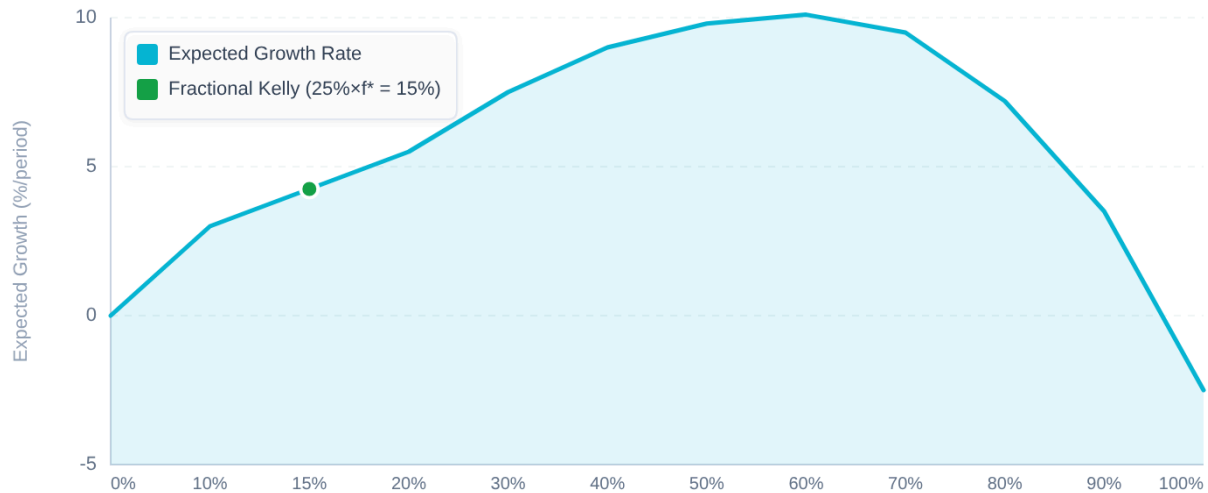
⚠ $p=0.55-0.65$ เป็นสมมติฐาน ไม่ใช่ความจริงที่รับประกัน

ค่า p ที่ถูกต้องขึ้นอยู่กับ: asset ที่เทรด, ช่วงเวลา, regime ปัจจุบัน — ค่าที่ backtest ได้จาก 90 วันมี Standard Error สูง

ตัวอย่าง error propagation (ที่ $b=1.0$ ตามตาราง sensitivity ด้านล่าง): p จริง = 0.60 แต่ estimate error $\pm 0.05 \rightarrow f^*$ คลาดเคลื่อนได้ถึง $\pm 50\%$ ($p=0.55 \rightarrow f^*=10\%$, $p=0.65 \rightarrow f^*=30\%$, เทียบกับ $p=0.60 \rightarrow f^*=20\%$)

แนะนำ: ใช้ Fractional Kelly ($25\% \times f^*$) เสมอ เพื่อ buffer uncertainty ใน p

Kelly Curve — Expected Growth Rate vs Fraction of Capital Risked



$f^* \approx 60\%$ (Full Kelly — peak) · $25\% \times f^* \approx 15\%$ (Fractional Kelly — safe zone จุดเดียว) · Over-bet > f^* → growth ลด → ruin ถ้า $2 \times f^*$

Kelly Curve: Growth Rate ขึ้นถึงจุดสูงสุดที่ f^* แล้วตกลงเมื่อ bet มากเกินไป — Fractional Kelly ($25\% \times f^*$) อยู่ในโซน safe (ในกราฟนี้ $f^* \approx 60\%$ เป็นตัวเลข illustrative — ค่าจริงขึ้นกับ backtest ของแต่ละ grid)

5.2 Kelly สำหรับ Grid Portfolio (Practical Approach)

วิธีง่ายสุด: backtest grid ด้วยข้อมูลย้อนหลัง → วัด win rate และ profit ratio ต่อ "month" → apply Kelly

ตัวอย่างจาก Backtest (90 วัน):

เดือนที่ผลรวม > 0: 2.5 เดือน จาก 3 → $p = 0.83$ (83%)

เดือนที่ผลรวม < 0: 0.5 เดือน → $q = 0.17$

Avg monthly win = +\$450 บน \$10,000 capital (+4.5%)

Avg monthly loss = -\$200 บน \$10,000 capital (-2.0%)

$b = \text{avg_win} / \text{avg_loss} = 450 / 200 = 2.25$

Kelly $f^* = (0.83 \times 2.25 - 0.17) / 2.25$
 $= (1.8675 - 0.17) / 2.25$
 $= 1.6975 / 2.25 \approx 0.754$ (75.4%)

Fractional Kelly (25%): $f_{\text{safe}} = 0.754 \times 0.25 = 0.189 \approx 19\%$

→ ควรใส่เงินในกลยุทธ์ grid นี้ไม่เกิน 19% ของ net worth

ถ้า net worth = \$100,000:

Grid capital = \$100,000 × 19% = \$19,000

Sample Size Warning — ทำไม 83% ไม่แม่นยำอย่างที่ตัวเลขดูดี

ตัวเลข $p=83\%$ ข้างบนมาจากการนับ "เดือนที่กำไร" แค่ 3 เดือน ($n=3$) — ไม่ใช่ 270 fills ที่เกิดขึ้นในช่วงเดียวกัน สถิติของสัดส่วนต้องนับที่หน่วยซึ่งเป็นการทดลองอิสระจริงๆ และที่หน่วยนั้นคือ "เดือน" ไม่ใช่ "fill ย่อยในเดือน"

Standard Error ของสัดส่วน: $SE = \sqrt{p(1-p)/n}$ — ที่ $n=3$: $SE = \sqrt{0.83 \times 0.17 / 3} \approx \pm 22\%$ (ไม่ใช่ $\pm 2-3\%$ ที่จะได้ถ้าเผลอนับ $n=270$) แปลว่าค่า p จริงอาจอยู่ที่ไหนก็ได้ตั้งแต่ 60% ถึง 100% แล้วก็ไม่ขัดกับข้อมูล 3 เดือนที่มี

ถ้าอยากได้ SE แคบกว่านี้ ต้องนับ win/loss เป็นรายรอบ grid จริง (270 รอบใน 90 วัน ที่ $p=83\%$ เดียวกัน ให้ $SE = \sqrt{0.83 \times 0.17 / 270} \approx \pm 2.3\%$ ตามตัวอย่างด้านล่าง) — ถ้ายังไม่มีข้อมูลระดับนั้น ให้สมมติว่า SE กว้างแบบนี้ไว้ก่อน แล้วดูหัวข้อถัดไปว่าทำไมมันสำคัญกว่าที่คิด

Kelly Sensitivity Analysis — ผลกระทบของ p ที่คาดเคลื่อน

p (win rate)	b (win/loss ratio)	f* (Full Kelly)	25% × f* (Fractional)	Max Q ต่อ Capital
0.50 (coin flip)	1.0	0%	0%	ไม่เล่น
0.55	1.0	10%	2.5%	ต่ำมาก
0.60	1.0	20%	5%	ปานกลาง
0.65	1.0	30%	7.5%	สูง
0.55 (overestimate)	1.0	10%	2.5%	—

หาก p จริงต่ำกว่าที่ estimate 5% → f* ลดลงครึ่งหนึ่ง — ใช้ Fractional Kelly ป้องกัน overbet

ทำไมถึงใช้แค่ 1 ใน 4 ของ Kelly — คำตอบที่ดีกว่า "เผื่อไว้ก่อน"

ถึงตรงนี้คุณคำนวณ f* ได้แล้ว (75.4% จากตัวอย่าง 5.2) และเราสั่งให้หารสี่ก่อนใช้เสมอ ถ้าคุณเคยแอบคิดว่า "โอเวอร์ไปไหม ถ้าไรหายไปถึงสามในสี่" — คำถามนี้สมควรได้คำตอบจริงๆ ไม่ใช่แค่ "เผื่อไว้ก่อน"

ขั้นแรก: ตัวเลขที่ป้อน Kelly นั้นแม่นยำน้อยกว่าที่คิดมาก — กล้อง Sample Size Warning ข้างบนเพิ่งแสดงว่า p=83% มี SE กว้างถึง ±22% และ Kelly มีนิสัยร้ายอยู่ข้อหนึ่ง: **แพ้ทางการมองโลกแง่ดี** ถ้า p จริงต่ำกว่าที่คิด full Kelly ไม่ใช่แค่กำไรน้อยลง แต่พาทั้งพอร์ตตึงได้ (ตาราง sensitivity ข้างบนแสดงแล้วว่า p ประเมินพลาดแค่ 5% ทำให้ f* หายไปครึ่งหนึ่ง) การเติมพันด้วยเศษเสี้ยวของ Kelly คือการซื้อประกันความไม่รู้ของตัวเอง

ขั้นที่สอง — คำถามที่เปลี่ยนวิธีคิด: แทนที่จะถามว่า "โตเร็วสุดต้องเติมพันเท่าไร" ให้ถามกลับด้านว่า **"ถ้ายอมให้โอกาสเจอ drawdown ทะลุ -8% (เส้น HALT ของเราเองใน 5.5) มีได้ไม่เกิน 1 ใน 20 ครั้ง — เติมพันได้มากที่สุดเท่าไร?"** คำถามนี้มีคำตอบทางคณิตศาสตร์ และสำหรับพารามิเตอร์แบบ grid ทั่วไป (win rate สูง กำไรต่อรอบเล็ก ความไม่แน่นอนของ p กว้างแบบที่เพิ่งเห็น) คำตอบจะตกอยู่แถว **0.2–0.3 เท่าของ f*** แทบทุกครั้ง

มองเห็นไหมว่าเกิดอะไรขึ้น: กฎ "หารสี่" ที่ใช้มาตลอดเล่มไม่ได้เปลี่ยนสัณนิษ แต่สถานะของมันเปลี่ยน — จากกฎที่ขอให้เชื่อ กลายเป็นผลลัพธ์ที่คำนวณย้อนได้จากเส้น drawdown ที่คุณเป็นคนเลือกเองใน 5.5 ระบบ sizing กับระบบเบรกของเล่มนี้เป็นเรื่องเดียวกันมาตั้งแต่ต้น เราเพิ่งต่อสายให้เห็น

► ตารางปรับ fraction เอง ถ้าไม่อยากใช้ 25% ตายตัว (กดเปิด)

ดูรายละเอียดเพิ่มเติมใน [Statarb Ch.12](#)

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่มนี้): Busseti, Ryu & Boyd, "Risk-Constrained Kelly Gambling" (2016)

5.3 Q (Quantity) ต่อ Level — จาก Kelly

คำนวณ Q ต่อ Level จาก Kelly fraction:

Grid capital = net_worth × f_safe = \$100k × 0.19 = \$19,000

N levels (total) = (Zone_high - Zone_low) / Step = (\$110k - \$90k) / \$2k
= 10 levels

Avg price = (\$90k + \$110k) / 2 = \$100k

Q per level = grid_capital / (N × avg_price)

= \$19,000 / (10 × \$100,000)

= \$19,000 / \$1,000,000

= 0.019 BTC (ใช้ precision 3 ตำแหน่งเหมือน Exercise 2 ด้านล่าง –
ไม่ต้องปัดขึ้นเป็น 0.02

เพราะ 0.019 ก็อยู่ในงบ \$19,000 พอดีอยู่แล้ว การปัดขึ้นเป็น 0.02 จะ
ทำให้เกินงบ)

ตรวจสอบ: 10 levels × 0.019 BTC × avg_price \$100k = \$19,000 ✓ ตรงงบพอดี
(ใช้ avg_price เดียวกับที่คำนวณไว้ข้างบน – ไม่ใช่ผลรวม price แยกที่ได้ avg ต่างไป)

5.4 Max Notional ต่อ Level — Position Limit

นอกจาก Kelly ที่กำหนด grid size รวม ควรมี position limit ต่อ level เพื่อป้องกันความเสี่ยงกระจุก — "Notional" คือมูลค่าเต็มของ position ต่อ level ($Q \times \text{ราคา}$) ไม่ใช่แค่ทุนที่วางค้ำ:

Position Limit Rules:

Rule 1: Max Notional ต่อ Level $\leq 5\%$ ของ total capital

$\text{Max_notional_level} = \$100\text{k} \times 5\% = \$5,000 \text{ per level}$

$\text{Max } Q \text{ per level} = \$5,000 / P = \$5,000 / \$100,000 = 0.05 \text{ BTC}$

Rule 2: ในช่วงที่ Hurst สูง (trending) → ลด limit ลงอีก

ถ้า $H > 0.55$: $\text{max_q} \times 0.7$ (ลด 30%)

ถ้า $H > 0.58$: $\text{max_q} \times 0.4$ (ลด 60% — เข้าเขต GRID OFF ของ Part 4 พอดี)

Rule 3: Concentration cap — ไม่ให้ 3 levels ติดกันมี Q รวม $> 10\%$ ของ capital (ป้องกัน bottom-heavy bloating ที่เกินไป)

```
def check_position_limits(grid_levels, total_capital, hurst):
    max_per_level = total_capital * 0.05
    if hurst > 0.58:
        max_per_level *= 0.4
    elif hurst > 0.55:
        max_per_level *= 0.7

    for level in grid_levels:
        notional = level.qty * level.price
        if notional > max_per_level:
            level.qty = max_per_level / level.price # trim
    return grid_levels
```

5.5 Drawdown Control — DD Halt

Drawdown (DD) = peak portfolio value ลบด้วย current value ÷ peak:

$$DD_t = \frac{P_{\text{peak}} - P_t}{P_{\text{peak}}}$$

DD Halt System:

```
def check_drawdown_halt(portfolio_value_history, halt_threshold=-0.08):  
    """  
    ถ้า DD < threshold (เช่น -8.5% < -8%) → หยุด grid ทั้งหมด  
    threshold = -0.08 (-8% ของ peak, ปรับได้ตาม risk tolerance)  
    """  
    peak = max(portfolio_value_history)  
    current = portfolio_value_history[-1]  
    dd = (current - peak) / peak # จะเป็นลบเมื่อ current < peak  
  
    if dd < halt_threshold:  
        return "HALT", dd, peak  
    elif dd < halt_threshold * 0.5: # -4% = warning zone  
        return "WARNING", dd, peak  
    return "OK", dd, peak
```

Halt ทำอะไร:

1. Cancel orders ทั้งหมด
2. ไม่เปิด order ใหม่ (แต่ไม่ขาย inventory – Close System)
3. รอให้ DD พ้นกลับ > threshold/2 ก่อน restart
4. Log และ alert

Resume Condition:

- DD พ้นกลับ > -4% (ครึ่งหนึ่งของ halt threshold)
- + H < 0.55 (กลับมา mean-reverting)
- + ผ่านไปอย่างน้อย 24 ชั่วโมง (cooling off period)

Risk Level	DD Halt Threshold	ลักษณะ
Conservative	-5%	หยุดเร็ว ทำกำไรน้อยลง แต่ปลอดภัย
Moderate (แนะนำ)	-8%	สมดุล — ให้ grid ทำงานในความผันผวนปกติ
Aggressive	-15%	ทน volatility ได้มาก แต่ DD ลึก
Close System Pure	ไม่มี (ถึง floor)	รอ BTC กลับเสมอ — ต้องมี capital พอ

5.6 Running Example — Full Sizing Calculation

Running Example: Sizing BTC Grid จาก Kelly

หมายเหตุ: ตัวอย่างนี้ใช้ *net worth* \$200,000 · $p=0.78$ · $b=1.9$ ซึ่งต่างจากตัวอย่างใน 5.2 (\$10k · $p=0.83$ · $b=2.25$) — เพื่อ illustrate ด้วยพารามิเตอร์ที่สมจริงกับ account ขนาดใหญ่

ที่มาของ $p=0.78$, $b=1.9$: backtest BTC grid Zone \$90k–\$110k · Step \$2k · 6 เดือน $\approx$ 540 grid cycles — $SE \approx \sqrt{(0.78 \times 0.22 / 540)} \approx \pm 1.8\%$ (เชื่อถือได้พอสมควร แต่ยังแนะนำ Fractional Kelly)

โจทย์: Net worth \$200,000 · Backtest 6 เดือน: $p=0.78$ · $b=1.9$ · Grid Zone \$90k–\$110k · Step \$2k

Step 1: คำนวณ Kelly f^*

$$\begin{aligned} f^* &= (0.78 \times 1.9 - 0.22) / 1.9 \\ &= (1.482 - 0.22) / 1.9 \\ &= 1.262 / 1.9 \approx 0.664 \text{ (66.4\%)} \end{aligned}$$

Step 2: Fractional Kelly (25%)

$$f_{\text{safe}} = 0.664 \times 0.25 = 0.166 = 16.6\%$$

Step 3: Grid Capital

$$\text{Grid capital} = \$200,000 \times 16.6\% = \$33,200$$

Step 4: Q per Level

$$\begin{aligned} N &= (110k - 90k) / 2k = 10 \text{ levels} \\ \text{Avg price} &= \$100k \\ Q &= \$33,200 / (10 \times \$100k) = 0.0332 \text{ BTC} \approx 0.033 \text{ BTC} \end{aligned}$$

Step 5: ตรวจ Position Limit (5% rule)

$$\begin{aligned} \text{Max notional} &= \$200k \times 5\% = \$10,000 \\ Q_{\text{max}} &= \$10,000 / \$100k = 0.10 \text{ BTC} \\ Q_{\text{actual}} &= 0.033 < 0.10 \checkmark \text{ (ผ่าน)} \end{aligned}$$

Step 6: Zone Waterfall

$$\begin{aligned} \text{Active (50\%)} &: \$16,600 \rightarrow \text{deploy as orders} \\ \text{Standby (30\%)} &: \$9,960 \rightarrow \text{lending/yield} \\ \text{Floor (20\%)} &: \$6,640 \rightarrow \text{reserve} \end{aligned}$$

Summary:

Grid capital: \$33,200 (16.6% ของ net worth)

Q per level: 0.033 BTC

Daily P&L estimate: 1.5 รอบ $\times$ (\$66 - fee) $\approx$ \$90/วัน = 0.27%/วัน

5.7 แบบฝึกหัด

แบบฝึกหัด Part 5

- ▶ ข้อ 1: Backtest: $p=0.70$, $\text{win}=\$300$, $\text{loss}=\$150$ — Kelly f^* เป็นเท่าไร? ควรใช้ capital เท่าไหร่จาก \$50k?
- ▶ ข้อ 2: Grid capital \$10k · Zone \$85k–\$115k · Step \$2k · BTC = \$100k — Q ต่อ level เท่าไร?
- ▶ ข้อ 3: Portfolio peak = \$115,000 · Current = \$105,000 · Halt threshold = -8% — ควร halt หรือไม่?
- ▶ ข้อ 4 (Challenge): Hurst = 0.62 · กำลังจะเปิด grid ด้วย $Q = 0.03$ BTC — ควรปรับ Q อย่างไร?

Pair Grid + Relative Value Switching

BTC/ETH Spread Grid · Cointegration Test · Relative Value Signal

Multi-Asset Grid Portfolio · Switching Between Instruments

อ้างอิง: [Statarb Ch.4-6](#) (Cointegration, Spread, OU Process)

แก่นของ PART นี้

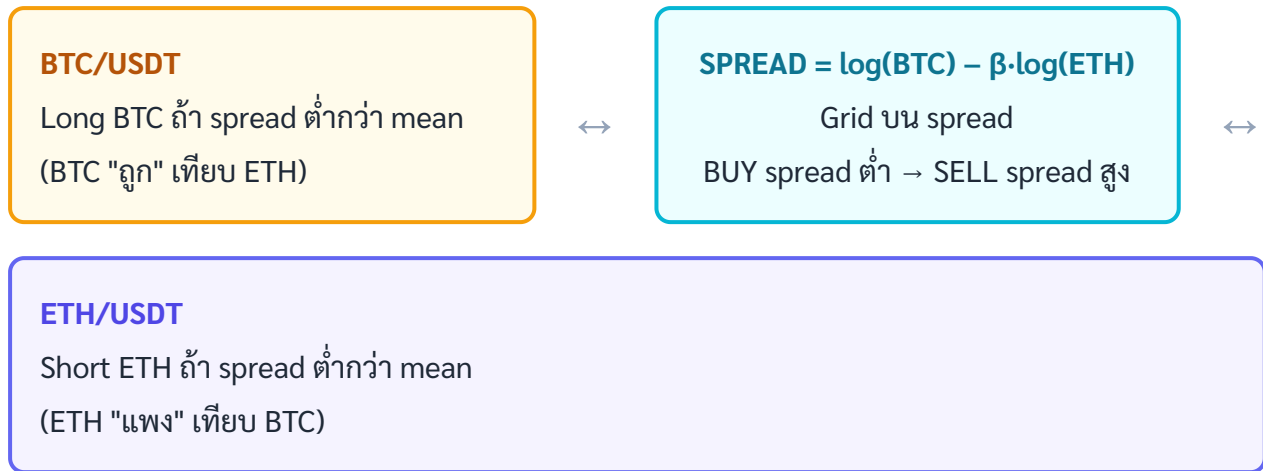
Pair Grid ไม่ใช่ grid บน BTC price โดยตรง แต่ใช้ grid บน **spread** ระหว่างสองสินทรัพย์ (เช่น BTC/ETH ratio) — spread cointegrated มี mean-reversion ที่แข็งแกร่งกว่าราคาเดียว + ไม่ขึ้นกับทิศทางตลาด

Spread = $\log(\text{BTC/USDT}) - \beta \times \log(\text{ETH/USDT})$. Grid on spread reversion to mean

6.1 ทำไม Pair Grid?

ปัญหาของ Single-Asset Grid: ถ้า BTC uptrend → grid ขาย inventory เร็ว (ดี) แต่ถ้า BTC downtrend → inventory สะสม (ไม่ดี)

Pair Grid แก่ด้วย: สร้าง grid บน spread ที่ cointegrated — spread revert ไม่ว่า BTC จะขึ้นหรือลง (ตราบไคที่ BTC/ETH ratio กลับสู่ค่าเฉลี่ย)



6.2 Cointegration — พื้นฐาน Pair Grid

Correlation $\neq$ Cointegration

BTC และ ETH อาจมี correlation สูง (ขึ้นลงพร้อมกัน) แต่นั่นไม่รับประกันว่า spread จะ mean-revert — ต้องผ่าน Cointegration Test จึงจะ trade ได้อย่างมั่นใจ

สถิติพื้นฐานที่ต้องรู้ก่อนอ่านต่อ (ใช้ตลอดหัวข้อนี้)

Null Hypothesis (H_0): ข้อสมมติฐาน "ค่าเริ่มต้น" ที่เราพยายามหาหลักฐานมาหักล้าง — ในที่นี้ H_0 คือ "spread ไม่ mean-revert" (มี unit root แปลว่าเดินสุ่มไม่มีจุดดึงกลับ) เราอยาก **reject H_0** (ปฏิเสธมันได้) เพื่อสรุปว่า spread mean-revert จริง

ADF Test statistic กับ Critical Value: ADF test คำนวณตัวเลขหนึ่งตัว (test statistic) แล้วเทียบกับ "เกณฑ์ตัดสิน" (critical value) ที่กำหนดไว้ล่วงหน้า — ถ้า test statistic *ต่ำกว่า* (ติดลบมากกว่า) critical value ถือว่ามีหลักฐานพอจะ reject H_0

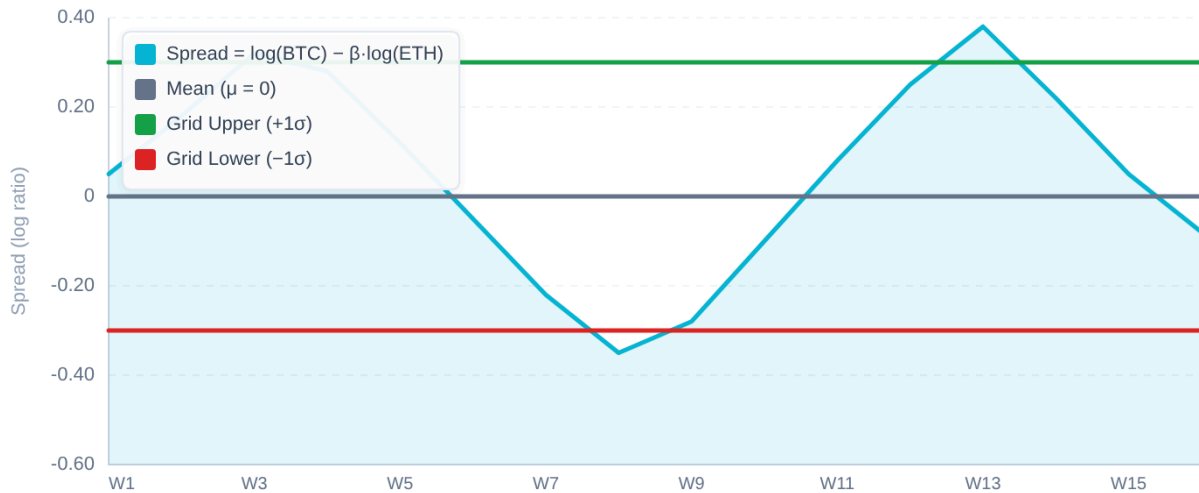
p-value: ความน่าจะเป็นที่จะเห็นผลลัพธ์แบบนี้ (หรือรุนแรงกว่า) ถ้า H_0 เป็นจริง — p-value ยิ่งต่ำ ยิ่งมีหลักฐานค้าน H_0 มาก ธรรมเนียมทั่วไปคือ $p < 0.05$ ถือว่า reject H_0 ได้ (นี่คือที่มาของ "ระดับนัยสำคัญ 5%" ที่เห็นซ้ำๆ ด้านล่าง)

Cointegration (จาก [Statarb Ch.5](#)) หมายความว่า: แม้อราคาดสองตัวจะ random walk แต่ linear combination ของมันเป็น stationary:

$$\text{Spread}_t = \log(P_{BTC,t}) - \beta \cdot \log(P_{ETH,t}) - \alpha \sim \text{Stationary}$$

β (beta) คือความชันจาก linear regression ระหว่าง $\log(BTC)$ กับ $\log(ETH)$ — บอกว่าถ้า $\log(ETH)$ เปลี่ยน 1% แล้ว $\log(BTC)$ มักเปลี่ยนตามกี่% โดยเฉลี่ยในอดีต เช่น $\beta=0.80$ หมายความว่า $\log(ETH)$ เปลี่ยน 1% $\rightarrow$ $\log(BTC)$ เปลี่ยนตามเฉลี่ย 0.80% ใช้คำนวณล่วงหน้า ETH ในสมการ spread เพื่อให้ spread หนึ่งที่สุดเต๋าที่จะทำได้ (ไม่ใช่แค่ลบราคาตรงๆ)

BTC/ETH Log Spread — Cointegration Mean-Reversion (สมมติ 16 สัปดาห์)



เมื่อ spread $> +1\sigma$: Short BTC / Long ETH → เมื่อ spread $< -1\sigma$: Long BTC / Short ETH → profit เมื่อ revert

BTC/ETH Spread (log ratio) — mean-revert กลับสู่ค่าเฉลี่ยอยู่เสมอ → Grid บน spread ทำกำไรได้ต่อเนื่อง

ทดสอบ Cointegration สำหรับ Pair Grid:

```
def test_cointegration(btc_prices, eth_prices, significance=0.05):
    """
    Engle-Granger 2-step cointegration test
    Returns: (is_cointegrated, beta, alpha, adf_stat, p_value)
    """
    import numpy as np
    from scipy import stats

    log_btc = np.log(btc_prices)
    log_eth = np.log(eth_prices)

    # Step 1: OLS regression log_BTC = alpha + beta * log_ETH
    slope, intercept, r, p, se = stats.linregress(log_eth, log_btc)
    beta = slope
    alpha = intercept

    # Step 2: คำนวณ spread (residuals)
    spread = log_btc - beta * log_eth - alpha

    # Step 3: ADF test บน spread
    adf_stat = adf_test(spread) # implementation ดูใน statarb
    p_value = adf_p_value(adf_stat, len(spread))

    is_cointegrated = p_value < significance
    return is_cointegrated, beta, alpha, spread
```

```
# ตัวอย่าง BTC/ETH (180 วัน):
# OLS:  $\log(\text{BTC}) = 2.14 + 0.82 \times \log(\text{ETH}) \rightarrow \text{beta} = 0.82$ 
# ADF stat = -3.47 (critical at 5% = -2.89)  $\rightarrow p < 0.05 \rightarrow \text{cointegrated} \checkmark$ 
# Half-life of spread reversion =  $\ln(2) / |\text{AR coeff}| \approx 8 \text{ days}$ 
# (สูตรเต็มคือ  $-\ln(2)/\ln(\varphi)$  โดย  $\varphi = \text{AR}(1) \text{ coefficient}$  —  $\ln(2)/|\text{AR coeff}|$ )
# ด้านบนคือ approximation เมื่อ  $k=1-\varphi$  มีค่าน้อย ซึ่ง  $\ln(\varphi) \approx -k$ 
```

Johansen Test — ดีกว่า Engle-Granger สำหรับ Pair Grid

Engle-Granger มี low statistical power ("power" ในที่นี้คือความสามารถของ test ในการจับ cointegration ที่มีอยู่จริงให้เจอ — power ต่ำ = พลาด cointegration จริงบ่อย แม้มันมีอยู่จริง) — ใช้ Johansen เป็น confirmation:

```
from statsmodels.tsa.vector_ar.vecm import coint_johansen

def johansen_cointegration(btc_prices, eth_prices,
    significance_level=0.05):
    """
    Johansen test — ดีกว่า Engle-Granger สำหรับ 2+ variables
    Returns True ถ้า cointegrated
    """
    import numpy as np
    data = np.column_stack([np.log(btc_prices), np.log(eth_prices)])
    result = coint_johansen(data, det_order=0, k_ar_diff=1)

    # Trace statistic: critical values at 5%
    # result.cvt[:, 1] = 5% critical values
    trace_stat = result.lr1[0] # rank=0 hypothesis
    crit_val = result.cvt[0, 1] # 5% critical value

    is_cointegrated = trace_stat > crit_val
    return {
        "cointegrated": is_cointegrated,
        "trace_stat": trace_stat,
        "crit_val_5pct": crit_val,
        "eigenvectors": result.evec # loading vectors
    }

# ใช้ทั้งสอง test ร่วมกัน:
```

```
# ถ้า Engle-Granger AND Johansen ทั้งคู่ = cointegrated → เชื่อถือได้สูง
# ถ้าแค่อันเดียว → ระวัง (อาจเป็น spurious)
```

⚠ Cointegration Break Risk

Cointegration ไม่ถาวร — ความสัมพันธ์ระหว่าง BTC/ETH อาจพังได้เมื่อ:

- Structural regime change (เช่น ETH ถูก SEC classify เป็น security)
- Exchange listing/delisting ของ asset ใดสักตัว
- Regulatory event ที่กระทบ asset ตัวใดตัวหนึ่งเฉพาะ

เมื่อ cointegration พัง: spread ไม่ revert → Pair Grid สะสม unrealized loss ไม่จำกัด ต้องปิด position ทันที

Re-validate ทุก 30 วัน: run ADF test + Johansen test ใหม่ ถ้า p-value > 0.05 → หยุด Pair Grid ทันที

```
def rolling_cointegration_check(btc_prices, eth_prices, window=60,
step=5):
    """
    ตรวจสอบ cointegration แบบ rolling — ความสัมพันธ์อาจเปลี่ยนตาม regime
    window: วันที่ใช้ test
    step: ขยับ window ทีละกี่วัน
    """
    from statsmodels.tsa.stattools import coint
    results = []

    for start in range(0, len(btc_prices) - window, step):
        end = start + window
        _, pvalue, _ = coint(btc_prices[start:end],
eth_prices[start:end])
        results.append({
            "start_day": start,
            "end_day": end,
            "p_value": pvalue,
            "cointegrated": pvalue < 0.05
        })

    # นับ % ของ windows ที่ cointegrated
    cointegrated_pct = sum(r["cointegrated"] for r in results) /
len(results)
```

```
return results, cointegrated_pct
```

```
# ถ้า cointegrated_pct < 70% → relationship ไม่เสถียร → จะวิ่ง Pair Grid
```

6.3 Spread Grid — โครงสร้าง

เมื่อรู้ว่า spread cointegrated แล้ว วาง grid บน spread โดยตรง:

Spread Grid Setup:

```
spread_mean = 0.0      (โดย construction จาก OLS)
spread_std  = 0.045    (จาก historical spread distribution)
zone_width  = 3σ each side = ±0.135

# Grid levels (spread units)
spread_levels = [-0.135, -0.090, -0.045, 0.0, +0.045, +0.090, +0.135]
               = [-3σ,   -2σ,   -1σ,   0,   +1σ,   +2σ,   +3σ ]

# แต่ละ level: ถ้า spread < level → BUY BTC + SHORT ETH
#              ถ้า spread > level → SELL BTC + LONG ETH

def pair_grid_signal(current_spread, spread_levels, prev_spread, beta,
                    btc_price, eth_price, q_btc):
    """
    ตรวจสอบว่า spread ข้ามผ่าน level ไหน → ส่ง orders
    """
    orders = []
    q_eth = q_btc * btc_price / eth_price * beta # ETH qty ที่ match BTC

    for level in spread_levels:
        if prev_spread > level and current_spread <= level:
            # Spread ลงผ่าน level → spread กำลังต่ำ → BUY spread (Long
            # BTC, Short ETH)
            orders.append({"action": "BUY_SPREAD",
                          "btc": f"BUY {q_btc} BTC",
                          "eth": f"SHORT {q_eth:.4f} ETH",
                          "tp_spread": level + 0.045}) # TP ที่ level +
            1σ

        elif prev_spread < level and current_spread >= level:
            # Spread ขึ้นผ่าน level → spread กำลังสูง → SELL spread (Short
            # BTC, Long ETH)
            orders.append({"action": "SELL_SPREAD",
                          "btc": f"SHORT {q_btc} BTC",
```

```
1σ  
  
"eth": f"BUY {q_eth:.4f} ETH",  
"tp_spread": level - 0.045}) # TP ที่ level -  
  
return orders
```

ยิ่งไกลยิ่งน่าเข้า? — จุดที่ควรเลิกรอ

grid ข้างบนวางถึง $\pm 3\sigma$ ทุก level ถือว่า "น่าเข้า" พอๆ กันหมด แต่ยิ่งไกลจาก mean ยิ่งน่าเข้าจริงหรือ หรือมีจุดที่ควรเลิกคิดว่ามันจะกลับ?

ลองนึกภาพรอเพื่อนที่นัดไว้ — สาย 10 นาทีปกติ สาย 30 นาทีเริ่มกังวล สาย 2 ชั่วโมง ไม่ใช่ "ยิ่งสายนานยิ่งใกล้จะมาถึง" แต่กลับเป็นสัญญาณว่ามีอะไรผิดปกติ

spread ก็เหมือนกัน: ยิ่งเบี่ยงจาก mean ไกลเกินระดับหนึ่ง ความเป็นไปได้ที่ **cointegration พังไปแล้ว** (ไม่ใช่แค่รอ mean-reversion ตามปกติ) ก็ยิ่งสูงขึ้น เพราะเหตุนี้ "โซนที่ควรเข้า" จึงมีขอบเขตจำกัดเสมอ อยู่ก่อนถึงจุดยอมแพ้เสมอ (ไม่ใช่ยิ่งไกลยิ่งซื้อเพิ่มไม่มีที่สิ้นสุด — เลยจุดยอมแพ้ไปแล้วคือเลิกเข้าเพิ่ม ไม่ใช่ยิ่งเข้าหนักขึ้น)

เพราะฉะนั้น grid ของ spread ควรจะมี **จุดยอมแพ้ (give-up point)** กำกับไว้ 2 แบบ:

- **ตามราคา:** $\pm 4\sigma$ (นอกช่วง 7 levels ข้างบน) = ปิด position ฉุกเฉินทันที ไม่ว่าจะกำไรหรือขาดทุน — ระดับนี้ไม่ใช่ level ให้ "เข้าเพิ่ม" แต่เป็นสัญญาณให้ออก
- **ตามเวลา:** ถ้าถือ position เกิน ~ 3 เท่าของ half-life (half-life = เวลาเฉลี่ยที่ spread ใช้ "กลับมาครึ่งทาง" สู่ค่าเฉลี่ยหลังเบี่ยงออกไป — จาก §6.2 half-life ≈ 8 วัน $\rightarrow$ ยอมแพ้ที่ ~ 24 วัน) โดยยังไม่ถึง TP ให้ re-run cointegration test ทันที — ไม่ต้องรอรอบเช็ค 30 วันตามปกติ (ดูกล่อง Cointegration Break Risk ด้านบน)

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่นนี้): Leung & Li, "Optimal Mean Reversion Trading with Transaction Costs and Stop-Loss Exit" (2015) — พิสูจน์ว่าโซนเข้าซื้อของ mean-reversion ที่ optimal มีขอบเขตจำกัดเสมอ ไม่ใช่ยิ่งไกลยิ่งเข้า

6.4 Capital Allocation สำหรับ Pair Grid

Capital Allocation – Pair Grid:

สมมติ: $BTC = \$100k$ · $ETH = \$3,500$ · $\beta = 0.82$

$Q_{btc} = 0.01$ BTC per level

$Q_{eth} = Q_{btc} \times BTC_price / ETH_price \times \beta$
 $= 0.01 \times \$100,000 / \$3,500 \times 0.82$
 $= 0.01 \times 28.57 \times 0.82 = 0.2343$ ETH

ตรวจ dollar parity

$BTC_notional = 0.01 \times \$100k = \$1,000$

$ETH_notional = 0.2343 \times \$3,500 = \$820$

beta-adjusted hedge ratio ทำให้ไม่ต้องเท่ากันพอดี ($\beta = 0.82 \neq 1$)

Capital ต่อ level (worst case):

Long BTC: \$1,000 (ซื้อ BTC)

Short ETH: margin \$82 (10× leverage)

Total per level: \$1,082

7 levels total:

Total pair grid capital: $7 \times \$1,082 \approx \$7,574$

กำไรต่อรอบ (spread กลับ $1\sigma = 0.045$):

BTC leg: $0.01 \text{ BTC} \times \$100k \times 0.045 = \$45$ (ถ้า spread mean-reverts ผ่าน 1σ)

ETH leg: $0.2343 \text{ ETH} \times \$3,500 \times 0.045 \times 0.82 \approx \30

Net gain: $\approx \$45 - \$30 = \$15$ per cycle (net of both legs)

Note: กำไรจาก spread convergence ไม่ใช่ price direction

⚠ Beta (β) ระหว่าง BTC/ETH ไม่คงที่ตาม Regime

Beta ที่คำนวณจาก 60–90 วัน อาจเปลี่ยนมากใน regime ต่างกัน:

- Bull market: $\beta \approx 1.1\text{--}1.3$ (ETH outperforms BTC)
- Bear market: $\beta \approx 0.8\text{--}1.0$ (ETH underperforms ชัดกว่า)

- ถ้าใช้ static β ที่คำนวณในช่วง bull แล้วเข้าสู่ bear → spread signal ผิดทั้งหมด

แก้: ใช้ rolling β (window 30 วัน) และ recalibrate ทุก 2 สัปดาห์ หรือตรวจ β drift > 20% → หยุด Pair Grid ชั่วคราว

```
def rolling_beta(btc_returns, eth_returns, window=30):
    """คำนวณ beta แบบ rolling"""
    betas = []
    for i in range(window, len(btc_returns)):
        x = btc_returns[i-window:i]
        y = eth_returns[i-window:i]
        beta = np.cov(x, y)[0,1] / np.var(y)
        betas.append(beta)
    return betas

# ถ้า |beta_current - beta_initial| > 0.2 → recalibrate หรือหยุด Pair Grid
```

ข้อดีของ Pair Grid vs Single-Asset Grid

- Market-neutral: BTC pump/dump ไม่ affect pair grid มากนัก (ถ้า cointegration คงอยู่)
- Spread mean-reversion แข็งแกร่งกว่า price mean-reversion (half-life สั้นกว่า)
- 2 sources of income: spread reversion + funding rate ของ short leg

6.5 Relative Value Switching — เลือก Asset ที่ดีที่สุด

เมื่อมีหลาย pair หรือหลาย asset — Relative Value Switching เลือก grid บน pair/asset ที่มี mean-reversion signal แรงที่สุด ณ เวลานั้น

สังเกตในสูตรด้านล่างว่า Hurst ได้น้ำหนักตั้ง 40% — มากกว่า spread z-score เสียอีก ทำไม? ลองนึกภาพสปริงสองเส้น: เส้นหนึ่งแข็ง ดึงออกนิดเดียวก็ติดกลับเร็ว อีกเส้นอ่อน ดึงออกเท่ากันแต่คืนตัวช้ากว่ามาก **z-score** บอกแค่ "ตอนนี้ยืดออกไปเท่าไร" ส่วน Hurst บอกว่า "สปริงเส้นนี้แข็งหรืออ่อน" — **ก็จะติดกลับเร็วแค่ไหน** คู่ที่ z สูงมากแต่ H ก็สูงด้วย (สปริงอ่อน) อาจยืดต่อไปอีกนานกว่าจะกลับ ในขณะที่คู่ที่ z ปานกลางแต่ H ต่ำมาก (สปริงแข็ง) มีโอกาสกลับไวกว่าเยอะ — และยิ่งกลับไว ยิ่งเก็บรอบต่อวันได้มากกว่า break-even ที่ Part 0 พุดถึงได้ง่ายขึ้น นี่คือเหตุผลที่ H ควรมีน้ำหนักมากกว่า z-score ในการจัดอันดับว่าคู่ไหนน่าเล่นที่สุด

Relative Value Score — เปรียบ 3 grids:

ตัวเลือก:

- A: BTC/USDT Spot Grid
- B: BTC/ETH Pair Grid
- C: ETH/USDT Spot Grid

Score ต่อ asset (0-100):

$$= 0.40 \times \text{Hurst_score} + 0.30 \times \text{Spread_z_score} + 0.30 \times \text{Momentum_score}$$

```
def relative_value_score(hurst, spread_zscore, momentum_reversal):  
    """  
    spread_zscore = |current spread - mean| / std (ยิ่งสูง = ยิ่งน่า trade)  
    momentum_reversal = RSI extreme (ดูจาก Part 3B)  
    """  
    h_score = hurst_score_for_grid(hurst) # จาก Part 3B  
  
    # Spread z-score: ยิ่งสูง ยิ่งดี (spread ห่างจาก mean มาก)  
    z_score_clamped = min(3.0, abs(spread_zscore))  
    z_score_score = z_score_clamped / 3.0 * 100 # map to 0-100  
  
    mom_score = rsi_score_for_grid(momentum_reversal) # จาก Part 3B  
  
    return 0.40 * h_score + 0.30 * z_score_score + 0.30 * mom_score  
  
# ตัวอย่าง:
```

```
# A (BTC Spot): Hurst=0.48→h_score=65, z=0.5→z_score=16.67,  
RSI=45→mom=55  
#   score = 0.40×65 + 0.30×16.67 + 0.30×55 = 26.0+5.0+16.5 = 47.5  
# B (BTC/ETH): Hurst=0.43→h_score=85, z=2.1→z_score=70.0, RSI=32→mom=68  
#   score = 0.40×85 + 0.30×70.0 + 0.30×68 = 34.0+21.0+20.4 = 75.4 ←  
WINNER  
# C (ETH Spot): Hurst=0.53→h_score=40, z=0.8→z_score=26.67,  
RSI=50→mom=50  
#   score = 0.40×40 + 0.30×26.67 + 0.30×50 = 16.0+8.0+15.0 = 39.0  
  
# → Allocate 70% capital ไปที่ Pair Grid (B) + 30% ไปที่ BTC Spot (A)
```

6.6 Multi-Asset Grid Portfolio

เมื่อมี capital มากพอ — เปิดหลาย grid พร้อมกัน แต่ต้องจัดการ correlation ระหว่าง assets:

Grid	Asset	Correlation กับ BTC Grid	Max Allocation (ตามสูตรด้านล่าง)
Grid 1	BTC/USDT Spot	1.0 (ตัวเอง)	20% (corr สูงสุด → cap ต่ำสุด)
Grid 2	ETH/USDT Spot	0.75 (สูง)	25%
Grid 3	BTC/ETH Pair	0.10 (ต่ำ — market neutral)	50% (corr ต่ำสุด → cap สูงสุด)
Grid 4	SOL/USDT	0.55 (ปานกลาง)	35%

แต่ละแถวคือ ceiling อิสระต่อ grid เดียว (มาจาก max_allocation_by_correlation ด้านล่าง) ไม่ใช่สัดส่วนที่ต้องรวมกันได้ 100%
พอดี — portfolio จริงต้องเลือก allocation ของแต่ละ grid ให้อยู่ได้ cap ของตัวเอง และ รวมกันไม่เกิน 100% ของทุนทั้งหมด

Portfolio Correlation Management:

```
def max_allocation_by_correlation(correlation_with_btc):  
    """  
    ยิ่ง correlation สูง → max allocation น้อยลง (ป้องกัน concentration)  
    """  
    if correlation_with_btc > 0.80:  
        return 0.20 # max 20%  
    elif correlation_with_btc > 0.60:  
        return 0.25 # max 25%  
    elif correlation_with_btc > 0.40:  
        return 0.35 # max 35%  
    else:  
        return 0.50 # market-neutral: max 50%
```

ข้อดี Multi-Asset Grid:

- + Diversification ลด drawdown รวม
- + เมื่อ BTC trending (grid BTC หยุด) → Pair Grid ยังทำงาน
- + ลด "idle capital" ในช่วง single-asset dead zone

ข้อเสีย:

- ต้องการ capital มากกว่า

- การ monitor ชับซ้อนขึ้น
- Correlation พัง ใน market crash (all correlate to 1)

6.7 Running Example — BTC/ETH Pair Grid

Running Example: BTC/ETH Pair Grid Implementation

Setup: Capital \$15,000 · BTC=\$102k · ETH=\$3,600 · beta=0.80 · spread_std=0.040

Step 1: คำนวณ Q

$Q_{\text{btc}} = 0.008 \text{ BTC per level (จาก capital / N} \times \text{price)}$

$Q_{\text{eth}} = 0.008 \times \$102\text{k} / \$3,600 \times 0.80 = 0.1813 \text{ ETH}$

Step 2: Spread levels (7 levels $\pm 3\sigma$)

$\sigma = 0.040 \rightarrow \text{levels: } -0.12, -0.08, -0.04, 0, +0.04, +0.08, +0.12$

Step 3: Current spread

$\log(102,000) - 0.80 \times \log(3,600) = 11.533 - 0.80 \times 8.188 = 11.533 - 6.550 = 4.983$

$\text{spread_mean (OLS fit)} \approx 4.983 \rightarrow \text{normalized spread} = (4.983 - 4.983) / 0.040 = 0$

Day 5: BTC ลง \$4k $\rightarrow$ \$98k, ETH ขึ้น \$200 $\rightarrow$ \$3,800

$\text{new spread} = \log(98\text{k}) - 0.80 \times \log(3,800) = 11.493 - 0.80 \times 8.243 = 11.493 - 6.594 = 4.899$

$\text{normalized: } (4.899 - 4.983) / 0.040 = -2.1\sigma \rightarrow \text{ใกล้ } -2\sigma \text{ level}$

Action: BUY spread ที่ -2σ

BUY 0.008 BTC @ \$98,000 = \$784

SHORT 0.1813 ETH @ \$3,800 = \$689 notional (margin 10 $\times$ = \$69)

TP: spread กลับมา -1σ level

Day 8: BTC \$100k, ETH \$3,740 $\rightarrow$ spread = $\log(100\text{k}) -$

$0.80 \times \log(3,740) = 11.513 - 6.581 = 4.932$

$\text{normalized: } (4.932 - 4.983) / 0.040 = -1.3\sigma \rightarrow \text{ยังไม่ถึง TP } (-1\sigma) \text{ ยังถือ position}$

Day 10: BTC \$106k, ETH \$3,650 $\rightarrow$ spread = $\log(106\text{k}) -$

$0.80 \times \log(3,650) = 11.571 - 6.562 = 5.009$

$\text{normalized: } (5.009 - 4.983) / 0.040 = +0.66\sigma \rightarrow \text{ผ่าน TP } (-1\sigma) \text{ ไปแล้ว (spread น่าจะข้าม } -1\sigma$

จริงระหว่าง Day 8-10 แต่ตัวอย่างนี้ snapshot แค่วันเว้นวัน ไม่ได้ track ทุกวัน

เพื่อความง่าย จึงคำนวณ P&L จากราคา ณ Day 10 ไม่ใช่ราคา ณ จุดที่ TP ถูกแตะจริง –

P&L จริงถ้า execute ทันทีที่ -1σ อาจน้อยกว่านี้)

TP hit!

BTC gain: $0.008 \times (\$106k - \$98k) = \$64$

ETH short gain: $0.1813 \times (\$3,800 - \$3,650) = \$27.2$

Net P&L: $\$64 + \$27.2 = \$91.2$ per cycle (ใน 5 วัน)

$\approx \$91 / \853 capital used = 10.7% ใน 5 วัน ← best-case (spread revert ตรงเวลา, ไม่มี slippage)

Note: BTC ขึ้น \$8k แต่ ETH ขึ้นน้อยกว่า (relative) → spread converge → กำไรจาก spread ไม่ใช่ direction

ในทางปฏิบัติ spread ไม่ revert ทุก cycle – ผลจริงต่ำกว่านี้ได้

6.8 แบบฝึกหัด

แบบฝึกหัด Part 6

- ▶ ข้อ 1: ADF stat = -2.45 บน BTC/ETH spread · Critical value 5% = -2.89 — Cointegrated หรือไม่?
- ▶ ข้อ 2: Pair Grid BTC/ETH มี correlation กับ BTC Spot Grid ต่ำ (~ 0.10) — ทำไม Pair Grid ถึง market-neutral?
- ▶ ข้อ 3: $\beta = 0.80$ · $Q_{\text{btc}} = 0.01 \text{ BTC}$ · $\text{BTC} = \$100\text{k}$ · $\text{ETH} = \$4,000$ — Q_{eth} เท่าไหร่? Dollar value match ไหม?
- ▶ ข้อ 4: ทำไม correlation ระหว่าง assets พัง ($\rightarrow 1$) ใน market crash? และ Pair Grid จะเป็นอย่างไร?

GRID TRADING MASTERY · PART 6B

Grid Snowball + DCA Stack

3 รูปแบบ Compound · Profit-Triggered Layer · Zone Upgrade

Grid → DCA Stack: Trading Portfolio ป้อน Wealth Portfolio

Long-Term BTC Accumulation ผ่าน Grid Cashflow

Part 6 เน้นหา alpha เพิ่มผ่าน pair trading (BTC ↔ ETH spread) — Part นี้เปลี่ยนโฟกัสไปที่การจัดการ cashflow ที่ grid แบบใดก็ได้ (single-asset หรือ pair) ทำออกมาได้ ให้ compound ได้ระยะยาวแทนที่จะปล่อยเฉยๆ

แก่นของ PART นี้

Grid ที่อยู่ถูกสนาม ($H < 0.5$) เก็บ cashflow ได้เกือบทุกวัน แต่ถ้าแค่ "นำกำไรออก" compound growth จะไม่เกิดขึ้น ต้องใช้ Grid Snowball นำ cashflow กลับเข้า grid เพื่อเพิ่ม Q หรือเปิด zone ใหม่ นอกจากนี้ Grid-DCA Stack ยังส่งส่วนหนึ่งของกำไรไปซื้อ BTC ระยะยาว (Wealth Portfolio) อย่างเป็นระบบ

Snowball: $Q(t) = Q_0 \times (1 + \text{daily_return})^t$ · **DCA Stack:**
30% grid profit → weekly BTC buy

6B.1 Grid Snowball — 3 รูปแบบ

Type 1: Simple Compound

นำ profit ทั้งหมดกลับเข้า grid
เพิ่ม Q ทุก N cycles

Type 2: Profit-Triggered Layer

เปิด grid level ใหม่เมื่อ profit
สะสมถึง threshold

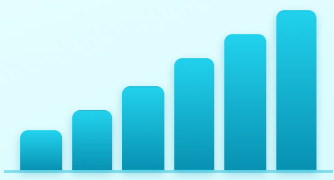
Type 3: Zone Upgrade

ขยาย zone หรือเพิ่ม N
เมื่อ capital เพิ่มพอ

GRID SNOWBALL 3 แบบ + DCA STACK

Type 1: Simple Compound

Q เพิ่มทุก 7 วัน



compound ราบเรียบ

Type 2: Profit-Triggered

Layer ใหม่เปิดเมื่อกำไรถึง threshold

Layer 3

▲ profit >= \$500

Layer 2

▲ profit >= \$500

Layer 1 (ต้นทุน \$10k)

compound แบบ discrete

Type 3: Zone Upgrade

กำไรขยาย zone coverage

compound แบบ discrete

\$90k – \$110k
(zone เดิม)

compound แบบ expanded coverage

Grid → DCA Stack

Trading Portfolio

Grid ~\$45/day
(active trading)

→ weekly

70% Compound

30% Transfer
(split allocation)

→ monthly

Wealth Portfolio

BTC accumulation
(passive long-term)

reinvest 70% → ขยาย Grid size · โอน 30% → สะสม BTC ทุกเดือน
compound สองมิติ: active (grid) + passive (BTC hodl)

กำไรจาก Grid (active) → สะสม BTC ระยะยาว (passive) = compound สองมิติ

6B.2 Type 1: Simple Compound

นำ realized profit กลับเข้า grid capital → Q เพิ่มขึ้น → cashflow ต่อรอบเพิ่มขึ้น → compound effect:

$$Q(t) = Q_0 \times (1 + r_{\text{daily}})^t$$

```
def simple_compound_grid(initial_q, daily_profit_rate, days,
rebalance_every=7):
```

```
    """
```

```
    Type 1: Compound Q ทุก N วัน
```

```
    daily_profit_rate = กำไรต่อวัน / grid capital
```

```
    rebalance_every = ปรับ Q ทุกกี่วัน
```

```
    """
```

```
    q = initial_q
```

```
    capital = initial_q * 100000 # สมมติ price $100k
```

```
    log = []
```

```
    for day in range(1, days + 1):
```

```
        # กำไรวันนี้
```

```
        daily_profit = capital * daily_profit_rate
```

```
        capital += daily_profit
```

```
        # ปรับ Q ทุก rebalance_every วัน
```

```
        if day % rebalance_every == 0:
```

```
            new_q = capital / 100000
```

```
            q = round(new_q, 4) # round เพื่อให้ practical
```

```
        log.append({"day": day, "q": q, "capital": capital})
```

```
    return log
```

```
# ตัวอย่าง: Q0=0.01 BTC · rate=0.27%/วัน · 365 วัน
```

```
# Q(365) = 0.01 × (1.0027)365 = 0.01 × 2.6756 = 0.0268 BTC
```

```
# Capital: $1,000 → $2,676 (+168% ใน 1 ปี)
```

ข้อดี: เรียบง่าย ไม่ต้องตัดสินใจ

ข้อเสีย: ถ้ากำไรไม่ realized (แปลว่ายังเป็นแค่กำไรบนกระดาษ inventory ยังไม่ถูกขายออกจริง) → Q ไม่เพิ่ม

6B.3 Type 2: Profit-Triggered Layer

รอให้ profit สะสมถึง threshold แล้วค่อยเปิด "grid level ใหม่" — ได้ compound แบบ discontinuous step:

```
def profit_triggered_layer(grid_bot, trigger_threshold=500):
    """
    เปิด grid level ใหม่เมื่อ accumulated profit ≥ threshold
    """
    accumulated_profit = 0
    layers_added = 0

    while True:
        # รับ daily profit จาก grid
        daily_profit = grid_bot.get_daily_realized_profit()
        accumulated_profit += daily_profit

        if accumulated_profit >= trigger_threshold:
            # เปิด level ใหม่ (extend zone ลงอีก 1 step)
            new_level_price = grid_bot.zone_low - grid_bot.step
            grid_bot.add_level(price=new_level_price,
                               qty=grid_bot.base_q)
            grid_bot.zone_low = new_level_price
            layers_added += 1

            # Reset accumulation counter
            accumulated_profit -= trigger_threshold

            print(f"Layer {layers_added} added at
                  ${new_level_price:,}")

# ตัวอย่าง (ใช้ rate สมมติที่เร็วกว่า baseline 0.27%/วันของเล่ม — เพื่อให้เห็น
# กลไก layer-adding ชัดภายในกรอบเวลา 1 เดือน ไม่ใช่ตัวเลขมาตรฐาน):
# Grid capital $10,000, กำไร $100/วัน (~1%/วัน)
# Trigger every $500 profit ≈ ทุก 5 วัน
# ใน 1 เดือน: เพิ่มประมาณ 6 layers ใหม่
```

เพิ่ม zone ลดต่ำกว่า หรือสูงกว่า:

กลยุทธ์ A: เพิ่ม lower zone → capture ถ้าราคาลงมา

กลยุทธ์ B: เพิ่ม upper zone → เตรียม TP ในชั้นสูงขึ้น

กลยุทธ์ C: สลับตาม Hurst (ถ้า H ต่ำ → ต่ำสุด, ถ้า H สูงขึ้น → บนสุด)

6B.4 Type 3: Zone Upgrade

เมื่อ grid capital สะสมถึง threshold ใหม่ — ปิด grid เดิม เปิด grid ใหม่ที่ใหญ่กว่า (zone กว้างขึ้น หรือ Q ใหญ่ขึ้น):

Zone Upgrade Criteria:

```
def check_zone_upgrade(current_capital, initial_capital,
    upgrade_threshold=0.50):
    """
    Upgrade เมื่อ capital เพิ่มขึ้น 50% จากตอนเริ่ม
    """
    growth = (current_capital - initial_capital) / initial_capital
    return growth >= upgrade_threshold
```

Zone Upgrade Process:

Initial: Capital \$10,000 · Q=0.01 BTC · Zone \$90k–\$110k

After 3 months: Capital \$15,000 (+50%)

Upgrade Options:

A) เพิ่ม Q: ใช้ \$15,000 กับ zone เดิม → Q = 0.015 BTC (+50%)

B) ขยาย Zone: ใช้ \$15,000 กับ zone ใหม่ \$80k–\$120k (width \$20k→\$40k = +100% wider)

C) เพิ่ม N: zone เดิม แต่ step เล็กลง (fill บ่อยขึ้น)

ตัวอย่าง (ตรวจสอบด้วยสูตร $\text{Grid_cap} = N \times Q \times \text{avg_price}$ — ใช้ Option B ด้านบน):

\$10,000 · Zone \$90k–\$110k · Step \$2k · N=10 · avg_price=\$100k →
Q=\$10,000/(10×\$100k)=0.01 BTC

↓ 3 months profit \$5,000

\$15,000 · Zone \$80k–\$120k (Option B) · Step \$2k · N=20 ·
avg_price=\$100k → Q=\$15,000/(20×\$100k)=0.0075 BTC

Note: Zone Upgrade ต้องผ่าน validation checklist ([Part 3](#)) ก่อน

6B.5 Snowball Comparison

Type	กลไก	Complexity	Growth Rate	Risk
Type 1 (Simple Compound)	Q เพิ่มทุก N วัน	ต่ำ	โตแบบ compound เรียบๆ ต่อเนื่อง	ต่ำ
Type 2 (Triggered Layer)	เปิด level ใหม่เมื่อ profit ถึง threshold	กลาง	โตเป็นขั้นบันได (กระโดดทีละ level)	กลาง (zone ขยาย)
Type 3 (Zone Upgrade)	เปิด grid ใหม่ทั้งหมดเมื่อ capital ถึง tier	สูง	Accelerated jump	สูง (ต้อง rebalance ทั้งหมด)

แนะนำ: ใช้ Type 1 ก่อน จน $\text{capital} \geq 2 \times \text{initial}$ → Type 3 Upgrade

ผู้เริ่มต้น: Type 1 — ง่าย ไม่ต้องตัดสินใจให้ compound ทำงานเอง

ผู้มีประสบการณ์: Type 2 — เพิ่ม level ตามโอกาส ใช้ Hurst เลือกทิศทาง

ผู้ advanced: Type 3 + ใช้ Donchian ใหม่ re-optimize zone ทุก upgrade cycle

6B.6 Grid-DCA Stack — สองพอร์ตโฟลิโอ

Grid Snowball ช่วยให้ Trading Portfolio โตขึ้น — แต่ DCA Stack เพิ่มมิติที่สอง: ส่งกำไรส่วนหนึ่งไป Wealth Portfolio (ซื้อ BTC ระยะยาว)

Trading Portfolio (Grid)

Grid ทำงานทุกวัน → Cashflow \$X/วัน

70% reinvest back into grid (Snowball) | 30% → Transfer

Weekly Transfer Rule

ทุกวันจันทร์: นำ 30% ของ weekly profit ซื้อ BTC ราคาตลาด
ไม่ limit order, ไม่ wait for dip — ซื้อ market อาทิตย์ละครั้ง

Wealth Portfolio (Long-term BTC)

BTC สะสม ถือตลอด ไม่ขาย

เป้าหมาย: สะสม BTC ระยะ 5–10 ปี

DCA Stack Implementation:

```
class GridDCAStack:
    def __init__(self, grid_bot, dca_pct=0.30, dca_interval_days=7):
        self.grid = grid_bot
        self.dca_pct = dca_pct          # 30% ของ profit ไป DCA
        self.interval = dca_interval_days # DCA ทุกกี่วัน
        self.wealth_btc = 0.0          # BTC สะสมใน wealth
        portfolio
        self.trading_capital = grid_bot.capital

    def weekly_rebalance(self, current_btc_price):
        """
        ทำทุกอาทิตย์:
        1. วัด realized profit สัปดาห์นี้
        2. แบ่ง 30% ไปซื้อ BTC → wealth portfolio
        3. ส่วนที่เหลือ compound กลับใน grid
        """
```

```

weekly_profit = self.grid.get_weekly_realized_profit()

if weekly_profit <= 0:
    return # ไม่มีกำไร ไม่ DCA

# DCA amount
dca_amount = weekly_profit * self.dca_pct
btc_bought = dca_amount / current_btc_price

self.wealth_btc += btc_bought
self.trading_capital += weekly_profit * (1 - self.dca_pct)

print(f"DCA: ซื้อ {btc_bought:.6f} BTC @ ${current_btc_price:,}")
print(f"    Wealth BTC รวม: {self.wealth_btc:.4f} BTC")
print(f"    Trading capital: ${self.trading_capital:,.0f}")

```

ตัวอย่าง 1 ปี (rate มาตรฐานของเล่ม 0.27%/วัน ทบตันรายสัปดาห์ (52 สัปดาห์) บน capital เริ่มต้น \$20,000 –

ตัวเลขเดียวกับปีที่ 1 ในตาราง §6B.7):

Weekly growth factor = $(1.0027)^{52} \approx 1.01905$ (กำไร 1.905%/สัปดาห์ ก่อนหัก DCA)

Capital growth factor (หลังหัก 70% reinvest ทบตัน 52 สัปดาห์) = $(1 + 0.7 \times 0.01905)^{52} \approx 1.9916$

Grid capital ล้วนปี = $\$20,000 \times 1.9916 \approx \mathbf{\$39,800 (+99.2\%)}$

กำไรรวมทั้งปี = $(\text{Capital ล้วนปี} - \text{Capital เริ่มต้น}) / 0.70 = \$19,800 / 0.70 \approx \$28,333$

DCA (30%): $\$28,333 \times 30\% \approx \$8,500 \rightarrow$ ซื้อ BTC ทุกอาทิตย์

Compound (70%): $\$28,333 \times 70\% \approx \$19,833 \rightarrow$ กลับเข้า grid (ตรงกับส่วนต่าง capital ด้านบน)

BTC ที่ซื้อ (avg \$100k/BTC): $\$8,500 / \$100k \approx 0.085$ BTC

6B.7 Long-term Projection — Grid Snowball + DCA Stack

⚠ คำเตือนสำคัญ — Projection นี้ไม่ใช่ผลลัพธ์ที่รับประกัน

ตารางด้านล่างเป็น **กรณีที่ดีที่สุด (best-case scenario)** ภายใต้สมมติฐานที่ไม่สมจริง:

- Grid ทำกำไร 0.27%/วัน ต่อเนื่องตลอด 5 ปี — ในความเป็นจริงมีช่วง Grid OFF, [DD Halt \(Part 5\)](#), Sideways ยาวนาน — เมื่อ DD Halt ทำงาน compound หยุดชั่วคราว
- ไม่รวมช่วง Bear Market ที่ BTC ลง 50–80% (เกิดทุก 2–4 ปีในประวัติศาสตร์)
- Monte Carlo simulation (1,000 paths) ของ return จริงๆ แสดง range: **ปีที่ 5 = \$50,000–\$180,000** (median ≈ \$95,000) ไม่ใช่ \$627,000
- **Realistic expectation:** 40–70% ของตัวเลขในตาราง ในปีที่ market conditions เอื้ออำนวย

ใช้ตารางนี้เพื่อเข้าใจ *กลไก compound* เท่านั้น — ไม่ใช่เพื่อวางแผนทางการเงิน

ปีที่	Grid Capital	Daily Cashflow	BTC สะสม	Wealth Value (@ \$100k/BTC)
เริ่มต้น	\$20,000	\$54/วัน (0.27%)	0 BTC	\$0
ปีที่ 1	\$39,800	\$108/วัน	0.085 BTC	\$8,500
ปีที่ 2	\$79,300	\$214/วัน	0.254 BTC	\$25,400
ปีที่ 3	\$158,000	\$427/วัน	0.591 BTC	\$59,100
ปีที่ 4*	\$314,700	\$850/วัน	1.263 BTC	\$126,300
ปีที่ 5	\$627,000	\$1,692/วัน	2.600 BTC	\$260,000

Growth factor/ปี = $(1 + 0.7 \times ((1.0027)^7 - 1))^5 \approx 1.9916$ (+99.2%/ปี) — BTC สะสม = 30% ของกำไรสะสมแต่ละปี (อีก 70% คือส่วนที่ทำให้ Grid Capital โตตามคอมพาวนด์) หากด้วย \$100k/BTC — เช่น ปีที่ 1 กำไรรวม = $\$19,800 / 0.7 \approx \$28,333 \rightarrow 30\% \approx \$8,500 \rightarrow 0.085 \text{ BTC}$

หมายเหตุ: สมมติ BTC price คงที่ที่ \$100k (ไม่รวม BTC appreciation), daily return คงที่ที่ 0.27%/วัน, DCA 30% ทุกสัปดาห์ *ปีที่ 4 ประมาณการ (interpolated) Projection ใช้เพื่อ illustrate compound

effect เท่านั้น — ผลจริงขึ้นอยู่กับ market regime

ปัจจัยเพิ่มเติมที่ตารางข้างบนไม่ได้รวม

- BTC price ที่เปลี่ยนจริง (ไม่คงที่ \$100k) → กระทบทั้งมูลค่า grid capital และราคาที่ใช้ DCA ซื้อ BTC ได้
- ค่าธรรมเนียมแพลตฟอร์มอาจเปลี่ยนแปลงตลอดช่วงเวลา 5 ปี

6B.8 Running Example — 6 เดือนแรก

Running Example: Grid Snowball Type 1 + DCA Stack

ตัวอย่างนี้ใช้ grid capital \$10,000 (ต่างจากตาราง 6B.7 ที่ใช้ \$20,000 เป็น base case) — เพื่อให้เห็นกลไก compound ชัดเจนในขนาดที่เล็กลง

Setup: Grid capital \$10,000 · Q=0.01 BTC · Daily profit \$27 (0.27%) · DCA 30% ทุกสัปดาห์

Month 1 (เดือนแรก):

Weekly profit avg: $\$27 \times 7 = \189

DCA each week: $\$189 \times 30\% = \$56.7 \rightarrow$ ซื้อ BTC @ \$100k = 0.000567 BTC/week

Compound: $\$189 \times 70\% = \$132.3 \rightarrow$ กลับเข้า grid

End of Month: Grid capital = $\$10,000 + \$132.3 \times 4 = \$10,529$

Wealth BTC = $0.000567 \times 4 = 0.00227$ BTC

Month 3:

Grid capital $\approx \$11,600$ (compound + standby yield)

New Q = Old Q $\times$ (New capital / Old capital) = $0.01 \times (\$11,600/\$10,000) \approx 0.0116$ BTC (+16%)

Daily cashflow $\approx \$31.3$ (+16% จากเดิม)

Wealth BTC ≈ 0.007 BTC

Month 6 (สิ้นสุด):

Grid capital $\approx \$13,900$

Daily cashflow $\approx \$37.5$

Wealth BTC สะสม ≈ 0.016 BTC ($\approx \$1,600$ @ \$100k)

สรุป 6 เดือน:

Trading capital growth: $\$10,000 \rightarrow \$13,900$ (+39%)

Wealth BTC: 0.016 BTC (\$1,600)

Total value: $\$13,900 + \$1,600 = \$15,500$ (+55% รวม)

vs Buy-and-hold \$10,000 BTC: ถ้า BTC flat = \$10,000 (0%)

ข้อสังเกต: ใน 6 เดือน BTC flat

Grid+DCA รวม: +55% (trading capital compound +39% ผสาน BTC สะสม)

B&H: +0% (รอ price)

DCA (BTC ที่สะสมได้ ยังไม่รวม appreciation): +0% (BTC ราคาเดิม)

6B.9 แบบฝึกหัด

แบบฝึกหัด Part 6B

- ▶ ข้อ 1: Grid profit \$150/วัน · DCA 25% ทุกสัปดาห์ · BTC = \$95,000 — DCA BTC ต่อเดือนเท่าไร?
- ▶ ข้อ 2: ทำไม DCA ทุกอาทิตย์แบบ market order ดีกว่ารอ dip?
- ▶ ข้อ 3: Grid capital เริ่ม \$15,000 · Type 3 Upgrade เมื่อ capital $\geq 150\%$ ของ initial — ต้องรอ profit เท่าไร? (daily return 0.27%)
- ▶ ข้อ 4: Type 2 vs Type 3 Snowball — เลือก Type ไหนถ้า BTC ลงมา 20% จากตอนตั้ง grid?

Advanced Integrations — IV, Funding Rate & Options

IV-Adaptive Grid · Funding Rate Grid · Options + Grid Combination

CSP ป้องกัน Inventory · Grid + Short Strangle

อ้างอิง: [Vol Series](#) (IV Rank, GARCH), [Arb Series](#) (Funding Rate), [PM Series](#) (CSP, Strangle)

แก่นของ PART นี้

Advanced Grid รวม 3 โลก: Grid mechanics (Part 1–6) + Volatility intelligence (Vol Series) + Derivatives income (Arb/PM Series) — เมื่อทำงานร่วมกัน ทำให้ capital efficiency สูงขึ้นโดยไม่เพิ่ม risk มากนัก

IV-Adaptive Step: $\text{Step} = 0.5 \times \sigma_{\text{implied}} \times P \cdot \text{Funding}$

Grid: $\text{income} = \text{short_notional} \times \text{rate} \times 3/\text{day}$

7.1 IV-Adaptive Grid — ปรับ Step ตาม Implied Volatility

แทนที่จะใช้ ATR (backward-looking) — ใช้ Implied Volatility จาก Options market ซึ่งเป็น forward-looking:

Implied Volatility (IV) คืออะไร: ตัวเลขที่ "สกัด" ออกมาจากราคา options ในตลาด สะท้อนว่าตลาดคาดว่าราคาสินทรัพย์จะแกว่งแรงแค่ไหนในอนาคต (ยิ่งคนคาดว่าจะแกว่งแรง ยิ่งยอมจ่าย premium สูงเพื่อซื้อ options ป้องกันความเสี่ยง → IV ยิ่งสูง) ต่างจาก ATR ที่มองย้อนหลัง (ราคาแกว่งแค่ไหนที่ผ่านมา) IV คือ "การคาดการณ์" ของตลาดไปข้างหน้า — สูตรด้านล่างหาร IV รายปีด้วย $\sqrt{365}$ เพื่อแปลงเป็นค่าแกว่งต่อ 1 วัน (ค่า volatility สเกลตามรากที่สองของเวลา เป็นหลักสถิติทั่วไป)

$$\text{Step}_{\text{IV}} = 0.5 \times \frac{\text{IV}_{\text{annual}}}{\sqrt{365}} \times P \quad (\text{1-day vol} \times \text{price})$$

IV-Adaptive Step Implementation:

```
def iv_adaptive_step(iv_annual_pct, current_price, fee_rate=0.001,
                    step_factor=0.5):
    """
    iv_annual_pct: Implied Volatility ต่อปี (เช่น 0.65 = 65%)
    """
    # Convert annual IV to daily IV
    iv_daily = iv_annual_pct / (365 ** 0.5)

    # 1-day expected move (1σ)
    expected_daily_move = iv_daily * current_price

    # Step = fraction of expected daily move
    step = step_factor * expected_daily_move

    # Ensure profitability (≥ 3× fee)
    min_step = 3 * current_price * fee_rate * 2
    step = max(step, min_step)

    return round(step / 100) * 100 # round to $100

# ตัวอย่าง:
# BTC IV (7d ATM) = 65% annualized, P = $100,000
# iv_daily = 65% / sqrt(365) = 3.40%/day
# expected_move = 3.40% × $100,000 = $3,400
# step = 0.5 × $3,400 = $1,700 → round → $1,700
```

เทียบ ATR-based (ATR=2,500) :
 # step_atr = 0.5 × \$2,500 = \$1,250
 # → IV-based step ใหญ่กว่า เพราะ market expect volatility สูง

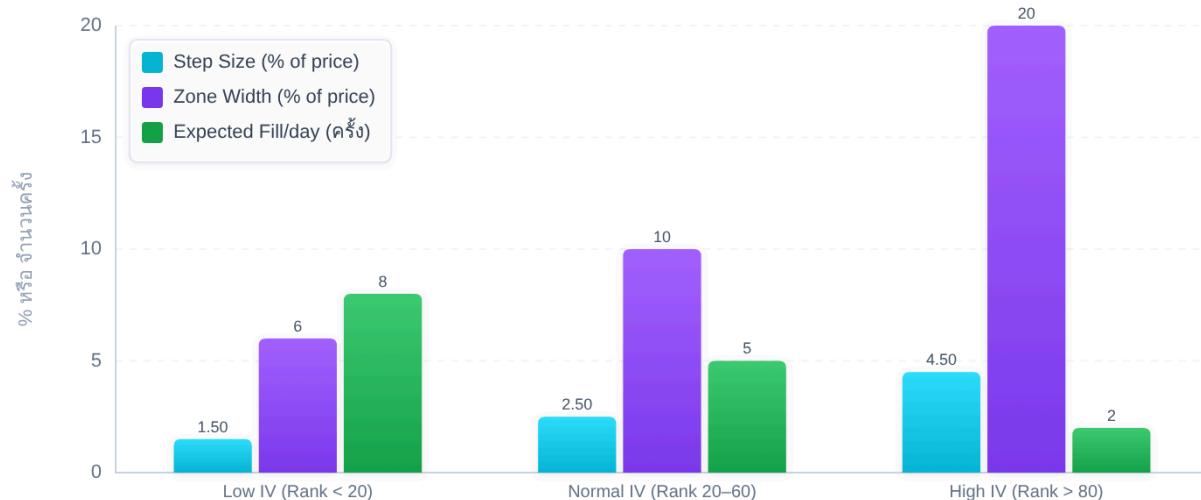
IV Rank — เลือก Zone Width ตาม IV Percentile

$$\text{IV Rank} = (\text{IV_current} - \text{IV_min_52w}) / (\text{IV_max_52w} - \text{IV_min_52w}) \times 100$$

- IV Rank < 20 (low IV): zone แคบ, step เล็ก → grid fill บ่อย (mean-reverting regime)
- IV Rank 20–60 (normal): zone ปกติ, step $\approx 0.5 \times \text{IV_daily} \times P$
- IV Rank 60–80 (elevated): zone กว้างขึ้น, step เพิ่ม 20–40% → ระวัง breakout
- IV Rank > 80 (high IV): zone กว้าง, step ใหญ่ → ป้องกัน excess fill ใน high vol

อ้างอิง: [Vol Series Part 2](#) สำหรับ IV Rank calculation

IV Rank Regime → Grid Parameters (BTC/USDT สมมติ Price=\$100k)



Low IV: Step เล็ก-Fill บ่อย-Zone แคบ / Normal: ปกติ / High IV: Step ใหญ่-Fill น้อย-Zone กว้าง (ป้องกัน excess inventory)

IV Rank กำหนด Step และ Zone Width — IV ต่ำ: grid แน่น / IV สูง: grid กว้าง ป้องกัน over-fill

7.2 Funding Rate Adaptive Grid

Funding Rate ของ BTC Perpetual เปลี่ยนตาม market sentiment — grid สามารถ adapt ตาม Funding Rate ได้ (กลไกว่าทำไม funding มีอยู่และทำไมพลิกทิศได้ ดู Part 1C §1C.3 "ค่าเช่าความห่าง")

เลข $\times 3/\text{วัน}$ ที่ใช้ทั้ง section นี้คือรอบเก็บ funding ของ exchange ที่เก็บทุก 8 ชั่วโมง ไม่ใช่กฎธรรมชาติตายตัว — ควรเช็คคู่มือ exchange จริงก่อนตั้งค่านี้นะบอท:

Funding Rate Grid Rules:

Funding Rate > 0 (Long จ่าย Short รับ):

- Short Layer (Part 1B) ได้ income เพิ่ม
- เพิ่ม Q ของ short layer
- เปิด Grid Hedge (Part 1C) ได้ดี

Funding Rate < 0 (Short จ่าย Long รับ):

- Short layer เสียต้นทุนเพิ่ม
- ลด Q short layer หรือปิด short ชั่วคราว
- ถ้า negative มาก → หยุด hedge

```
def funding_adaptive_short_size(funding_rate_8h, base_q_short,
                                threshold_high=0.001,
                                threshold_low=-0.0005):
    """
    funding_rate_8h: funding rate ต่อ 8 ชั่วโมง (เช่น 0.001 = 0.1%)
    """
    if funding_rate_8h > threshold_high:
        # High positive funding → เพิ่ม short (เก็บ funding มากขึ้น)
        multiplier = min(2.0, 1 + (funding_rate_8h - threshold_high) /
0.001)
    elif funding_rate_8h < threshold_low:
        # Negative funding → ลด short (ลด cost)
        multiplier = max(0.0, 1 + (funding_rate_8h - threshold_low) /
0.001)
    else:
        # Normal range → ไม่เปลี่ยน
        multiplier = 1.0

    return base_q_short * multiplier
```

Funding Income Calculation (สำหรับ Hedge Grid):

$\text{short_notional} = \text{inventory_btc} \times \text{current_price}$

$\text{daily_funding} = \text{short_notional} \times \text{funding_rate_8h} \times 3 \quad \# \text{ 3 periods/วัน}$

$\text{monthly_funding} = \text{daily_funding} \times 30$

ตัวอย่าง:

Inventory = 0.05 BTC, Price = \$100k, Rate = 0.05%/8h

Daily funding = $(0.05 \times \$100k) \times 0.05\% \times 3 = \$5,000 \times 0.15\% = \$7.50/\text{วัน}$

Monthly = $\$7.50 \times 30 = \$225/\text{เดือน}$ (ได้เพิ่มจาก hedge)

Options 101 — พื้นฐานที่ต้องรู้ก่อนเข้าเรื่อง CSP/Strangle/Iron Condor

PUT คือสัญญาที่ให้สิทธิ์ (ไม่ใช่ข้อผูกมัด) แก่ผู้ซื้อในการ "ขาย" สินทรัพย์ที่ราคาที่กำหนดไว้ล่วงหน้า (strike price) ภายในเวลาที่กำหนด (expiry) — **CALL** คือสิทธิ์ "ซื้อ" ในทำนองเดียวกัน

Premium คือราคาที่คุณซื้อ options จ่ายให้ผู้ขาย เพื่อแลกกับสิทธิ์นั้น — เล่นนี้เน้น **"ขาย" (SELL)**
options เป็นหลัก: เมื่อคุณ SELL PUT คุณรับ premium ทันที แต่แลกกับการมีข้อผูกมัด (ไม่ใช่สิทธิ์) ว่าถ้าผู้ซื้อใช้สิทธิ์ คุณต้องซื้อสินทรัพย์ที่ strike price นั้น — นี่คือการทำให้ "SELL PUT" เทียบเท่ากับการตั้ง BUY limit order ที่ strike price พร้อมได้ premium แถม

Assignment คือเหตุการณ์ที่ผู้ซื้อ options ใช้สิทธิ์จริง บังคับให้ผู้ขาย (คุณ) ต้องทำตามข้อผูกมัด — สำหรับ SELL PUT คือถูกบังคับซื้อสินทรัพย์ที่ strike

ITM (In-The-Money) / OTM (Out-of-The-Money): PUT เป็น ITM เมื่อราคาตลาดปัจจุบัน *ต่ำกว่า* strike (มีโอกาสถูก assign สูง) และเป็น OTM เมื่อราคาตลาด *สูงกว่า* strike (โอกาส assign ต่ำ, มักหมดอายุไร้ค่า) — CALL กลับกัน: ITM เมื่อราคาตลาดสูงกว่า strike

"Wings" (ใช้ใน Iron Condor ด้านล่าง) หมายถึง options เพิ่มเติมที่ซื้อ (ไม่ใช่ขาย) ไว้ไกลออกไปจาก short strike เพื่อจำกัด max loss — เหมือน "ปีก" ที่กันความเสี่ยงไม่ให้ลามไม่มีที่สิ้นสุด

7.3 Cash-Secured Put (CSP) + Grid — Inventory Entry

แทนที่จะซื้อ BTC ตรงๆ ผ่าน BUY limit order — ใช้ CSP (Cash-Secured Put) เพื่อซื้อ BTC ในราคาที่ต้องการ พร้อมรับ premium:

CSP + Grid Concept:

ปัญหาของ BUY limit order:

วาง BUY \$90,000 รอ → อาจรอนาน หรือไม่ fill เลย → capital idle

CSP Solution:

SELL PUT \$90,000 expiry 2 สัปดาห์

→ รับ premium \$500 ทันที

→ ถ้า $BTC < \$90,000$ → ถูก assign = ซื้อ BTC ที่ \$90,000 (ตามแผน)

→ ถ้า $BTC > \$90,000$ → PUT หมดอายุ = เก็บ premium \$500 โดยไม่ต้องซื้อ

Net effect:

ซื้อ BTC ที่ effective price = $\$90,000 - \$500 = \$89,500$ (ถูกกว่า limit order)

หรือเก็บ premium \$500 ถ้าไม่ถูก assign

```
def csp_vs_limit_order(target_price, option_premium):
```

```
    """
```

```
    เปรียบ CSP กับ limit order — คำนวณทั้ง 2 scenario ที่เป็นไปได้ ณ expiry  
    (ไม่ต้องรู้ expiry_price ล่วงหน้าเพื่อเปรียบเทียบ เพราะต้องการเปรียบเทียบทั้ง 2 กรณี  
    พร้อมกัน)
```

```
    """
```

```
    # Limit Order: รอ fill ที่ target_price
```

```
    limit_effective_cost = target_price
```

```
    # CSP: 2 scenario ที่เป็นไปได้เมื่อถึง expiry
```

```
    csp_cost_if_assigned = target_price - option_premium          # BTC <  
    strike → assign
```

```
    csp_income_if_not_assigned = option_premium                  # BTC >=  
    strike → หมดอายุ เก็บ premium
```

```
    return {
```

```
        "limit_cost": limit_effective_cost,
```

```
        "csp_cost_if_assigned": csp_cost_if_assigned,
```

```
        "csp_income_if_not_assigned": csp_income_if_not_assigned
```

}

สำหรับ Grid: แทน BUY order แต่ละ level ด้วย CSP

Level \$90k → SELL PUT \$90k strike

Level \$92k → SELL PUT \$92k strike

Level \$94k → SELL PUT \$94k strike

(ขาย multiple CSP แทน multiple limit orders)

ข้อจำกัดของ CSP + Grid

- ต้องการ capital เต็มสำหรับ assignment (เช่น SELL PUT \$90k ต้องมี \$90,000 พร้อม)
- BTC Options บน Deribit/Bybit — liquidity ต่ำกว่า spot หรือ perp
- Expiry ต้องตรงกับ grid horizon (ไม่ควร expiry นานเกินกว่า 2x grid rebalance cycle)
- ใน downtrend แรง: PUT ถูก assign ทุก level → ซื้อ BTC ที่ราคาต่ำลงต่อ (เหมือน Close System)

7.4 Short Strangle + Grid — Income Enhancement

เมื่อ grid อยู่ในโซน sideways + IV สูง: SELL CALL ที่ zone_high และ SELL PUT ที่ zone_low → เก็บ premium จาก "range-bound expectation":

Short Strangle + Grid:

Grid Zone: BTC \$90k-\$110k

Short Strangle:

SELL PUT \$90,000 (strike = zone_low)

SELL CALL \$110,000 (strike = zone_high)

Premium received: PUT \$800 + CALL \$600 = \$1,400 total

Scenarios:

1. BTC อยู่ \$90k-\$110k → Strangle ไม่ถูก assign → เก็บ \$1,400
Grid cashflow ยังทำงาน → Double income
2. BTC < \$90k → PUT assign → ซื้อ BTC \$90k (= zone_low grid ทำงาน)
Grid ก็ BUY ที่ \$90k อยู่แล้ว → Strangle ช่วยลด cost
3. BTC > \$110k → CALL assign → ขาย BTC \$110k (= zone_high grid ก็ขาย)
Grid SELL ที่ \$110k อยู่แล้ว → Strangle ช่วยเพิ่ม income

ข้อดี: ทั้ง 3 scenarios เป็นผลดี — strangle "align" กับ grid zone

ข้อระวัง: ถ้า BTC พุ่งเกิน \$110k ไกลมาก → CALL loss สูง (unlimited loss) → ต้อง hedge CALL

→ ซื้อ OTM CALL เป็น hedge → เปลี่ยนเป็น Iron Condor (max loss ถูก cap)

Iron Condor ตัวอย่าง (Grid zone \$90k-\$110k):

Short Strangle เดิม: SELL PUT \$90k + SELL CALL \$110k → premium \$1,400 (unlimited risk)

Iron Condor (เพิ่ม long wings + ขยับ short strike ออกไปอีกหน่อย (ไกลจาก spot มากขึ้น) ทำให้ premium ผิง short ต้องต่ำกว่าเดิม ไม่ใช่ premium เท่าเดิม):

SELL PUT \$88k + BUY PUT \$82k → net premium: +\$550 - \$100 = +\$450

SELL CALL \$113k + BUY CALL \$120k → net premium: +\$400 - \$120 = +\$280

Total net premium = +\$730 (ลดลงจาก \$1,400 ทั้งจาก long wings และจาก short strike ที่ไกลขึ้น แต่ risk ถูก cap)

Max loss ถ้าทดสอบฝั่ง PUT เท่านั้น: $(\$88k - \$82k) - \$730$ (net premium รวมทั้งคู่) = \$5,270

Max loss ถ้าทดสอบฝั่ง CALL เท่านั้น: $(\$120k - \$113k) - \$730$ (net premium รวมทั้งคู่) = \$6,270

(มีแค่ฝั่งเดียวที่ถูกทดสอบตอน expiry เสมอ – อีกฝั่งหมดอายุไร้ค่า เก็บ premium เต็ม ดังนั้น max loss รวม = $\max(\text{PUT width } \$6k, \text{CALL width } \$7k) - \text{net premium รวม } \730)

เปรียบเทียบ:

Short Strangle: premium \$1,400 · CALL loss ไม่จำกัด

Iron Condor: premium \$730 · max loss ~\$6,270 (defined risk – ไม่ใช่ผลรวม max loss ของแต่ละฝั่งแยกกัน)

```
def grid_strangle_income(zone_low, zone_high, put_premium,
call_premium,
                        grid_daily_cashflow, days_to_expiry):
    """
    ประมาณ total income ต่อ expiry cycle
    """
    option_income = put_premium + call_premium
    grid_income = grid_daily_cashflow * days_to_expiry
    total = option_income + grid_income
    return {
        "option_premium": option_income,
        "grid_cashflow": grid_income,
        "total": total,
        "daily_equivalent": total / days_to_expiry
    }
```

7.5 GARCH-Based Volatility Forecasting สำหรับ Step

GARCH (Generalized Autoregressive Conditional Heteroskedasticity) คือโมเดลทำนาย volatility ที่คำนึงถึง volatility clustering — ช่วง vol สูงมักตามด้วย vol สูงต่อเนื่อง (อ้างอิงพื้นฐานเต็มใน [Vol Series Part 2](#)) ส่วนวิธี fit และใช้ GARCH(1,1) ตั้ง step แบบเต็มรูปแบบนั้น สอนไว้แล้วใน [Part 3 §3.6](#) (รวม fit window 90 วัน, forecast horizon 7 วัน, และตัวอย่างว่า step ปรับตัวเร็วแค่ไหนหลัง flash crash)

GARCH เป็นทางเลือกหนึ่งสำหรับกำหนด step (แทนที่ ATR หรือ IV-adaptive step ใน §7.1) — ตัวอย่าง Running Example ใน §7.6 ด้านล่างใช้ IV-adaptive step เพื่อความง่าย ถ้าต้องการใช้ GARCH แทน ให้แทนที่ Step 1 ด้วยผลลัพธ์จาก Part 3 §3.6 แล้วขั้นตอนที่เหลือ (funding assessment, hedge, strangle) ทำงานเหมือนเดิมทุกประการ

7.6 Running Example — IV + Funding + Grid

Running Example: Advanced Grid ที่ใช้ IV + Funding Rate

Setup: BTC=\$100k · IV(7d)=70% · Funding rate=0.08%/8h · Grid capital=\$20,000

Step 1: IV-Adaptive Step

$iv_daily = 70\% / \sqrt{365} = 3.66\%/day$

$step = 0.5 \times 3.66\% \times \$100k = \$1,830 \rightarrow \$1,800 \text{ (round)}$

✓ ใหญ่กว่า ATR-based (\$1,250) เพราะ market expect vol สูง

Step 2: Funding Rate Assessment

$Rate = 0.08\%/8h = 0.24\%/วัน$

$threshold_high = 0.1\% \rightarrow Rate (0.08\%) < threshold \rightarrow multiplier = 1.0$ (ไม่ปรับ Q)

แต่ positive funding → เปิด hedge short เพื่อเก็บ funding income (ดู Part 1C)

Step 3: Grid Hedge (Part 1C)

Inventory BTC (5 levels): 0.05 BTC worst case

Perp short: $0.05 \text{ BTC} \times \$100k = \$5,000$ notional (10× → \$500 margin)

Daily funding: $\$5,000 \times 0.08\% \times 3 = \$12/วัน$

Step 4: Short Strangle

SELL PUT \$88,000 (zone_low - 2σ buffer) → premium \$400

SELL CALL \$113,000 (zone_high + 1σ buffer) → premium \$350

Total premium = \$750 / 14 วัน = \$53.6/วัน

Step 5: Total Daily Income

Grid cashflow: \$54/วัน

Funding income: \$12/วัน

Strangle income: \$53.6/วัน

Total: \$119.6/วัน = 0.60%/วัน บน \$20,000

✓ เทียบ grid alone: \$54/วัน = 0.27%/วัน (+121% จาก integration)

7.7 แบบฝึกหัด

แบบฝึกหัด Part 7

- ▶ ข้อ 1: BTC IV(7d)=80%, P=\$95,000 — IV-adaptive Step คือเท่าไร?
- ▶ ข้อ 2: Funding rate = $-0.03\%/8h$ · Short notional \$8,000 · Grid cashflow \$40/วัน — Net P&L/วัน?
- ▶ ข้อ 3: Short Strangle: SELL PUT \$88k + SELL CALL \$115k · Premium \$1,200 · Grid คาดว่า BTC อยู่ \$90k–\$112k · Expiry 2 สัปดาห์ — ความเสี่ยงหลักคืออะไร?
- ▶ ข้อ 4: เมื่อไหร่ควรใช้ CSP แทน BUY limit order ใน grid?

Grid-Framework + Indicator Execution Styles

Grid เป็น Framework (Zone/Step/Capital/Risk) · Indicator เป็น Execution Engine

4 Execution Styles: Mean-Reversion · Trend · Volume · Composite

การเลือก Style ตาม Market Condition

Part 7 รวม IV, funding rate, และ options เข้ากับ grid ในบริบทเฉพาะ — Part นี้ถอยกลับมาองภาพรวม: ทุกอย่างที่เราเพิ่งทำใน Part 7 ล้วนเป็นตัวอย่างของหลักการเดียวกัน คือแยก "grid math คงที่" ออกจาก "วิธี execute ที่ปรับได้"

แก่นของ PART นี้

"Grid" ไม่ใช่ strategy เดียว — มันคือ **framework** ที่กำหนด zone, step, capital และ risk — Indicator เป็น "execution engine" ที่ตัดสินใจว่าจะใช้ grid framework นั้น *อย่างไร* ในแต่ละช่วงเวลา ทั้ง 4 styles ใช้ grid math เหมือนกัน แต่ execution logic ต่างกัน

Grid Math (zone/step/Q/risk) = คงที่ · Execution Style =
เปลี่ยนตาม regime

7B.1 Grid เป็น Framework — ไม่ใช่ Strategy

ความเข้าใจผิดทั่วไป: "Grid trading" = strategy หนึ่ง ที่มี rule ชัดเจนว่า "ซื้อทุก \$2k ลง ขายทุก \$2k ขึ้น"

ความจริง: Grid คือ **framework** สำหรับ **organize capital และ risk** — ส่วน execution (ว่าจะ deploy เมื่อไหร่, ขนาดไหน, เพิ่ม/ลดอย่างไร) ขึ้นอยู่กับ indicator execution style



ผลลัพธ์ที่ได้ (สมมติ/illustrative — ไม่ใช่ backtest จริง)

Framework-only: Sharpe 0.65 · DD –4.2% + ADX filter: Sharpe 0.72 · DD –2.8% improved

ตัวอย่าง Style 2 (Trend-Adaptive) เท่านั้น — Style 1/3/4 มี trade-off ต่างกัน ดู §7B.2–7B.5

P&L รวมใกล้เคียงกัน แต่ risk-adjusted return ดีขึ้น — Framework คือ alpha หลัก, Indicator ปรับ Sharpe

ซ้าย: Framework กำหนดโครงสร้าง (zone/step/capital/risk) · ขวา: 4 Execution Styles ใช้ framework เดียวกัน ได้ผลต่างกัน

- Grid Framework Components (คงที่ ไม่เปลี่ยน):
1. Zone: [zone_low, zone_high] — จาก Donchian/Bootstrap (Part 3)
 2. Step: grid spacing — จาก ATR/IV (Part 3, 7)
 3. Capital: grid_capital, Q per level — จาก Kelly (Part 5)
 4. Risk: DD halt, position limits — จาก Part 5

Indicator Execution Engine (เปลี่ยนตาม style/regime):

- เมื่อไหร่ deploy capital (timing)
- ขนาดเท่าไรต่อ level (size)
- เพิ่ม Q ลด Q ตาม signal (adaptive)
- Stop/resume ตาม regime (state machine)

7B.2 Style 1 — Mean-Reversion Execution

Style 1: Mean-Reversion — เหมาะกับ $H < 0.50$, RSI extreme, BB near edge

Deploy ทุก level เท่ากัน, เน้น cashflow ทุกรอบ

```
class MeanReversionExecution:
    """
    Style 1: Deploy grid ทั้งหมดเมื่อ mean-reversion signal ชัด
    เน้น: ทุก level fill บ่อย, cashflow สม่ำเสมอ
    """
    def __init__(self, grid_framework):
        self.grid = grid_framework

    def should_deploy(self, indicators):
        score, _ = composite_grid_score(**indicators)
        return score > 60 # deploy เมื่อ score > 60

    def get_level_sizes(self, n_levels):
        # Equal weight ทุก level
        return [self.grid.base_q] * n_levels

    def should_adjust_size(self, indicators):
        # ไม่ปรับ size - คงที่ตาม Kelly
        return 1.0

    def get_execution_params(self, indicators):
        deploy = self.should_deploy(indicators)
        sizes = self.get_level_sizes(self.grid.n_levels)
        return {"deploy": deploy, "sizes": sizes, "style":
"mean_reversion"}
```

เหมาะกับ: Beginner → Advanced

ข้อดี: ง่าย, predictable, cashflow สม่ำเสมอ

ข้อเสีย: ไม่ adapt ตาม volatility regime

7B.3 Style 2 — Trend-Adaptive Execution

Style 2: Trend-Adaptive — ปรับ Step และ Q ตาม Hurst + ADX

ใช้ Step Pyramid (Part 2) อัลกอริทึมเพิ่ม step เมื่อ H สูง

```
class TrendAdaptiveExecution:
    """
    Style 2: ปรับ step + size ตาม trend strength
    H ต่ำ = step เล็ก Q เต็ม | H สูง = step ใหญ่ Q ลด
    """
    def __init__(self, grid_framework):
        self.grid = grid_framework

    def get_adaptive_step(self, base_step, hurst, adx):
        # Hurst multiplier: สูตรเดียวกับ Step Pyramid ใน Part 2 §2.6 (ไม่รวม
        # ถึงเต็ม% — สัดส่วน capital ที่ deploy ไปแล้วเทียบกับงบสูงสุด ดู Part 2 §2.2 —
        # ตรงนี้ เพราะ style นี้ไม่ track cumulative cost ต่อ level)
        hurst_mult = 1 + min(1.0, max(0, (hurst - 0.50) / 0.08)) * 0.5
        # ADX multiplier: threshold BTC-calibrated เดียวกับ
        get_adaptive_size_mult ด้านล่าง (40/50)
        adx_mult = 1 + min(1.0, max(0, (adx - 40) / 10)) * 0.3
        return base_step * hurst_mult * adx_mult

    def get_adaptive_size_mult(self, hurst, adx):
        # ลด Q เมื่อ trending มากขึ้น (threshold BTC-calibrated ตาม Part 4: ADX 40/50 ไม่ใช่ Forex 25/35)
        if hurst > 0.58 or adx > 50:
            return 0.4
        elif hurst > 0.55 or adx > 40:
            return 0.7
        return 1.0

    def get_execution_params(self, indicators):
        hurst = indicators["hurst30"]
        adx = indicators["adx14"]
```

```
step = self.get_adaptive_step(self.grid.base_step, hurst, adx)
size_mult = self.get_adaptive_size_mult(hurst, adx)
sizes = [self.grid.base_q * size_mult] * self.grid.n_levels

return {
    "deploy": True, # ไม่หยุด เพียงแต่ปรับ
    "step": step,
    "sizes": sizes,
    "style": "trend_adaptive"
}
```

เหมาะกับ: ผู้เข้าใจ Hurst + Step Pyramid

ข้อดี: ไม่ต้อง pause grid เมื่อ trend เริ่ม → ลดทอน trend ผ่าน step ใหญ่

ข้อเสีย: ถ้า trend แรงมาก ($H > 0.65$) ก็ยังขาดทุน unrealized

7B.4 Style 3 — Volume-Adaptive Execution

Style 3: Volume-Adaptive — เพิ่ม Q เมื่อ volume สูง (liquidity ดี), ลด Q เมื่อ volume ต่ำ
เน้น execution quality ใน high-liquidity windows

```
class VolumeAdaptiveExecution:
    """
    Style 3: ปรับ Q ตาม volume profile
    Volume สูง = fill ง่าย, spread แคบ → เพิ่ม Q
    Volume ต่ำ = fill ยาก, spread กว้าง → ลด Q
    """
    def __init__(self, grid_framework, volume_history=None):
        self.grid = grid_framework
        volume_history = volume_history or []
        recent_vols = [vol for vol, _ in volume_history[-20:]]
        self.avg_volume = sum(recent_vols) / len(recent_vols) if
recent_vols else 0
        self.volume_by_hour =
self._compute_hourly_profile(volume_history) if volume_history else {}

    def _compute_hourly_profile(self, volume_history):
        # คำนวณ avg volume ต่อชั่วโมง (0-23)
        hourly = {}
        for vol, timestamp in volume_history:
            hour = timestamp.hour
            hourly.setdefault(hour, []).append(vol)
        return {h: sum(v)/len(v) for h, v in hourly.items()}

    def get_size_multiplier(self, current_volume, current_hour):
        # Volume ratio vs historical average
        vol_ratio = current_volume / self.avg_volume

        # Hourly profile bonus
        hourly_avg = self.volume_by_hour.get(current_hour,
self.avg_volume)
        hourly_ratio = hourly_avg / self.avg_volume

        if vol_ratio > 1.5 and hourly_ratio > 1.2:
```

```
        return 1.3 # เพิ่ม 30% ในช่วง high-liquidity
    elif vol_ratio < 0.5 or hourly_ratio < 0.6:
        return 0.6 # ลด 40% ในช่วง low-liquidity
    return 1.0
```

```
# ช่วงเวลา High Volume สำหรับ BTC:
```

```
# 13:00-15:00 UTC (Europe afternoon + US morning overlap)
```

```
# 20:00-22:00 UTC (US afternoon peak)
```

```
# หลีกเลียง: 03:00-06:00 UTC (US night/sleep – ตรงกับ Asia กลางวันแต่ volume  
รวมยังต่ำสุดของวัน)
```

7B.5 Style 4 — Composite Score Execution

Style 4: Composite Score — รวม RSI + BB + Volume + Hurst เป็น score 0–100

ใช้ score กำหนด deploy % และ size multiplier (จาก Part 3B)

```
class CompositeScoreExecution:
    """
    Style 4: Composite Score จาก Part 3B กำหนดทุกอย่าง
    """
    def __init__(self, grid_framework):
        self.grid = grid_framework

    def get_execution_params(self, indicators):
        score, sub_scores = composite_grid_score(**indicators)

        # Deploy percentage — ใช้ tier เดียวกับ ACTION_TABLE ใน Part 3B
        §3B.6
        # (resolve_composite_action คือ single source of truth ของ tier
        75/55/35)
        action_name, _ = resolve_composite_action(score)
        deploy_pct = {"FULL_DEPLOY": 1.00, "PARTIAL_DEPLOY": 0.60,
                      "MINIMAL_DEPLOY": 0.30, "WAIT": 0.00}
        [action_name]

        # Size multiplier (สูงกว่าก็ deploy per level มากขึ้น)
        size_mult = 0.5 + (score / 100) * 0.5 # 0.5× ถึง 1.0×

        # Level allocation (emphasis on most signal levels)
        rsi_score = sub_scores["rsi"]
        if rsi_score > 70: # RSI extreme → เน้น lower levels
            level_weights = "bottom_heavy"
        elif sub_scores["bb"] > 70: # Near BB → เน้น near-edge
            level_weights = "bell"
        else:
            level_weights = "equal"

        sizes = self.grid.allocate_by_weight(level_weights, size_mult)
```

```
return {  
    "deploy": deploy_pct > 0,  
    "deploy_pct": deploy_pct,  
    "sizes": sizes,  
    "size_mult": size_mult,  
    "score": score,  
    "style": "composite"  
}
```

เหมาะกับ: Quant / systematic trader

ข้อดี: รวม signal ทุกตัว ไม่มี blind spot

ข้อเสีย: ซับซ้อน ต้อง calibrate weights

7B.6 เลือก Execution Style ตาม Trader Profile

Style	Trader Profile	Time Commitment	Key Metric	Expected Return
Mean-Reversion	Beginner / Long-term	ต่ำ (weekly check)	Cashflow/day	0.20–0.35%/วัน
Trend-Adaptive	Intermediate	กลาง (daily check)	Cashflow + less DD	0.18–0.30%/วัน
Volume-Adaptive	Active Trader	สูง (hourly)	Execution quality	0.22–0.38%/วัน
Composite Score	Quant / Advanced	Automated (real-time)	Risk-adjusted return	0.25–0.45%/วัน

Style ไม่ใช่สิ่งที่เลือกครั้งเดียว

ใน practice: ใช้ Mean-Reversion เป็น baseline | เพิ่ม Trend-Adaptive เมื่อ Hurst เริ่มขึ้น | เพิ่ม Volume-Adaptive เมื่อเป็น professional automated grid

Composite Score เป็น superset — รวมทุก style ไปด้วยกัน แต่ต้องการ infrastructure ที่ดีกว่า

7B.7 Unified Grid Controller — รวมทุก Style

```
class UnifiedGridController:
    """
    Grid controller ที่รองรับทุก execution style
    ใช้ config เพื่อเลือก style
    """
    STYLES = {
        "mean_reversion": MeanReversionExecution,
        "trend_adaptive": TrendAdaptiveExecution,
        "volume_adaptive": VolumeAdaptiveExecution,
        "composite": CompositeScoreExecution,
    }

    def __init__(self, grid_framework, style="mean_reversion"):
        self.grid = grid_framework
        self.execution = self.STYLES[style](grid_framework)
        self.state_machine = GridStateMachine() # Part 4
        self.dd_monitor = DrawdownMonitor() # Part 5

    def run_cycle(self, market_data):
        """
        ทำงาน 1 รอบ (ทุก 1 ชั่วโมง):
        1. ตรวจสอบ DD halt
        2. ตรวจสอบ regime state
        3. รับ execution params จาก style
        4. Execute orders
        """
        # Step 1: DD Check (Part 5)
        dd_status, dd_value, _ = self.dd_monitor.check(
            market_data["portfolio_history"]
        )
        if dd_status == "HALT":
            return {"action": "HALT", "reason": f"DD={dd_value:.1%}"}

        # Step 2: Regime Check (Part 4)
        state, signals = self.state_machine.evaluate(
            market_data["hurst30"],
            market_data["adx14"],
            market_data["bb_width"]
        )
```

```
)
if state == "GRID_OFF":
    return {"action": "PAUSE", "state": state}

# Step 3: Execution Style
exec_params = self.execution.get_execution_params(market_data)

# Merge regime multiplier
size_mult = self.state_machine.get_size_multiplier()
exec_params["sizes"] = [q * size_mult for q in
exec_params["sizes"]]

# Step 4: Place orders
orders = self.grid.create_orders(exec_params,
market_data["current_price"])
return {"action": "EXECUTE", "orders": orders, "params":
exec_params}
```

7B.7b Running Example — Style 2 (Trend-Adaptive)

หนึ่งรอบ

Running Example: `run_cycle()` ผ่าน `TrendAdaptiveExecution`

Setup: `base_step=$2,000`, `base_q=0.01 BTC`, `n_levels=5` · Market data: `Hurst=0.54`, `ADX=42`, `DD=-3%` (ไม่ชน `HALT`), state machine ประเมินแล้วได้ `GRID_ON` (regime `size_mult=1.0`)

Step 1: DD Check → `DD=-3%` ไม่ถึง `-8%` → ผ่าน ไม่ `HALT`

Step 2: Regime Check → `state=GRID_ON` (ไม่ใช่ `GRID_OFF`) → ผ่าน ไม่ `PAUSE`

Step 3: Execution Style (Trend-Adaptive, §7B.3)

`hurst_mult = 1 + min(1, max(0, (0.54-0.50)/0.08)) × 0.5 = 1 + 0.5×0.5 = 1.25`

`adx_mult = 1 + min(1, max(0, (42-40)/10)) × 0.3 = 1 + 0.2×0.3 = 1.06`

`step = $2,000 × 1.25 × 1.06 = $2,650`

`size_mult (get_adaptive_size_mult): Hurst=0.54 ไม่>0.58, ADX=42 ไม่>50`

→ เช็คเงื่อนไขถัดไป: `Hurst=0.54 ไม่>0.55` แต่ `ADX=42>40` → เข้าเงื่อนไข → `0.7`

`sizes = [0.01×0.7] × 5 = [0.007, 0.007, 0.007, 0.007, 0.007] BTC`

คูณด้วย regime `size_mult` (จาก state machine, Step 2) = `1.0` → `sizes` ไม่เปลี่ยน

Step 4: Place Orders

`step` ใหญ่ขึ้นจาก `$2,000` → `$2,650` (ตาม trend strength ที่เพิ่มขึ้น)

`Q` ต่อ level ลดลงจาก `0.01` → `0.007 BTC` (ลด exposure เพราะ `ADX` เริ่มสูง)

`action = EXECUTE`, `orders = 5 levels` ที่ `step $2,650`, `Q 0.007`

`BTC/level`

ลองเปลี่ยน Hurst เป็น 0.60 หรือ ADX เป็น 55 ดูว่า size_mult ตกไปที่ 0.4 (เงื่อนไขแรก) แทน — นี่คือการที่ execution style "ตอบสนอง" ต่อ regime โดยไม่ต้องเปลี่ยน grid framework (zone/capital) เลย

7B.8 แบบฝึกหัด

แบบฝึกหัด Part 7B

- ▶ ข้อ 1: เหตุใด Grid Framework (zone/step/risk) ถึงไม่ใช่ส่วนที่ "ปรับตาม market" แต่ Execution Style ควรเปลี่ยนได้?
- ▶ ข้อ 2: $\text{Composite Score} = 48 \cdot \text{Style} = \text{Mean-Reversion}$ — deploy ที่เปอร์เซ็นต์? $\text{Style} = \text{Composite}$ — deploy ที่เปอร์เซ็นต์?
- ▶ ข้อ 3: Volume-Adaptive Style ทำไม่ถึงเท่ากับ "Active Trader" มากกว่า Beginner?
- ▶ ข้อ 4: UnifiedGridController ตรวจสอบ DD Halt ก่อน Regime State — ทำไมลำดับนี้สำคัญ?

5 Supplementary Strategies

Rebalancing Grid · Stablecoin Grid · Flash Crash Grid

Cross-Exchange Arbitrage Grid · Laddered CSP Grid

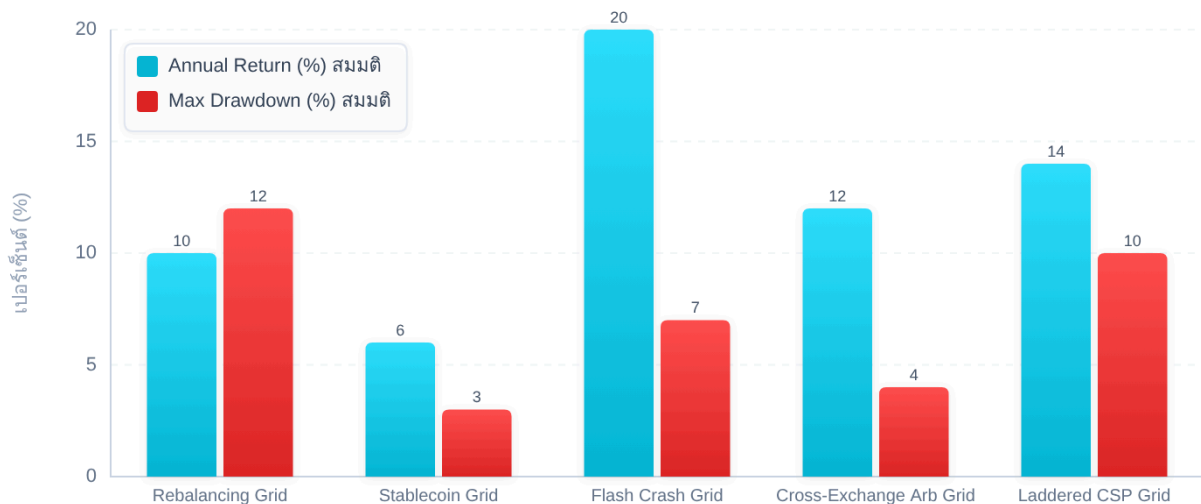
แต่ละ strategy ใช้ core grid mechanics แต่ context ต่างกัน

เหมือนที่ Part 7B แยก Framework (grid math คงที่) ออกจาก Execution Style (ปรับตาม regime) — 5 กลยุทธ์นี้คือตัวอย่างว่า framework เดิม (zone/step/Q/Close System) ถูกนำไป apply ในบริบทที่ต่างจาก standard grid อย่างไร

แก่นของ PART นี้

5 strategies นี้ไม่ใช่ "แบบอื่น" ของ grid — ทั้งหมดใช้ core mechanics เดิม (zone, step, Q, Close System) แต่ apply ใน context พิเศษ เพื่อ capture โอกาสที่ standard grid ทำไม่ได้

5 Supplementary Grid Strategies — Return vs Risk Profile (สมมติ)



Flash Crash: return สูงสุดแต่เกิดน้อยครั้ง / Stablecoin: risk ต่ำสุด / Rebalancing: สม่ำเสมอตลอดปี — ตัวเลขเป็น illustration เท่านั้น

เปรียบเทียบ 5 กลยุทธ์เสริม — แต่ละกลยุทธ์เหมาะกับ market regime ที่ต่างกัน

STRATEGY 1

8.1 Rebalancing Grid — ทำกำไรจาก Portfolio Drift

ใช้ grid mechanics เพื่อ rebalance portfolio โดยอัตโนมัติ — แทนที่จะ rebalance ทุกเดือนแบบ fixed, grid ทำ continuous rebalancing ตาม price movement:

Rebalancing Grid Setup:

เป้าหมาย: รักษา BTC/USDT = 50%/50% ของ portfolio

Portfolio: \$20,000 total (\$10,000 BTC + \$10,000 USDT)

BTC = \$100k → จำนวน BTC = 0.10

เมื่อ BTC ขึ้น:

BTC value เพิ่ม → % เกิน 50% → grid SELL BTC อัตโนมัติ (TP fills)

ขาย BTC → ได้ USDT → ratio กลับมา 50/50

เมื่อ BTC ลง:

BTC value ลด → % ต่ำกว่า 50% → grid BUY BTC อัตโนมัติ (BUY fills)

ซื้อ BTC → ใช้ USDT → ratio กลับมา 50/50

กำไรจาก: "Buy low, sell high" ทุกครั้งที่ rebalance

→ ได้ทั้ง rebalancing benefit + grid cashflow

```
def rebalancing_grid(total_value, target_btc_pct, btc_price,
step_pct=0.02):
    """
    Grid ที่ทำ portfolio rebalancing
    step_pct: % deviation ก่อน trigger
    """
    target_btc_value = total_value * target_btc_pct
    target_btc_qty = target_btc_value / btc_price

    # Grid levels รอบ target allocation
    levels = []
    for i in range(-5, 6):
        level_price = btc_price * (1 + i * step_pct)
        if i < 0: # BTC ลง → BUY
            levels.append({"type": "BUY", "price": level_price,
                           "qty": target_btc_qty * abs(i) *
0.01})
        elif i > 0: # BTC ขึ้น → SELL
```

```
levels.append({"type": "SELL", "price": level_price,  
              "qty": target_btc_qty * i * 0.01})  
  
return levels
```

⚠️ ความเสี่ยง

Whipsaw cost: ถ้าราคาแกว่งกลับไปกลับมารอบ level เดียวกันบ่อยๆ จะเสีย fee ซ้ำๆ โดยไม่ได้ประโยชน์จาก rebalancing จริง (คล้าย whipsaw ใน trend-following ทั้งหมด)

Underperform buy-and-hold ใน trend แรง: ถ้า BTC ขึ้นต่อเนื่องยาวๆ กลยุทธ์นี้ขาย BTC ออกเรื่อยๆ ตาม rebalance ทำให้ได้ upside น้อยกว่าแค่ถือเฉยๆ — นี่คือ trade-off ปกติของการ rebalance เพื่อคุม risk ไม่ใช่ bug

STRATEGY 2

8.2 Stablecoin Grid — ทำกำไรใน Bear Market

เมื่อ BTC ขาลงและ Hurst สูง (trending down) — grid BTC หยุด แต่ยังสามารถทำ Stablecoin Grid ได้ (ไม่ต้องถือ BTC):

Stablecoin Grid Options:

1. USDT/USDC Grid: ซื้อ USDT ตอนราคาต่ำกว่า USDC ขายตอนราคาสูงกว่า USDC (ช่วงเกิด depeg event)

Zone: \$0.995-\$1.005 · Step: \$0.0005

กำไรต่อรอบ: $Q \times \$0.0005 - \text{fee}$ (ต่ำมาก แต่ปลอดภัย)

2. Stablecoin Yield + Grid:

USDT ใน Standby Zone (Part 1A) → ผักใน lending protocol (Aave/Compound —

ดู "Counterparty Risk" box ใน Part 1A §1A.3 เรื่อง smart contract risk/liquidity risk)

ได้ yield 4-8% APY บน idle capital

+ เมื่อ BTC กลับมา → นำ USDT ออก deploy grid ใหม่

3. USDT/BTC ที่ระดับ Support แรง:

ถ้าเชื่อว่า BTC มี hard support เช่น \$70k → เปิด grid ที่ \$65k-\$80k เป็น "Bottom Fishing Grid" ใน Bear Market

```
def stablecoin_grid_yield(standby_capital, lending_apy=0.05,
grid_days=90):
```

```
    """
```

```
    คำนวณ yield จาก idle capital ใน bear market
```

```
    """
```

```
    daily_rate = (1 + lending_apy) ** (1/365) - 1
```

```
    total_yield = standby_capital * ((1 + daily_rate) **
grid_days - 1)
```

```
    return total_yield
```

```
# ตัวอย่าง: Standby $6,000 · APY 5% · 90 วัน
```

```
# Yield = $6,000 × ((1.000133)^90 - 1) = $6,000 × 1.21% ≈ $72.6
```

 ความเสี่ยง

Depeg risk ไม่ได้มีแค่โอกาสทำกำไร — ถ้า stablecoin ที่ถืออยู่ depeg ถาวร (ไม่กลับ \$1) เงินต้นเสียหายจริง ไม่ใช่แค่โอกาส ไม่ใช่แค่ upside

Option 2 (lending yield) มี counterparty/smart-contract risk เหมือนที่เตือนไว้ใน Part 1A §1A.3 — yield ไม่ใช่ risk-free

STRATEGY 3

8.3 Flash Crash Grid — Capture Extreme Dips

Flash Crash Grid วางเกณฑ์และแผนไว้ล่วงหน้า เพื่อจับช่วง "ต่ำผิดปกติ" ที่ตลาดน่าจะ recover ได้ภายใน 24–72 ชั่วโมง — แต่ order จริงจะถูกวางทันทีหลัง crash เกิดขึ้น ไม่ใช่วางรอไว้ก่อน (รายละเอียดดู Setup ด้านล่าง):

Flash Crash Grid:

ลักษณะ Flash Crash ที่เหมาะ:

- BTC ลง > 15% ภายใน 1 ชั่วโมง (liquidation cascade)
[เหตุผล 15%: ได้ระดับนี้ market makers มักเข้าซื้อ + liquidation ล้างแล้ว + ราคาต่ำกว่า fair value ชัดเจน]
- Volume spike > 5× average
- ระดับ macro เหมือนเดิม (on-chain ยังแข็งแกร่ง)
- ไม่มี fundamental bad news

Setup:

วาง BUY orders ทันทีหลัง crash (ไม่ใช่ก่อน crash)

Zone: ราคาปัจจุบัน (หลัง crash) $\pm 5\%$

Step: ATR ปัจจุบัน (อาจสูงมาก)

Close System: ใช้ capital ที่เตรียมไว้ในส่วน "Flash Crash Reserve" เท่านั้น

```
class FlashCrashGrid:
```

```
    def __init__(self, crash_capital_pct=0.10,
recovery_hours=72):
        """
        crash_capital_pct: % ของ total capital สำหรับ flash crash
        recovery_hours: เวลาที่คาดว่าจะ recover ได้
        """
        self.capital_pct = crash_capital_pct
        self.recovery_hours = recovery_hours

    def is_flash_crash(self, drop_pct, volume_ratio,
time_elapsed_hours):
        """
        ตรวจสอบว่าเป็น flash crash หรือ trend reversal
        """
        is_large_drop    = drop_pct > 0.15           # ลง > 15% (ตรง
```

กับเกณฑ์ด้านบน)

```
is_fast          = time_elapsed_hours < 1  # เร็ว < 1 ชั่วโมง  
is_vol_spike     = volume_ratio > 5.0      # volume > 5x
```

ปกติ

```
return is_large_drop and is_fast and is_vol_spike
```

Risk: Flash Crash กลายเป็น Trend Reversal → Close System รักษา inventory

TP: ปิด grid เมื่อ recover 50% ของ crash depth หรือภายใน recovery_hours

ถ้าไม่ recover ภายใน recovery_hours: ลด inventory 50% เพื่อ free capital → ถือ BTC ที่เหลือในฐานะ long-term หรือโอนเข้า Stablecoin Grid (Strategy 2)

STRATEGY 4

8.4 Cross-Exchange Arbitrage Grid

เมื่อ BTC price บน 2 exchanges ต่างกัน — ใช้ grid ทั้งสองฝั่งพร้อมกัน capture spread:

Cross-Exchange Grid:

Exchange A (เช่น Binance): BTC = \$100,050

Exchange B (เช่น Bybit): BTC = \$99,950

Spread = \$100 (ต่าง 0.1%)

Grid Strategy:

Exchange A: SELL BTC @ \$100,050 (ขายแพง)

Exchange B: BUY BTC @ \$99,950 (ซื้อถูก)

Net profit per cycle: \$100 - fees

```
def cross_exchange_spread_grid(price_a, price_b, q, fee_a,
fee_b):
    """
    คำนวณกำไรจาก cross-exchange spread
    """
    spread = abs(price_a - price_b)
    fee_cost = q * (price_a * fee_a + price_b * fee_b)
    net_profit = q * spread - fee_cost

    return {
        "gross_spread": spread,
        "fee_cost": fee_cost,
        "net_profit": net_profit,
        "profitable": net_profit > 0
    }
```

ความท้าทาย:

1. Transfer BTC ระหว่าง exchange ใช้เวลา (10-60 นาที) → spread อาจหายไปแล้ว
2. ต้องมี capital บน ทั้งสอง exchange ตลอด
3. ค่า withdrawal fee + network fee กินกำไร
4. Slippage เมื่อ Q ใหญ่

วิธีแก้: ใช้ Perpetual บน Exchange A (short) + Spot บน Exchange B

(long)

→ ไม่ต้อง transfer, ใช้ funding rate เป็น P&L indicator

STRATEGY 5

8.5 Laddered CSP Grid — ขาย Put เป็น Grid

กลไก CSP พื้นฐาน (ขาย put รับ premium แทนวาง BUY limit) สอนเต็มแล้วใน [Part 7 §7.3](#) — กลยุทธ์นี้ต่อยอดด้วยการขายเป็น "ladder" ข้าม expiry หลายช่วงพร้อมกัน แทนที่จะใช้ expiry เดียวทุก level:

Laddered CSP Grid:

ตัวอย่าง: BTC = \$100k · Grid capital \$30,000 · 5 levels

Level 1: SELL PUT \$98k, expiry 2W, premium \$600

Level 2: SELL PUT \$95k, expiry 2W, premium \$450

Level 3: SELL PUT \$92k, expiry 4W, premium \$700

Level 4: SELL PUT \$88k, expiry 4W, premium \$550

Level 5: SELL PUT \$84k, expiry 8W, premium \$800

Total premium = \$3,100 (รับทันที)

ถ้าไม่มี level ถูก assign ตลอด 8 สัปดาห์:

Income = \$3,100 premium = 10.3% บน \$30,000 capital ใน 8 สัปดาห์

ถ้า BTC ลงมา \$94k ตอน Level 1-2 หมดอายุ (ทั้งคู่ expiry 2W เท่ากัน):

Level 1 (strike \$98k): $\$94k < \$98k \rightarrow \text{ITM (In-The-Money - ราคาตลาดต่ำกว่า strike ของ PUT ที่ขายไว้)} \rightarrow \text{assign} \rightarrow \text{ซื้อ BTC ที่ } \$98k \text{ (ต้นทุนสุทธิหลังหัก premium} = \$97,400)$

Level 2 (strike \$95k): $\$94k < \$95k \rightarrow \text{ITM} \rightarrow \text{assign} \rightarrow \text{ซื้อ BTC ที่ } \$95k \text{ (ต้นทุนสุทธิ} = \$94,550)$

Level 3-5 (strike \$92k/\$88k/\$84k, expiry 4W/8W ยังไม่ถึง): ยังคง pending

ข้อดีที่ต่างจาก CSP เดี่ยว (Part 7 §7.3):

- + Expiry กระจาย (2W/4W/8W) → assignment ไม่มาพร้อมกันทั้งหมดในวันเดียว
- + Strike ไกลออกไป + expiry ยาวขึ้น (Level 5) → premium ต่อสัญญาสูงกว่า

ข้อจำกัดเพิ่มจากการ ladder ข้าม expiry:

- ต้องบริหารหลาย expiry date พร้อมกัน (roll/ปิดแต่ละ level ไม่พร้อมกัน)
- ข้อจำกัดพื้นฐานของ CSP (liquidity, capital lock, downtrend assign ทุก level) ยังคงอยู่ — ดู Part 7 §7.3

```
def laddered_csp_grid(grid_levels, btc_price, put_premiums):  
    """  
    คำนวณ total income และ effective buy prices  
    """  
    results = []  
    for price, premium in zip(grid_levels, put_premiums):  
        effective_cost = price - premium  
        results.append({  
            "strike": price,  
            "premium": premium,  
            "effective_cost": effective_cost,  
            "discount_pct": premium / price * 100  
        })  
    return results
```

8.6 แบบฝึกหัด

แบบฝึกหัด Part 8 — Quick Review

- ▶ ข้อ 1: ใน Bear Market ที่ BTC downtrend ชัด — Strategy ไหนในทั้ง 5 ข้อที่เหมาะสมที่สุด?
- ▶ ข้อ 2: Cross-Exchange Grid กำไรจากอะไร และทำไมถึงทำได้ยากในทางปฏิบัติ?
- ▶ ข้อ 3: Laddered CSP Grid ให้ effective buy price ต่ำกว่า Limit Order Grid อย่างไร?
- ▶ ข้อ 4: Flash Crash Grid ต่างจาก Standard Buy-Only Grid อย่างไร และทำไมต้องใช้ capital แยก (Flash Crash Reserve)?
- ▶ ข้อ 5: Rebalancing Grid — Portfolio \$20,000 (target 50% BTC ที่ $BTC = \$100k$), $step_pct = 2\%$ — ที่ level $i = +2$ (ราคาขึ้นเป็น \$104k) ต้องขาย BTC เท่าไหร่ตามสูตร?

GRID TRADING MASTERY · PART 9

Python Implementation Guide

Project Structure · Core Modules · Bybit API Integration

Backtesting Framework · Paper Trading → Live Migration

การ deploy Bot บน Server + Monitoring



แก่นของ PART นี้

Part นี้ นำ pseudocode จาก Part 1–8 มาประกอบเป็น Python project ที่ runnable ได้จริง
ทุกโค้ดที่เขียนในเล่มนี้ออกแบบมาให้ต่อกันได้ใน architecture ที่แสดงด้านล่าง

9.1 Project Structure

อ่านยังไงถ้าไม่เขียนโค้ด

Part นี้เต็มไปด้วยโค้ด Python — ถ้าไม่ได้ตั้งใจจะ implement เองก็ไม่จำเป็นต้องอ่านทุกบรรทัด ให้เน้นอ่าน 3 อย่าง: (1) โครงสร้างไฟล์ด้านล่าง เพื่อเห็นภาพว่าแนวคิดจาก Part 1–8 แต่ละเรื่อง ถูกแบ่งเป็นไฟล์ไหน (2) comment ในโค้ด (บรรทัดที่ขึ้นต้นด้วย #) ซึ่งอธิบายเหตุผล เป็นภาษาคนอ่านง่าย และ (3) กล่อง "ตัวอย่าง"/"สรุป" ท้ายแต่ละหัวข้อ ส่วน syntax เฉพาะของ Python (เช่น @dataclass, class, self) ข้ามได้ถ้าไม่คุ้น — comment ข้างๆ จะบอกว่าโค้ดส่วนนั้นทำอะไรอยู่แล้ว

```
grid-trading-bot/
├── core/
│   ├── __init__.py
│   ├── grid_framework.py      # Zone/Step/Capital/Risk (Part 1A, 3, 5)
│   ├── state_machine.py      # Regime Detection (Part 4)
│   ├── execution_styles.py    # 4 Execution Styles (Part 7B)
│   └── dd_monitor.py          # Drawdown Monitor (Part 5)
├── strategies/
│   ├── buy_only.py            # Part 1A
│   ├── bidirectional.py       # Part 1B
│   ├── hedge_grid.py          # Part 1C
│   ├── pair_grid.py           # Part 6
│   └── snowball.py             # Part 6B
├── indicators/
│   ├── hurst.py               # Hurst Exponent (Part 4)
│   ├── atr_donchian.py        # ATR, Donchian, Keltner (Part 3)
│   ├── composite_score.py     # RSI/BB/Vol/Hurst score (Part 3B)
│   └── kelly.py               # Kelly Criterion (Part 5)
├── exchange/
│   ├── bybit_client.py        # Bybit API wrapper
│   └── data_feed.py           # Market data fetcher
├── backtest/
│   ├── backtest_engine.py     # Event-driven backtester
│   ├── metrics.py             # Sharpe, Calmar, Max DD
│   └── bootstrap_zone.py       # Block Bootstrap (Part 3)
├── monitoring/
│   └── watchdog.py            # Grid Watchdog (Part 4)
```

```
|   ├── alerts.py           # LINE/Telegram notify
|   └── dashboard.py        # Simple web dashboard
└── config/
    ├── config.yaml         # All parameters
    └── secrets.yaml        # API keys (gitignored)
└── tests/
    └── test_*.py           # Unit tests
└── main.py                 # Entry point
└── requirements.txt
```



```

"""
สร้าง GridFramework จากข้อมูลตลาด (Part 3) + คำนวณ grid_capital
จาก
Kelly sizing (Part 5) ในฟังก์ชันนี้เอง – รับ net_worth ดิบเข้ามา
(ไม่ใช่ grid_capital ที่คุณมาแล้ว) เพื่อบังคับให้ kelly_fraction
ถูกใช้จริงทุกครั้งที่เราเรียกฟังก์ชันนี้ ไม่ใช่แค่ parameter ธรรมดา
"""

from indicators.atr_donchian import compute_atr,
compute_donchian

grid_capital = net_worth * kelly_fraction
current_price = prices[-1]

# Zone from Donchian (Part 3)
dc_high, dc_low = compute_donchian(highs, lows, period=30)
zone_high = dc_high * 1.05
zone_low = dc_low * 0.95

# Step from ATR (Part 3) – ปิดทั้งสองขา (ATR-based กับ fee-floor)
เป็น
# หลักร้อยก่อนเทียบ max กัน เพื่อให้ step สุดท้ายเป็นจำนวนเต็มร้อยเสมอ
atr = compute_atr(highs, lows, prices, period=14)
atr_step = round(0.5 * atr / 100) * 100
fee_floor = round(3 * current_price * 0.001 * 2 / 100) * 100
step = max(atr_step, fee_floor)

n = int((zone_high - zone_low) / step)
avg_p = (zone_high + zone_low) / 2
base_q = grid_capital / (n * avg_p)

return cls(symbol=symbol, zone_high=zone_high,
zone_low=zone_low,
step=step, grid_capital=grid_capital,
base_q=round(base_q, 4))

def create_buy_levels(self, current_price, sizes=None, step=None) -
> List[GridLevel]:
    """วาง BUY orders ต่ำกว่า current_price (step=None ใช้ self.step
ปกติ,
หรือรับ step ที่ปรับแล้วจาก execution style เช่น Step Pyramid)"""
    step = step if step is not None else self.step
    levels = []
    price = current_price - step
    i = 0

```

```

while price >= self.zone_low:
    q = sizes[i] if sizes else self.base_q
    levels.append(GridLevel(
        price=round(price, 0),
        qty=q,
        side="BUY",
        tp_price=round(price + step, 0)
    ))
    price -= step
    i += 1
return levels

def allocate_by_weight(self, style, size_mult=1.0) -> List[float]:
    """คำนวณ sizes ตาม allocation style (Part 2)"""
    import math
    n = self.n_levels
    if style == "equal":
        weights = [1.0] * n
    elif style == "bell":
        center = n // 2
        weights = [math.exp(-0.5 * ((i-center)/(n/4))**2) for i in
range(n)]
    elif style == "bottom_heavy":
        weights = [1.0/(n-i) for i in range(n)]
    else:
        weights = [1.0] * n
    total = sum(weights)
    return [self.base_q * (w/total) * n * size_mult for w in
weights]

def create_orders(self, exec_params, current_price) ->
List[GridLevel]:
    """
    แปลง exec_params จาก execution style (Part 7B) เป็น order list
จริง
step/sizes มาจาก exec_params ถ้ามี ไม่จ้้นใช้ default ของ framework
เอง
    """
    if not exec_params.get("deploy", True):
        return []
    return self.create_buy_levels(
        current_price,
        sizes=exec_params.get("sizes"),

```

```
        step=exec_params.get("step")  
    )
```

9.3 Bybit API Integration

ทำไมต้องมี "signature"

Bybit (และ exchange อื่นแทบทุกที่) ต้องพิสูจน์ว่า request ที่ส่งเข้ามาจริงๆ มาจากเจ้าของ API key — ไม่ใช่ใครก็ได้ที่ดักฟัง network แล้วปลอมคำสั่งซื้อขาย วิธีที่ใช้กันคือ HMAC (Hash-based Message Authentication Code): เอา secret key มา "เซ็นชื่อ" ข้อความ (params + timestamp) ให้กลายเป็น hash หนึ่งก้อน แล้วแนบ hash นั้นไปกับ request แทนที่จะส่ง secret key ตรงๆ ฝั่ง server คำนวณ hash แบบเดียวกันด้วย secret key ที่เก็บไว้ แล้วเทียบว่าตรงกันไหม ถ้าตรง = request นี้มาจากเจ้าของ key จริง โค้ดด้านล่าง (`_sign()`) ทำขั้นตอนนี้

```
# exchange/bybit_client.py

import hmac
import hashlib
import time, requests
from typing import Optional

class BybitClient:
    BASE_URL = "https://api.bybit.com"

    def __init__(self, api_key: str, api_secret: str, testnet: bool = True):
        self.api_key = api_key
        self.api_secret = api_secret
        if testnet:
            self.BASE_URL = "https://api-testnet.bybit.com"

    # ⚠ Security: Never log api_secret or the resulting signature.
    # Load api_secret from env var only: os.environ["BYBIT_API_SECRET"]
    def _sign(self, params: dict, timestamp: int) -> str:
        query = "&".join(f"{k}={v}" for k, v in sorted(params.items()))
        msg = f"{timestamp}{self.api_key}5000{query}"
        return hmac.new(self.api_secret.encode(), msg.encode(),
                        hashlib.sha256).hexdigest()

    def _request(self, method: str, endpoint: str, params: dict = None):
```

```

ts = int(time.time() * 1000)
params = params or {}
sig = self._sign(params, ts)
headers = {
    "X-BAPI-API-KEY": self.api_key,
    "X-BAPI-SIGN": sig,
    "X-BAPI-TIMESTAMP": str(ts),
    "X-BAPI-RECV-WINDOW": "5000",
}
url = self.BASE_URL + endpoint
if method == "GET":
    resp = requests.get(url, params=params, headers=headers)
else:
    resp = requests.post(url, json=params, headers=headers)
return resp.json()

def place_order(self, symbol: str, side: str, qty: float, price:
float,
                order_type: str = "Limit") -> dict:
    params = {
        "category": "spot",
        "symbol": symbol,
        "side": side,
        "orderType": order_type,
        "qty": str(qty),
        "price": str(price),
        "timeInForce": "GTC",    # Good Till Cancel
    }
    return self._request("POST", "/v5/order/create", params)

def cancel_all_orders(self, symbol: str) -> dict:
    return self._request("POST", "/v5/order/cancel-all",
                        {"category": "spot", "symbol": symbol})

def get_orderbook(self, symbol: str, limit: int = 5) -> dict:
    return self._request("GET", "/v5/market/orderbook",
                        {"category": "spot", "symbol": symbol,
"limit": limit})

def get_wallet_balance(self, coin: str = "USDT") -> float:
    resp = self._request("GET", "/v5/account/wallet-balance",
                        {"accountType": "UNIFIED"})
    for item in resp.get("result", {}).get("list", [{}])
[0].get("coin", []):

```

```
        if item["coin"] == coin:
            return float(item["walletBalance"])
    return 0.0
```

9.4 Backtesting Engine

```
# backtest/backtest_engine.py

import pandas as pd
import numpy as np
from core.grid_framework import GridFramework, GridLevel

class GridBacktester:
    """
    Event-driven backtester สำหรับ Buy-Only Spot Grid
    """
    def __init__(self, grid: GridFramework, fee_rate: float = 0.001):
        self.grid = grid
        self.fee_rate = fee_rate

    def run(self, ohlcv_df: pd.DataFrame) -> dict:
        """
        ohlcv_df: DataFrame ที่มีคอลัมน์ open, high, low, close, volume
        """
        capital = self.grid.grid_capital
        usdt = capital      # เริ่มต้นด้วย USDT ทั้งหมด
        btc = 0.0           # ไม่มี BTC เริ่มต้น
        inventory = {}      # {buy_price: qty}

        trades = []
        portfolio_values = []

        # สร้าง BUY levels
        current_price = ohlcv_df["close"].iloc[0]
        buy_levels = self.grid.create_buy_levels(current_price)

        for idx, row in ohlcv_df.iterrows():
            low_price = row["low"]
            high_price = row["high"]
            close = row["close"]

            # หมายเหตุ: logic นี้เป็น optimistic assumption
            # ราคา candle แต่ละ level ไม่ได้หมายความว่า fill ทุกครั้ง
            # ใน production ให้ใช้ WebSocket order events แทน
```

```

        # Check BUY fills (price ลงถึง BUY level)
        for level in buy_levels:
            if not level.filled and low_price <= level.price:
                cost = level.qty * level.price * (1 +
self.fee_rate)

                if usdt >= cost:
                    usdt -= cost
                    btc += level.qty
                    inventory[level.price] = level.qty
                    level.filled = True
                    trades.append({"date": idx, "side": "BUY",
                                "price": level.price, "qty":
level.qty})

        # Check SELL fills (TP hit)
        for buy_price, qty in list(inventory.items()):
            tp_price = buy_price + self.grid.step
            if high_price >= tp_price:
                proceeds = qty * tp_price * (1 - self.fee_rate)
                usdt += proceeds
                btc -= qty
                del inventory[buy_price]
                # Reset BUY level
                for level in buy_levels:
                    if level.price == buy_price:
                        level.filled = False

                trades.append({"date": idx, "side": "SELL",
                            "price": tp_price, "qty": qty})

        # Portfolio value
        portfolio_values.append(usdt + btc * close)

    # Metrics
    pv = pd.Series(portfolio_values, index=ohlcv_df.index)
    returns = pv.pct_change().dropna()
    max_dd = (pv / pv.cummax() - 1).min()
    sharpe = returns.mean() / returns.std() * np.sqrt(365) if
returns.std() > 0 else 0

    return {
        "final_portfolio": portfolio_values[-1],
        "total_return": (portfolio_values[-1] / capital - 1),
        "max_drawdown": max_dd,

```

```

"sharpe_ratio": sharpe,
"n_trades": len(trades),
"trades": pd.DataFrame(trades),
"portfolio_values": pv,
}

```

เช็คลิสต์อ่านผล backtest ที่คนแชร์กัน — ก่อนเชื่อตัวเลขใคร (รวมของตัวเอง)

Backtester ข้างบนสอนให้เราสร้างผลทดสอบ — ทักษะคู่กันที่สำคัญไม่แพ้กันคือการอ่านผลทดสอบของคนอื่นให้ออก โดยเฉพาะระบบกริด/ถัวเฉลี่ยที่แชร์กันตามโซเชียล ตัวเลขสวยๆ ผ่าน 5 ข้อนี้ก่อนค่อยเชื่อ:

1. **Sanity check ขนาด position เทียบทุน** — จำนวนเองเสมอว่าราคาขยับ 1 หน่วยแล้วพอร์ตขยับกี่ % ตัวอย่างที่เจอบ่อย: น้ำมัน WTI 0.3 lot (300 barrels) บนทุน \$2,000 → ราคาขยับ \$1 = $\pm \$300$ = **15% ของพอร์ต** ถ้าระบบเคลม Max DD -16% แปลว่าระบบแทบไม่เคยถือไม้สวนเกิน \$1 เลยตลอดหลายปี — เป็นไปได้แต่ต้องการหลักฐานระดับเดียวกับความมหัศจรรย์ของเคลม
2. **Profit Factor สูงผิดปกติ + Sharpe ธรรมดา = ลายเซ็นของระบบ basket/close** — ระบบที่ realize กำไรเล็กน้อยครั้งแต่ช้อนขาดทุนไว้ใน floating drawdown จะมี PF สูงเวอร์โดยโครงสร้าง (PF นับเฉพาะเทรดที่ปิดแล้ว — ดู Basket TP ใน Part 2 §2.6.1) ให้น้ำหนักกับ Sharpe, Calmar และ Max DD มากกว่า PF เสมอสำหรับระบบตระกูลนี้
3. **ตรวจหน่วยของ metric** — เช่น "Expected Payoff 2.8 เท่า" เป็นการใช้นิยามผิด (Expected Payoff มีหน่วยเป็นเงินต่อเทรด ไม่ใช่เท่า) ใครใช้หน่วย metric ผิด มีแนวโน้มอ่านรายงาน backtest ของตัวเองไม่แตกด้วย
4. **หาต้นทุนที่หายไป** — swap/rollover สำหรับ CFD ที่ถือข้ามคืน (หนักมากใน commodity), contango/backwardation, spread ที่ถ่างช่วงข่าว, funding rate สำหรับ perpetual — ระบบถัวเฉลี่ยถือไม่นานโดยนิยาม ต้นทุนพวกนี้จึงไม่ใช่เศษเงิน
5. **สงสัยคุณภาพข้อมูลช่วง event** — เหตุการณ์สุดขั้วอย่างน้ำมันติดลบ เม.ย. 2020 มักไม่มีอยู่จริงในข้อมูลย้อนหลังของ broker (โดน clip/filter ทิ้ง) — คำเคลมว่า "backtest ผ่านวิกฤต X มาแล้ว" จึงต้องถามต่อว่าข้อมูลช่วงนั้นสมบูรณ์แค่ไหน

หลักคิดรวบยอด: metric ทุกตัวมีเงื่อนไขเชิงโครงสร้างของระบบกำกับอยู่ — ตัวเลขเดียวกันจากระบบคนละโครงสร้างมีความหมายคนละอย่าง อ่านตัวเลขโดยไม่รู้โครงสร้าง = อ่านไม่ออก

9.5 Main Entry Point + Config

```
# config/config.yaml

grid:
  symbol: "BTCUSDT"
  net_worth: 100000      # ใช้ baseline เดียวกับตัวอย่างทั้งเล่ม (Part 1-8) เพื่อให้เทียบตัวเลขกันได้
  kelly_fraction: 0.19  #  $f_{\text{safe}} = 25\% \times f^* \approx 19\%$  (ตาม Part 5 ตัวอย่าง  $f^*=0.754$ ) →  $\text{grid\_capital} \approx \$19,000$ 
  candle_interval: "1d" # เรือนวัด – ทุก indicator ด้านล่างคำนวณจากแท่งนี้ (Part 3 §3.0)
  donchian_period: 30
  atr_period: 14
  step_factor: 0.5      #  $\text{Step} = \text{step\_factor} \times \text{ATR}$ 

risk:
  dd_halt_threshold: -0.08 # -8% drawdown → HALT (Part 5)
  dd_resume_threshold: -0.04 # -4% → resume
  max_notional_per_level: 0.05 # 5% of capital (Part 5)

regime:
  hurst_on: 0.50          # Grid ON ถ้า  $H < 0.50$ 
  hurst_caution: 0.58    # CAUTION ถ้า  $H$  0.50-0.58
  adx_caution: 40        # BTC-calibrated (Part 4): ADX 40-50 → CAUTION
  adx_off: 50             # ADX > 50 → GRID OFF
  bb_width_caution: 0.04
  bb_width_off: 0.08

execution:
  style: "composite"      # mean_reversion / trend_adaptive / volume_adaptive / composite
  recheck_hours: 1        # ตรวจ regime ทุก 1 ชั่วโมง – เรือนตัดสินใจ (Part 3 §3.0)
  tp_policy: "one_step"   # ค่าเดียวที่ GridFramework รองรับตอนนี้ (§9.2) – ทางเลือกอื่นดู Part 2 §2.6.1

snowball:
  enabled: true
  type: 1                 # 1=simple compound, 2=triggered layer, 3=zone
```

```

upgrade
    dca_pct: 0.30          # 30% ของ profit ไป DCA

# === ไฟล์แยก: main.py ===

import yaml
import json
from core.grid_framework import GridFramework
from core.execution_styles import UnifiedGridController
from exchange.bybit_client import BybitClient
from exchange.data_feed import DataFeed
import time, logging

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s [%(levelname)s] %(message)s")

def main():
    # Load config
    with open("config/config.yaml") as f:
        cfg = yaml.safe_load(f)
    with open("config/secrets.yaml") as f:
        secrets = yaml.safe_load(f)
    # Alternative (recommended): load from env vars instead of
    secrets.yaml
    # api_key = os.environ["BYBIT_API_KEY"]
    # api_secret = os.environ["BYBIT_API_SECRET"]

    # Initialize
    client = BybitClient(secrets["api_key"], secrets["api_secret"],
testnet=True)
    data = DataFeed(client, cfg["grid"]["symbol"])

    # Get market data
    prices, highs, lows, volumes = data.get_ohlcw(days=60)

    # Build framework
    framework = GridFramework.from_market_data(
        symbol=cfg["grid"]["symbol"],
        prices=prices, highs=highs, lows=lows,
        net_worth=cfg["grid"]["net_worth"],
        kelly_fraction=cfg["grid"].get("kelly_fraction", 0.19)
    )
    logging.info(f"Grid: zone=[{framework.zone_low:.0f},
{framework.zone_high:.0f}] ")

```

```

        f"step={framework.step:.0f} Q={framework.base_q:.4f}
N={framework.n_levels}")

    # Controller – run_cycle() ทำครบในเรียกเดียว: DD Halt (Part 5) →
regime
    # (Part 4) → execution style (Part 7B) → order placement. ไม่ต้องมี
    # watchdog แยกมาห่อหุ้มอีกชั้น เพราะ watchdog ที่แยกออกไปมักลิมเช็ค DD Halt
    # และ execution style ครบทั้งคู่ – รวมไว้ในฟังก์ชันเดียวปลอดภัยกว่า
    controller = UnifiedGridController(framework,
style=cfg["execution"]["style"])

    # Main loop
    portfolio_history = [framework.grid_capital]

    while True:
        # ต้อง refresh ข้อมูลตลาดทุกรอบ ไม่ใช่ prices ก่อนเดิมจากตอน start
        prices, highs, lows, volumes = data.get_ohlcw(days=60)
        market_data = data.get_indicators(prices, highs, lows, volumes)
        market_data["current_price"] = prices[-1]
        market_data["portfolio_history"] = portfolio_history

        result = controller.run_cycle(market_data)
        logging.info(f"Action: {result['action']}")

        if result["action"] == "HALT":
            client.cancel_all_orders(cfg["grid"]["symbol"])
            logging.warning(f"HALTED: {result['reason']}")

        elif result["action"] == "PAUSE":
            logging.info(f"Paused (state={result['state']}) – ไม่เปิด
order ใหม่รอบนี้")

        elif result["action"] == "EXECUTE":
            for order in result["orders"]:
                resp = client.place_order(
                    symbol=cfg["grid"]["symbol"],
                    side=order.side,
                    qty=order.qty,
                    price=order.price
                )
                logging.info(f"Order placed: {resp}")

        # Update portfolio value
        balance = client.get_wallet_balance("USDT")

```

```

portfolio_history.append(balance)

# เขียน state ล่าสุดให้ dashboard (§9.8) อ่านได้
with open("state.json", "w") as f:
    json.dump({"action": result["action"], "portfolio_value":
balance,
                "n_levels": framework.n_levels}, f)

# ⚠️ Production Note: sleep(3600) จะพลาด order fills ระหว่างรอ
# ใช้ WebSocket private channel
(wss://stream.bybit.com/v5/private) แทน polling
# ดู Bybit API docs: subscribe to "order" channel เพื่อรับ fill
events real-time
# ⚠️ Production: implement exponential back-off reconnect –
connection drop = missed fills
# while True:
#     try:
#         ws.run_forever()
#     except ConnectionError:
#         time.sleep(min(2 ** attempt, 60)) # รอเพิ่มขึ้นเรื่อยๆ
สูงสุด 60s
#         attempt += 1
#         time.sleep(cfg["execution"]["recheck_hours"] * 3600)

if __name__ == "__main__":
    main()

```

9.6 Paper Trading → Live Migration

Phase	ระยะ เวลา	ทำอะไร	Success Criteria
Phase 0: Backtest	1 สัปดาห์	Run backtester บน 1–2 ปีย้อนหลัง	Sharpe > 1.5, Max DD < 15%
Phase 1: Paper Trade	2–4 สัปดาห์	Run bot บน Testnet (สภาพแวดล้อมทดสอบของ Bybit — API จริง แต่เงินจำลอง ตั้ง testnet=True)	ไม่มี bug, state machine ทำงาน
Phase 2: Small Live	1 เดือน	\$1,000 จริง บน Bybit	P&L $\approx$ backtest, DD $\geq$ -8% (ไม่ชน HALT)
Phase 3: Scale Up	ต่อเนื่อง	เพิ่ม capital หลังผ่าน Phase 2	Compound + DCA Stack

9.7 Pre-Live Checklist — สิ่งที่ต้องตรวจก่อน Live

ตรวจทุกข้อก่อน switch testnet=False

- API Rate Limit: Bybit อนุญาต 600 requests/นาที → grid 30 levels × 2 (place/cancel) = 60/cycle ✓
- Minimum Order Size: BTC บน Bybit ขั้นต่ำ 0.001 BTC → Q ต้องมากกว่านี้
- Price Precision: BTC round ถึง \$0.01 → step ต้องเป็น multiple ของ \$0.01
- Network Redundancy: รัน bot บน VPS (Virtual Private Server — เซิร์ฟเวอร์เช่าที่ online ตลอด 24/7) พร้อม reconnect logic ไม่ควรรัน bot บน local machine ที่อาจปิดหรือ internet หลุด
- Secret Management: API key เก็บใน environment variable (ค่าที่ตั้งไว้นอกโค้ด อ่านตอนรัน โปรแกรม) ไม่ใช่ hardcode (พิมพ์ค่าจริงติดอยู่ในซอร์สโค้ดตรงๆ) เพราะไฟล์โค้ดอาจหลุดเข้า git repo หรือถูกแฮกโดยไม่ตั้งใจ

9.8 Monitoring Dashboard

Flask คืออะไร

Flask เป็น web framework เล็กๆ ของ Python — ใช้เปิดหน้าเว็บง่ายๆ ที่รับ request แล้วตอบกลับเป็น HTML หรือ JSON ในที่นี้ใช้ทำ dashboard ดู status ของ bot ผ่านเบราว์เซอร์: โค้ดด้านล่างอ่าน state.json (ไฟล์ที่ main loop ใน §9.5 เขียนสถานะล่าสุดไว้ทุกรอบ หลังจากอัปเดต portfolio value) แล้วส่งกลับเป็น JSON ให้หน้าเว็บแสดงผล ส่วน HTML/JS ที่ฝังไว้ใน string เป็นแค่หน้าแสดงผลง่ายๆ ฝัง browser ไม่เกี่ยวกับ logic การเทรดใดๆ

```
# monitoring/dashboard.py (Flask simple dashboard)

from flask import Flask, jsonify, render_template_string
import json

app = Flask(__name__)

HTML = """
<!DOCTYPE html><html><head><title>Grid Dashboard</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script></head>
<body style="font-
family:monospace;padding:20px;background:#0f172a;color:#e2e8f0">
<h2>Grid Bot Dashboard</h2>
<div id="status">Loading...</div>
<canvas id="chart" width="800" height="300"></canvas>
<script>
fetch('/api/status').then(r=>r.json()).then(data=>{
  document.getElementById('status').innerHTML =
    '<pre>' + JSON.stringify(data, null, 2) + '</pre>';
});
</script>
</body></html>
"""

@app.route("/")
def dashboard():
    return render_template_string(HTML)
```

```

@app.route("/api/status")
def status():
    # อ่านจาก state file ที่ bot เขียนไว้
    try:
        with open("state.json") as f:
            return jsonify(json.load(f))
    except FileNotFoundError:
        return jsonify({"error": "Bot not running"})

# รัน: python -m monitoring.dashboard
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

สรุปเนื้อหาทั้งเล่ม — Cheat Sheet

GRID TRADING MASTERY – Quick Reference

TIMEFRAME: แท่งรายวัน (1D) เป็น default ทุก indicator ด้านล่าง (Part 3 §3.0)

เปลี่ยนแท่งต้อง derive สูตรใหม่เอง – ห้ามเอาตัวเลขจาก daily ไปใช้
ตรงๆ

TP: 1-step (default, ตายตัวทุกสูตรในเล่มนี้) | ทางเลือกอื่นดู Part 2 §2.6.1

ZONE: Donchian(30) $\pm$ 5% buffer | min_width $\geq$ 6×ATR

STEP: $\max(0.5 \times \text{ATR}(14), 3 \times \text{fee_per_cycle}) \rightarrow$ round to \$100

Q: $\text{grid_capital} / (N \times \text{avg_price}) \rightarrow$ ตรวจ 5% max notional

N: $\text{zone_width} / \text{step} \rightarrow$ ควรอยู่ระหว่าง 15-25

KELLY: $f^* = (p \times b - q) / b$ | $f_{\text{safe}} = 0.25 \times f^*$

$\text{grid_capital} = \text{net_worth} \times f_{\text{safe}}$

REGIME (Part 4, BTC-calibrated):

$H < 0.50 + \text{ADX} < 40 + \text{BB_width} < 4\% \rightarrow$ GRID ON (full deploy)

$H 0.50-0.58 + \text{ADX} 40-50 + \text{BB_width} 4-8\% \rightarrow$ CAUTION (50% size)

$H > 0.58 + \text{ADX} > 50 + \text{BB_width} > 8\% \rightarrow$ GRID OFF (pause)

RISK (Part 5):

$\text{DD} \leq -8\% \rightarrow$ HALT (cancel all, wait for recovery)

$\text{DD} \geq -4\% + H < 0.55 + 24\text{h cooloff} \rightarrow$ RESUME

SNOWBALL (Part 6B):

Type 1: $Q(t) = Q_0 \times (1 + r_{\text{daily}})^t$
DCA Stack: 30% profit → weekly BTC buy

COMPOSITE SCORE (Part 3B):

$30\% \times \text{RSI} + 30\% \times \text{BB} + 20\% \times \text{Vol} + 20\% \times \text{Hurst} \rightarrow 0-100$

>75: Full Deploy | 55-75: 60% | 35-55: 30% | <35: Wait

แบบฝึกหัด Part 9

- ▶ ข้อ 1: อ่าน config.yaml แล้วอธิบายว่า `kelly_fraction=0.19` หมายความว่าอย่างไร และ `net_worth=$100,000` ใน config ทำให้ `GridFramework.from_market_data()` คำนวณ `grid_capital` ได้เท่าไร?
- ▶ ข้อ 2: ใน `run_cycle()` ถ้า `DD = -9%` เกิดอะไรขึ้น? bot ยังคงทำงานไหม?
- ▶ ข้อ 3: ทำไม `testnet=True` ต้องถูกเปลี่ยนเป็น `False` ก่อน Live? มีความเสี่ยงอะไรถ้าลืม?
- ▶ ข้อ 4: Backtester ใน Part 9 มี "optimistic assumption" อะไรที่ทำให้ผลดีกว่าความเป็นจริง?

สูตร · Glossary · Cross-Book Index

ตารางสูตรทั้งหมด · คำศัพท์ไทย-อังกฤษ · ดัชนีข้ามเล่ม

A — สูตรหลัก (One-Page Cheat Sheet)

ต้องการ cheat sheet แบบสั้น เน้นใช้งานหน้างาน (operational, ใช้เวลา deploy จริง) ดู "สรุปเนื้อหาทั้งเล่ม" ท้าย Part 9 §9.8 — ส่วนนี้คือ reference สูตรฉบับเต็มพร้อมที่มาแต่ละสูตร

Grid Mechanics (Part 1A, 2, 3)

กำไรต่อรอบ = $Q \times (P_{\text{sell}} - P_{\text{buy}}) - \text{Fees}$ Capital ถึง Floor = $\sum_{i=1}^N Q_i \times P_i$ Soft Martingale = $Q_i = Q_0 \times r^{(i-1)}$, $r \in [1.1, 1.5]$
รอบต่อวัน (ประมาณ) = $\text{ATR}_{\text{daily}} / \text{Step}$ Grid step (ATR) = $k \times \text{ATR}_t$, $k \in [0.2, 1.0]$ Bell curve $Q_i \propto \exp(-(P_i - \mu)^2 / 2\sigma^2)$

Kelly Sizing (Part 5)

Kelly $f^* = (p \times b - q) / b$ Fractional Kelly = $0.25 \times f^*$
(conservative) Capital deployed = Fractional Kelly $\times$ total_capital
(ใช้ $0.25 \times f^*$ เสมอ ไม่ใช่ f^* เต็ม) p = win probability, b = win/loss ratio, $q = 1 - p$

Hurst Exponent (Part 4)

$\log(R/S) = H \times \log(N) + C$ $H < 0.5$: mean-reverting (Grid ดี) $H = 0.5$: random walk $H > 0.5$: trending (Grid หยุด)

Risk Management (Part 5)

Drawdown $DD_t = (P_{peak} - P_t) / P_{peak}$ (แสดงเป็นค่าบวก – ต่างจาก Part 9's config.yaml ที่เก็บเป็นค่าลบ เช่น `dd_halt_threshold=-0.08`; ทั้งสองหมายถึง "ขาดทุน 8% จาก peak" เหมือนกัน) $Max\ Notional \leq 5\% \text{ capital per grid level}$ Stop condition $DD > max_dd_threshold$ (e.g. 8% – ดู Part 5 §5.5)

Performance Metrics — คำเป้าหมายสำหรับ Grid Bot (Part 5, 9)

Metric	สูตร	เป้าหมาย Grid Bot
Sharpe Ratio	$(Return - R_f) / \sigma_{annual}$	> 1.0 (ผ่าน) · > 1.5 (ดี) · > 2.0 (ยอดเยี่ยม)
Sortino Ratio	$(Return - R_f) / \sigma_{downside}$	> 1.5 (grid มี asymmetric loss — Sortino สูงกว่า Sharpe)
Calmar Ratio	$Annual\ Return / Max\ DD $	> 2.0 (เช่น 20% return / 10% max DD)
Max Drawdown	$(Peak - Trough) / Peak$	< 10% (conservative) · < 15% (moderate)
Win Rate (grid cycle)	Cycles ที่ TP hit / Total fills	> 60% ใน mean-reverting regime ($H < 0.5$)
Profit Factor	Total wins / Total losses	> 2.0 (Close System ไม่มี hard loss → PF สูงตามธรรมชาติ)

B — Glossary (ไทย-อังกฤษ)

คำศัพท์ (ไทย)	English	ความหมาย
กริด	Grid	กรอบราคาที่จัด order ไว้เป็นระดับ
ขั้น/ก้าว	Step	ระยะห่างระหว่าง grid level
วงรอบ	Cycle	การ BUY 1 ครั้ง + SELL 1 ครั้ง ที่ระดับถัดไป
โซนใช้งาน	Active Zone	ส่วนของ capital ที่ deploy เป็น order
โซนสำรอง	Standby Zone	ทุนที่รอ deploy + ฝากรับดอกเบี้ย
โซนพื้น	Floor Zone	ทุนสำรองสุดท้าย ไม่แตะยกเว้น worst case
ระบบปิด	Close System	วาง capital ครบถึง Floor Zone (ด้านบน) ก่อนเปิด grid — ไม่มีการเติมทุนเพิ่มระหว่างทาง
บวมบน	Top-Heavy	inventory สะสมมากในโซนราคาสูง
บวมล่าง	Bottom-Heavy	inventory สะสมมากในโซนราคาต่ำ
บวมกลาง	Center-Heavy / Bell	inventory หนาแน่นรอบราคาเฉลี่ย (mean)
ลูกหิมะ	Snowball	การ compound กำไร grid เพิ่ม capital
มาร์ติงเกล	Martingale	กลยุทธ์เพิ่ม size เป็น 2 เท่าทุกครั้งที่ขาดทุน ($Q_i = Q_0 \times 2^i$) เพื่อเฉลี่ยต้นทุนลง — เสี่ยงสูงเพราะทุนโตแบบทวีคูณ ไม่มีเพดาน
ซอฟต์มาร์ติงเกล	Soft Martingale	เวอร์ชันเบาของ Martingale (ด้านบน) — เพิ่ม size ด้วย $r \in [1.1, 1.5]$ แทน 2x ตายตัว จำกัดความเสี่ยงทวีคูณ
ต้นทุนเฉลี่ยถ่วงน้ำหนัก	VWAP (Volume-Weighted Average Price)	ต้นทุนเฉลี่ยของทุกไม้ที่ถือ โดยถ่วงน้ำหนักด้วยขนาดแต่ละไม้ $= \sum(Q_i \times P_i) / \sum Q_i$ — ต่ำกว่าไม้แรกเสมอเมื่อถัวขายลง
ทีพีรวบทุกไม้	Basket TP	TP เดียวคำนวณจาก VWAP (ด้านบน) + $k \times \text{ATR}$ แล้วปิดทุกไม้พร้อมกัน — โหมดกู่สถานการณ์ ดู Part 2 §2.6.1
เลขชี้กำลัง Hurst	Hurst Exponent	ตัวเลขวัดว่าราคาย้อนกลับหาค่าเฉลี่ย (mean-reverting) หรือวิ่งเป็นแนวโน้ม (trending) โดยดูจากชุดข้อมูลราคาย้อนหลัง (time series)
ระบอบตลาด	Regime	สภาวะตลาด: mean-reverting, trending, volatile

อัตราส่วนเคลลี	Kelly Fraction	สัดส่วน capital ที่ optimal ตาม win%/odds
ดัชนีทิศทางเฉลี่ย	ADX (Average Directional Index)	ตัววัดความแรงของแนวโน้ม (ไม่บอกทิศทาง) — ค่าสูง = trend แรง, ค่าต่ำ = sideways/range
ช่วงเฉลี่ยที่แท้จริง	ATR (Average True Range)	ค่าเฉลี่ยความผันผวนของราคาต่อแท่ง ใช้กำหนดระยะ Step ของ grid
ช่องดอนเซียน	Donchian Channel	กรอบราคาจาก highest-high/lowest-low ในช่วง N แท่งที่ผ่านมา ใช้กำหนดขอบ Zone
ช่องเคลท์เนอร์	Keltner Channel	กรอบราคาจาก $EMA \pm k \times ATR$ ใช้เป็นทางเลือกกำหนดขอบ Zone แทน Donchian
สมการถดถอยเชิงเส้น	Linear Regression (OLS)	วิธีหาเส้นตรงที่ fit ข้อมูลได้ดีที่สุด — ความชัน (slope) ของเส้นใช้วัด drift ของราคา (Part 4 §4.2.1) และหา hedge ratio ของ pair (Part 6)
อันดับ IV	IVR (IV Rank)	ตำแหน่งของ Implied Volatility ปัจจุบันเทียบกับช่วง 1 ปีที่ผ่านมา (0-100%) — สูง = vol แพง
อัตราส่วนชาร์ป	Sharpe Ratio	ผลตอบแทนส่วนเกิน (เหนือ risk-free rate) หารด้วยความผันผวนรวม — วัด return ต่อความเสี่ยง
อัตราส่วนซอร์ติโน	Sortino Ratio	เหมือน Sharpe แต่หารด้วยความผันผวนขาลงเท่านั้น (downside deviation) — เหมาะกับกลยุทธ์ที่ risk ไม่สมมาตร เช่น grid
อัตราส่วนคาลมาร์	Calmar Ratio	ผลตอบแทนต่อปี หารด้วย Max Drawdown — วัด return ต่อความเสี่ยงขาดทุนสูงสุด
ตัวประกอบกำไร	Profit Factor	ผลรวมกำไรทั้งหมด หารด้วยผลรวมขาดทุนทั้งหมด — มากกว่า 1 คือกำไรสุทธิ
อัตราปลอดความเสี่ยง	Rf (Risk-Free Rate)	ผลตอบแทนของสินทรัพย์ที่ถือว่าไม่มีความเสี่ยง (เช่น พันธบัตรรัฐบาล) ใช้เป็น baseline ในสูตร Sharpe/Sortino

C — Cross-Book Index

Grid Concept	Grid Part	เล่มที่เชื่อมต่อ	บท/ตอน	การเชื่อมต่อ
Mean-reversion	Part 4	Statistical Arbitrage	Ch.1–3 (OU Process — Ornstein-Uhlenbeck, โมเดล stochastic ที่ราคาถูกดึงกลับหาค่าเฉลี่ย)	Grid สมมติ OU reversion
Hurst Exponent	Part 4	Statistical Arbitrage	Ch.6	ตัดสินใจ grid on/off
Cointegration	Part 6	Statistical Arbitrage	Ch.5	Pair grid ต้องการ cointegrated
Kelly Criterion	Part 5	Statistical Arbitrage	Ch.12	Sizing per grid level
Regime Detection	Part 4	Statistical Arbitrage	Ch.9	Switch grid on/off
GARCH Vol	Part 3	Statistical Arbitrage	Ch.7	Predict step size
IV, VIX, ATR	Part 3, 7	Volatility Mastery	Part 1–3	IV-adaptive step size
Vol Skew	Part 7	Volatility Mastery	Part 4	Asymmetric zone sizing
Funding Rate	Part 1C	Arbitrage	Part 2A	Spot-perp grid hedge
Cash-Carry Arb	Part 1C	Arbitrage	Part 2B	Grid Hedge structure
CSP Payoff (Cash-Secured Put — payoff diagram ของการขาย PUT ที่มีเงินสดค้ำประกันเต็มจำนวน)	Part 7	Payoff Mastery	Part 3A	Options-as-Grid Level

Strangle	Part 7	Payoff Mastery	Part 3B	Short strangle ที่ชอบ zone
Normal Distribution	Part 3	คณิตศาสตร์	Part 1 (Statistics)	Bell curve zone density
OLS Regression	Part 4 §4.2.1, Part 6	คณิตศาสตร์	Part 5 (Regression)	Hedge ratio สำหรับ pair grid + slope filter วัด drift